

사용자 매뉴얼



작성자: hcchoi@spirallab.co.kr

샘플 코드 및 GitHub 저장소: <https://github.com/labspiral/sirius3>

문서 이력

2026.1.26

버전 1.2.7 기반

계측(Measurement) 과 ALC(Automatic Laser Control) 사용에 대한 다양한 예제 추가
개체(EntityPen) 및 레이어(EntityLayerPen) 펜 예제 추가

2026.1.7

버전 1.2.4 기반

2025.12.22

버전 1.0.0 기반

단축키 및 주요 개체 페이지 추가

2025.12.5

버전 0.9.3 기반

초안 작성

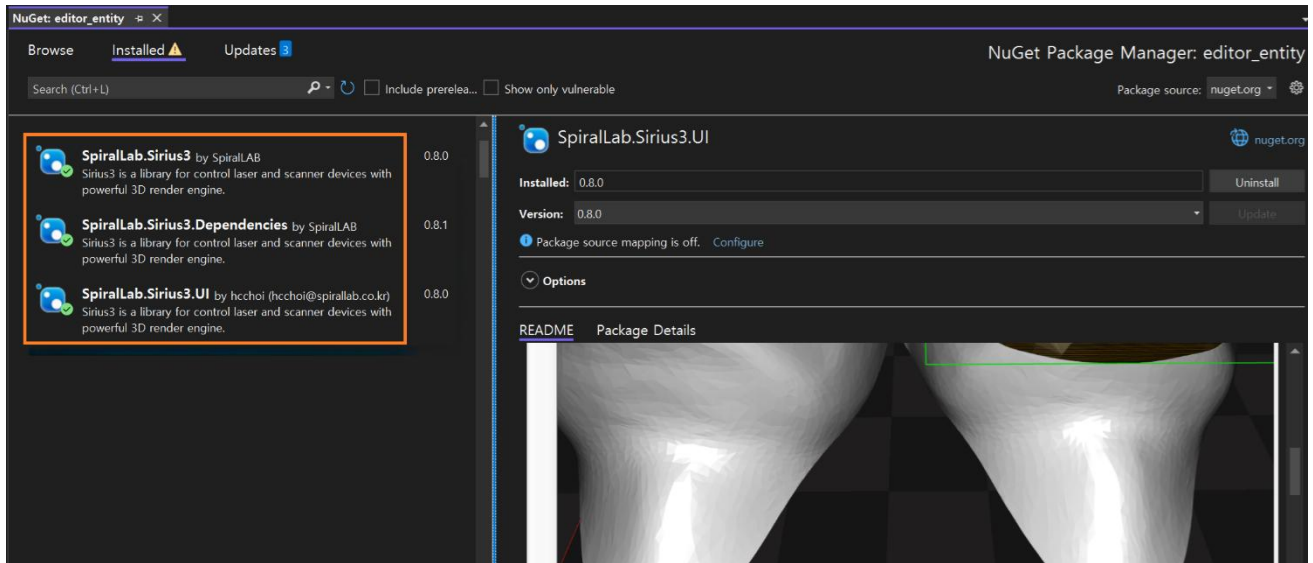
목차

1. 라이브러리 설치	5
2. 라이브러리 초기화 (Pseudo code)	7
3. 주요 특징 (Main features)	8
4. 문서(Document) 와 기능들	10
5. 편집기(Editor)	12
6. 기본 개체들(Primitive Entities)	15
6.1 기본 도형 엔티티 (Primitive Entities)	15
6.2 3D 메쉬 엔티티 (3D Mesh Entities)	17
6.3 컨테이너 엔티티 (Container Entities)	17
6.4 제어 엔티티 (Control Entities)	17
7. 펜 속성 (Pen Properties)	19
7.1 레이어 펜(Layer Pen) 속성 항목	21
7.2 개체 펜(Entity Pen) 속성 항목	23
7.3 펜(EntityPen, EntityLayerPen) 생성시 초기화 처리	25
8. 계측 (Measurement)	26
8.1 레이어 펜(Layer Pen) 속성의 Skywriting 기능을 사용한 결과	28
8.2 개체 펜(Entity Pen) 속성의 Wobbel 기능을 사용한 결과	29
8.3 Defined Vector 기반 자동 레이저 제어 (Automatic Laser Control)	30
8.4 Speed Dependent 기반 자동 레이저 제어 (Automatic Laser Control)	31
8.5 Position Dependent 기반 자동 레이저 제어 (Automatic Laser Control)	33
9. 마커(Marker)	34
9.1 MarkerRtc 객체	34
10. 수동(Manual)	37
11. 스캐너(Scanner)	38
12. 레이저(Laser) 소스	39
12.1 레이저 소스 제어부 구현	39

12.2 레이저 소스 UI 구현	39
13. 디지털 입출력 (DIO)	41
14. 파워미터(PowerMeter)와 파워맵(PowerMap)	42
15. 해치(Hatch)	45
15.1 선분 해치	45
15.2 폴리곤 해치	49
15.3 코드를 통한 해치 처리	52
16. 슬라이서 (Slicer)	54
17. 텍스트 데이터 변환 (Text Converter)	56
17.1 이벤트 (Event)	56
17.2 단순 스크립트 (Simple Script)	57
17.3 파일 (File)	57
17.4 오프셋 (Offset)	58
18. 마킹 온더 플라이(Marking On the Fly)	59
19. 그룹(Group) 및 블록(Block)	61
20. 웨이퍼(Wafer), 기판(Substrate) 맵(Map)	63
21. 파일 가져오기(Import)	64
22. syncAXIS (XL-SCAN) 사용법	65
23. 환경 설정(Config)	69
24. 편집기 단축키(Shortcut)	70

1. 라이브러리 설치

시리우스 3 라이브러리는 마이크로 소프트의 NuGet 패키지 설치 관리자를 이용해 최신 버전의 라이브러리 설치 및 업데이트가 지원됩니다.



라이브러리는 총 3 개의 DLL 파일로 구성되어 있으며 각각 다음과 같은 기능을 제공합니다.

- **SpiralLab.Sirius3.Dependencies**
 - SCANLAB RTC4/5/6 관련 프로그램 및 DLL 파일들
 - syncAXIS 관련 프로그램 및 DLL 파일들
 - 사용자 폰트 및 샘플 파일들
 - 런타임에 필요한 필수 파일들
- **SpiralLab.Sirius3**
 - 스캐너, 레이저, 파워미터와 같은 제어 장치들에 대한 구현부
- **SpiralLab.Sirius3.UI**
 - 문서 및 기본 개체, 가공용 개체, 제어용 개체 제공
 - 셰이더 및 3D 렌더링 엔진 제공
 - 원폼(WinForms) 기반 컨트롤 제공

지원 플랫폼

- .NET Framework 4.8.1 이상
- .NET8.0-windows 이상
- (기타) 윈도우 10.11 이상, 64 비트 환경, OpenGL 3.3 이상 드라이버 설치

사용자 프로젝트 설정

- NuGet 관리자를 통해 3 개의 DLL 이 참조되도록 설치(혹은 업데이트) 합니다.
- 기타 의존성 있는 DLL 파일들(Microsoft.Extensions.Logging, OpenTK, Newtonsoft.Json 등)을 참조합니다.
- 스캐너(IScanner), 레이저 소스(ILaser), DIO, 파워미터(IPowerMeter), 마커(IMaker) 등의 제어 객체를 생성 및 초기화 합니다.
- SiriusEditorControl 사용자 컨트롤을 생성시키고, 제어 객체를 서로 연결(Assign) 합니다.

(참고) 인증 정보가 포함된 USB 동글키가 없을 경우 최대 30 분 동안 평가판 모드(Evaluation Copy Mode)로 실행되며, 일부 기능의 사용이 제한될 수 있습니다.

2. 라이브러리 초기화 (Pseudo code)

```

// Initialize sirius3 library
SpiralLab.Sirius3.Core.Initialize();

// Create scanner device
var fov = 100.0; // 100mm
var kfactor = Math.Pow(2, 20) / fov; // 20 bits resolution
var rtc = ScannerFactory.CreateRtc5(0, kfactor, Yag1, ActiveHigh, ActiveHigh,
@"C:\...\cor_1to1.ct5");
rtc.Initialize();
rtc.CtlFrequency(50_000, 2); // default frequency, pulse width: 50KHz, 2usec
rtc.CtlSpeed(500, 500); // default jump, mark speed: 500mm/s

// Create D.IO devices
var dInExt1 = IOFactory.CreateInputExtension1(rtc);
dInExt1.Initialize();
var dOutExt1 = IOFactory.CreateOutputExtension1(rtc);
dInExt1.Initialize();
// and mores ...

// Create laser source device
var laser = LaserFactory.CreateVirtualDutyCycle(0, 20.0, 0, 99);
laser.Scanner = rtc;
laser.Initialize();
laser.CtlPower(20 * 0.1);

// Create powermeter and power map
var powerMeter = PowerMeterFactory.Create...;
powerMeter.Initialize();
var powerMap = PowerMapFactory.Create...;
laser.PowerMap = powerMap;

// Create marker controller
var marker = MarkerFactory.CreateRtc(0);
marker.Initialize();

// Assign devices into editor
siriusEditorControl.Scanner = rtc;
siriusEditorControl.Laser = laser;
siriusEditorControl.PowerMeter = powerMeter;
siriusEditorControl.Marker = marker;

// Do marker as ready status
marker.Ready(siriusEditorControl.Document, siriusEditorControl.View, rtc, laser,
powerMeter);

// Your application is running ...

// Clean-up sirius3 library
SpiralLab.Sirius3.Core.Cleanup();

```

3. 주요 특징 (Main features)

3.1 플랫폼 개요

통합 환경: 스캐너, 레이저, 파워미터, UI 렌더링 엔진을 단일 라이브러리로 통합

고성능 렌더링: OpenGL 3.3/4.x 기반의 고속 3D 뷰어 및 셰이더 엔진 탑재

유연한 확장성: 사용자 정의 Entity 및 오픈 아키텍처 지원 (Nuget 배포)

3.2 강력한 하드웨어 지원 (Hardware Abstraction)

* SCANLAB RTC 제어기:

* RTC4, RTC5, RTC6, RTC6e 지원

* syncAXIS (XL-SCAN) 제어 통합

* 다양한 레이저 소스:

* IPG (YLP), Coherent (Avia, Diamond), JPT, SPI, Photonics Industry 등

* 범용 아날로그/디지털 제어 및 주파수/Duty 가변 제어

* 계측 장비:

* Coherent, Ophir, Thorlabs, Gentec-EO 파워미터 연동

* 실시간 파워 모니터링 및 PowerMap 기반 출력 보정

3.3 고급 제어 기능:

* MoF (Marking on the Fly): 엔코더 연동 이동체 마킹 (RTC5/6)

* SkyWriting: 레이저 On/Off 지연에 따른 코너 품질 향상 (Mode 1~4 지원)

* 3D 제어: 3 축 스캐너를 이용한 3D 곡면 매핑 및 Z 축 동적 포커싱

* Wobble: 다양한 형상(원형, 8자 등)의 빔 회전 가공

* 정밀 보정 (Correction):

* 2D/3D 필드 보정 파일 (.ct5, .ctb) 로드 및 관리

* 행렬 기반 좌표 변환 (Matrix Stack) 지원

* 레이어 및 개체 펜(Pen) 파라미터를 통한 다양한 가공 옵션 제어 지원

3.4 방대한 가공 도형 (Entities)

* 기본 도형: 점, 선, 호(Arc), 사각형, 원, 스플라인(Bezier, Cubic 등)

* 특수 나선(Spiral) 18 종:

- * Archimedean, Logarithmic, Golden, Euler(Clothoid), Rose Curve 등
- * 정밀 패턴 가공 및 텍스처링에 최적화
- * 고급 객체:
 - * 텍스트: TrueType 폰트, 단선(Stroke) 폰트, 바코드(QR, DM, PDF417)
 - * 이미지: 래스터 마킹, 디더링 지원
 - * 3D 메쉬: PLY, STL, OBJ 파일 로드 및 슬라이싱(Slicing) 지원
- * 편집 기능: 블록(Block), 그룹, 레이어 관리, Undo/Redo

3.5 최적화 및 해칭 알고리즘

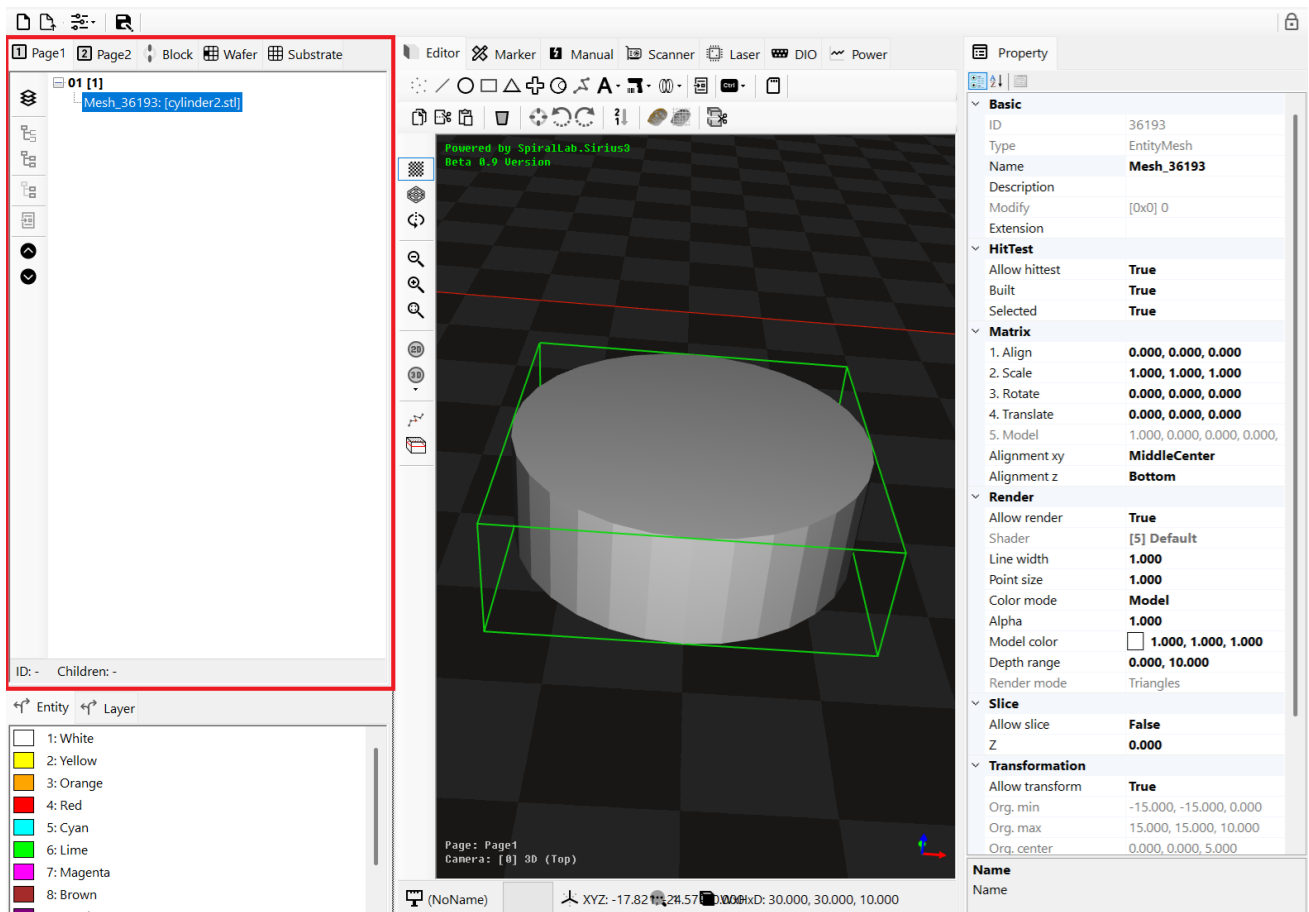
- * 스마트 해칭 (Hatching):
 - * Line, Polygon 해칭 조합 지원
 - * 알고리즘: Nearest(최단거리), Global(최적경로) 정렬 지원
 - * Zigzag 경로 생성, 반복 가공, 해칭 우선 가공 등 지원
- * 경로 최적화 (Path Optimizer):
 - * 가공 시간 단축을 위한 점프 경로 최소화 (TSP 유사 알고리즘) 지원

4. 문서(Document) 와 기능들

저장 및 불러오기를 이용해 처리되는 문서(IDocument)는 내부에 실제 내부 데이터(IDocumentData) 객체로 관리됩니다. 이 내부 데이터 객체는 아래와 같이 다양한 아이템으로 구성되어 있습니다.

IDocument 내부에 실제 IDocumentData 데이터를 구성하는 목록

- **Blocks:** 개체들을 묶어 Block 개체를 만들어 생성되는 목록으로, 추후 BlockInsert 로 사용됩니다.
- **Pages:** 최대 4 개의 Page 가 지원되며¹, Page 별로 서로 다른 데이터 편집, 가공 등이 가능합니다.
- **EntityPens:** 마커(IMarker) 가공시 개별 개체(Entity)에 적용되는 개체용 펜 목록
- **LayerPens:** 마커(IMarker) 가공시 개별 레이어(Layer)에 적용되는 레이어용 펜 목록
- **Wafers:** 웨이퍼 맵 항목
- **Substrates:** 기판 맵 항목

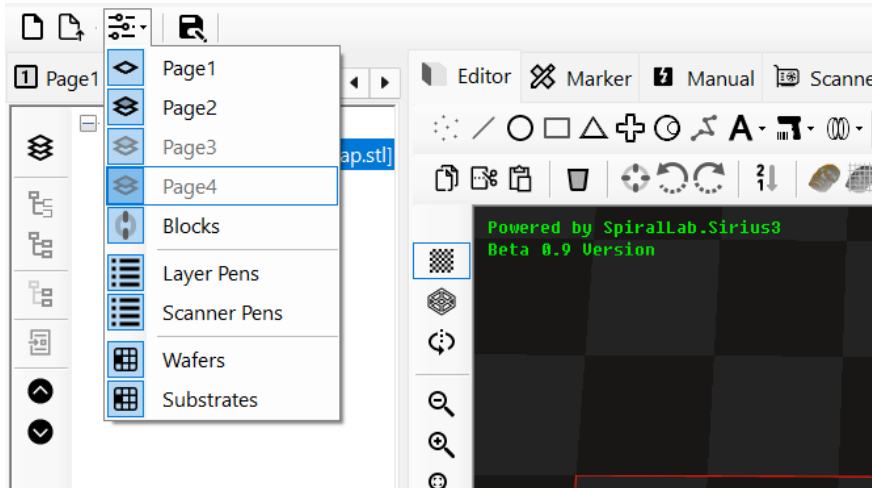


편집기 화면에서 좌측 탭에 IDocumentData 의 하위 항목들이 트리 컨트롤과 리스트 컨트롤 형태로 출력되며, 탭 변경(혹은 이동)시 활성화 페이지(Page) 변경을 통해 편집기(Editor)의 출력 내용도 같이 업데이트 됩니다.

¹ 기본적으로 2개가 지원되며, 최대 4개까지 확장을 원할 경우 공개된 SiriusEditorControl 사용자 컨트롤의 탭(TabControl) 관련 코드를 변경해야 합니다.

- **Save (문서저장):** IDocumentData 의 하위 항목들이 모두 저장됩니다. 이때 파일 크기를 줄이기 위해 기본적으로 압축되어 저장됩니다. 만약 사용자가 저장된 데이터 포맷을 읽어 처리하고자 한다면 Config.IsCompressedFileFormat 를 False 로 설정해 압축 기능을 사용하지 않도록 할 수 있습니다.
(참고) IDocument.ActSave 함수를 통해 파일 저장이 지원됩니다.

- **New (새 문서) 및 Open (문서 열기)**

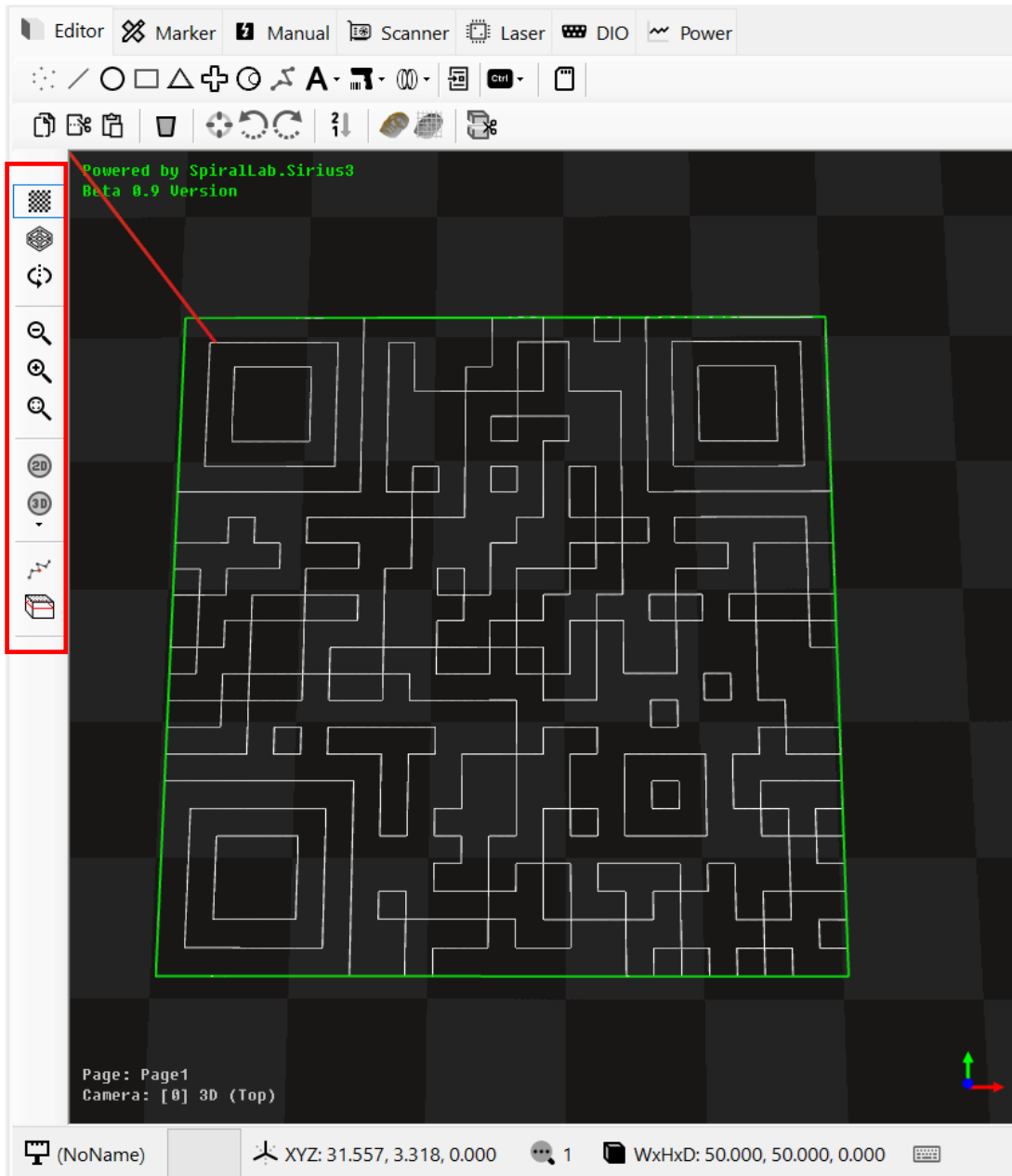


새 문서(New) 혹은 문서 열기(Open)를 하게 되면 기존 데이터는 모두(삭제 혹은 변경) 됩니다. 만약 사용자가 특정 아이템이 변경되는 것을 막고자 하는 특별한 경우에는 위와 같이 아이템 옵션 항목들에 대해 선택해 처리가 가능합니다.

(참고) IDocument.ActNew 및 IDocument.ActOpen 함수를 통해 새 파일 및 파일 열기가 지원됩니다.

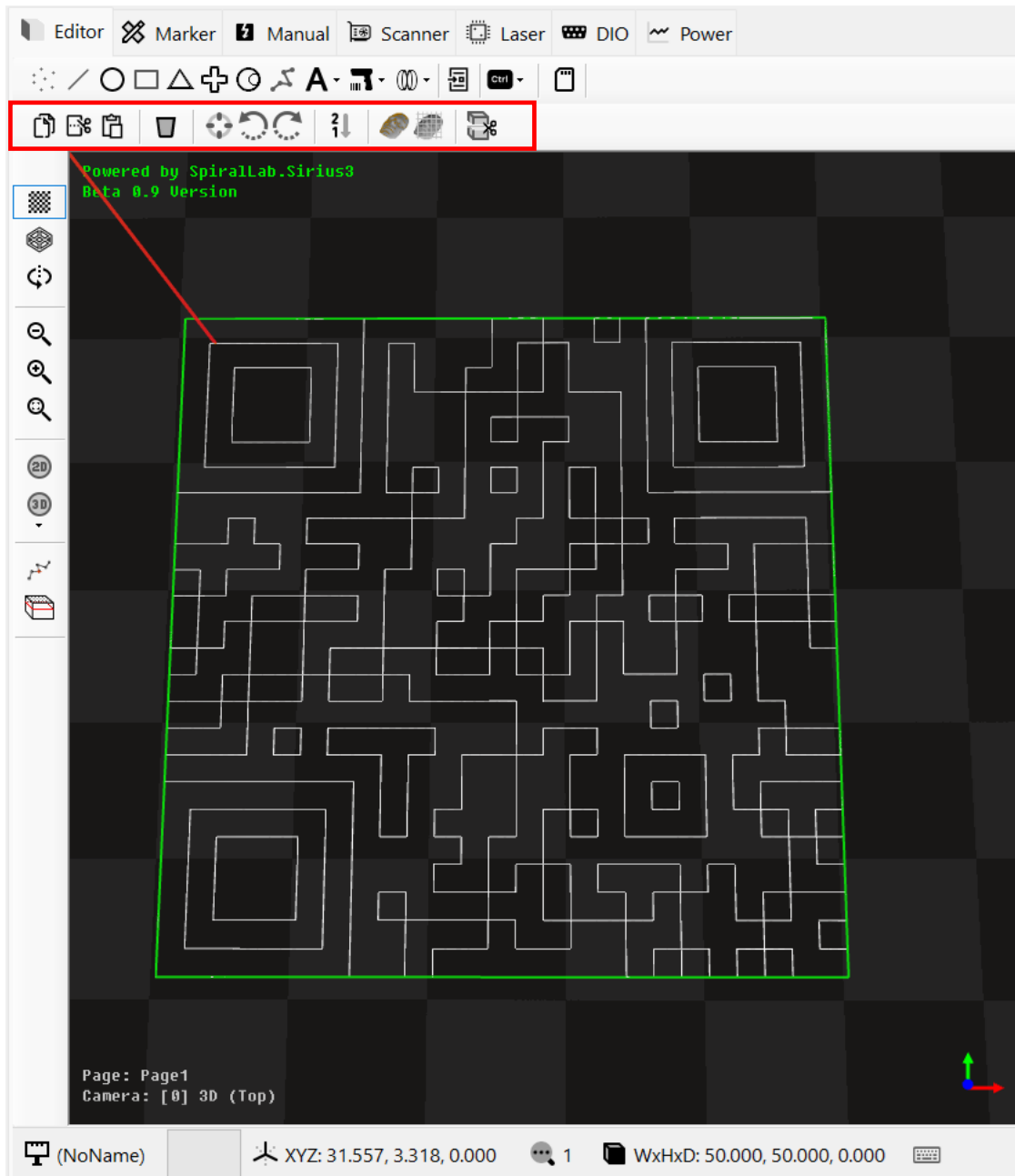
5. 편집기(Editor)

편집기에서는 개체의 생성, 선택, 삭제, 편집 등 모든 데이터 조작이 가능합니다. 또한 확대, 축소, 뷰 카메라 시점 변경, 레이저 가공 시뮬레이션 등의 부가기능도 지원됩니다.



- CheckerBoard/Grid: $Z = 0$ 평면에 대해 격자 혹은 체커보드 로 배경을 렌더링 합니다.
- Polygon Mode: 폴리곤을 점, 선, 면으로 렌더링 처리합니다.
- Rotating: 카메라를 자동으로 회전시키며 렌더링 합니다. (개체 선택시 개체를 회전 중심으로 사용)
- Zoom Out, Zoom In, Zoom Fit: 카메라 축소, 확대, 맞추기를 처리합니다.
- Camera 2D(Ortho): 편집 화면을 2D 카메라, 즉 직교 좌표계로 설정합니다.
- Camera 3D(Perspective): 편집 화면을 3D 카메라로 설정합니다. 또한 상단(Top), 앞면(Front), Right(우측면), Rear(후면), Left(좌측면) 순서로 변경합니다.

- **Simulation:** 선택된 개체에 대해 레이저 가공 시뮬레이션을 시작합니다.
단축키: F1(고속으로 시작), CTRL+ F1(중속으로 시작), CTRL+ALT+F1(저속으로 시작), ESC(중지)
- **Preview:** 선택된 개체에 대해 지시빔(Guide Beam)을 이용해 외곽 영역에 대해 미리 보기를 시작합니다.
단축키: F4 (지시빔 시작), F6(마커 중지)
(참고) 구현된 레이저 소스(ILaser)객체가 ILaserGuideControl 인터페이스를 상속 구현되어 있어야 합니다.
(참고) 내부적으로 IMarker.Preview 함수가 호출되어 실행됩니다.
- 추가적인 단축키(Shortcut)는 편집기 하단에 키보드 아이콘을 눌러 확인해 주시기 바랍니다.



- **Copy:** 선택된 개체(들)을 내부 클립보드로 복사합니다.
(참고) IDocument.ActCopy 함수가 사용됩니다.

- **Cut**: 선택된 개체(들)을 내부 클립보드로 잘라내기 합니다. 원본 개체(들)은 삭제됩니다. (참고) `IDocument.ActCut` 함수가 사용됩니다.
- **Paste**: 클립보드에 있는 개체(들)을 현재 마우스 위치로 붙여넣기 합니다. (참고) `IDocument.ActPaste` 함수가 사용됩니다.
- **Delete**: 선택된 개체(들)을 삭제합니다.
- **Align to Center**: 선택된 개체(들)을 원점을 중심으로 정렬되도록 이동시킵니다.
- **CCW**: 선택된 개체(들)을 반시계 방향으로 회전시킵니다. (회전 중심 변경시 **Align** 위치 사용가능)
- **CW**: 선택된 개체(들)을 시계 방향으로 회전시킵니다. (회전 중심 변경시 **Align** 위치 사용가능)
- **Reverse**: 좌표의 순서를 반전시킵니다.
- **PointCloud**: 선택된 3D 메쉬 개체의 정점들과 방향벡터 $(0, 0, -1)$ 가 이루는 각도가 $-90 \sim 90$ 이내의 모든 정점을 추출합니다. (참고) 카메라 Z 상단에서 바라봤을 때 보여지는 표면 정점들을 추출하는 것을 의미합니다.
- **GridCloud**: 선택된 3D 메쉬 개체에 대해 고정된 격자 간격(`Config.GridCloudInterval`)과 교차되는 모든 정점을 추출합니다. (참고) 카메라 Z 상단에서 바라봤을 때 표면을 고정된 격자 간격으로 교차되는 정점들을 추출하는 것을 의미합니다.
- **Slice**: 선택된 3D 메쉬 개체에 대해 지정된 Z 높이(`SliceZ`) 위치에서 절단하고 이후 생성되는 경로(`Contours`)들을 모아 2차원 폴리라인(`Polyline2D`)을 생성하는 기능입니다. (참고) 해당 폴리라인 개체들을 모아 **Mixed Group** 개체로 생성됩니다.

6. 기본 개체들(Primitive Entities)

6.1 기본 도형 엔티티 (Primitive Entities)

메서드 이름	생성 객체	처리 데이터 및 역할	주요 속성/함수
CreateLine	EntityLine	선분(Line Segment): 두 지점을 연결하는 직선. 2D/3D 좌표 처리	Start (시작점), End (끝점)
CreateLines	EntityLines	선분 집합(Lines): 여러 선분을 효율적으로 처리. 정점 목록 입력	Vertices (정점 배열)
CreatePoint	EntityPoints	점(Point): 단일 위치. 레이저 샷이나 특정 위치 이동용	Location (위치 좌표)
CreatePoints	EntityPoints	점 집합(Points): 여러 점 데이터를 하나의 엔티티로 관리	Locations (점 위치 배열)
CreateArc	EntityArc	호(Arc) 또는 원(Circle): 중심점, 반지름, 각도로 정의	Center, Radius, StartAngle, SweepAngle
CreateEllipse	EntityEllipse	타원(Ellipse): 중심점, 장축, 단축값으로 정의	Center, Major, Minor
CreateRectangle	EntityRectangle	직사각형(Rectangle): 중심점과 너비, 높이로 정의	Center, Width, Height
CreateTriangle	EntityTriangle	삼각형(Triangle): 밑변 중심, 길이, 높이로 정의	BaseCenter, BaseLength, Height
CreateCross	EntityCross	십자 마크(Cross): 정렬 마크용 십자 형태. 두께 설정 가능	Center, Width, Height, Thickness
CreateTrepan	EntityTrepan	트레판(Trepan): 도넛/링 가공용 나선형 경로	Center, InnerDiameter, OuterDiameter, Revolutions
CreateSpiral	EntitySpiral	나선(Spiral): 아르키메데스 나선 등 지원. 깊이 변화 가능	Center, Diameter, Depth, Turns, Type
CreateSpiralClassic	EntitySpiralClassic	나선(Classic): 시작 지름과 끝 지름이 다른 나선	StartDiameter, EndDiameter, Revolutions
CreateLissajous	EntityLissajous	리사주(Lissajous): 리사주 곡선 패턴. 위블링 가공용	Center, Diameter, Turns, LissajousType, Direction
CreatePolyline3D	EntityPolyline3D	3D 폴리라인: 3 차원 좌표를 잇는 연속된 선	Vertices, IsClosed

CreatePolyline2D	EntityPolyline2D	2D 폴리라인: 2 차원 좌표를 잇는 연속된 선	Vertices, IsClosed
CreateBezierSpline	EntityBezierSpline	베지어 스플라인: 제어점을 통한 곡선 생성 (2 차/3 차)	ControlPoints (제어점)
CreateCatmullRomSpline	EntityCatmullRomSpline	Catmull-Rom 스플라인: 모든 제어점을 통과하는 곡선	ControlPoints, IsClosed
CreateHermiteSpline	EntityHermiteSpline	Hermite 스플라인: 제어점과 접선 벡터로 정의	ControlPoints, Tangents
CreateBSpline	EntityBSpline	B-Spline: 매듭 벡터와 차수를 사용하는 스플라인	ControlPoints, Knots, Degree
CreateNURBSpline	EntityNURBSpline	NURBS: 가중치를 포함하는 비균일 유리 B-스플라인	ControlPoints, Knots, Weights, Degree
CreateImage	EntityImage	래스터 이미지: 비트맵 파일을 레이저 가공용으로 변환	FileName, Bitmap, Width, Height
CreateImageZPL	EntityImageZPL	ZPL 이미지: Zebra 프린터 언어 텍스트를 이미지로 변환	ZplText, DotsPerMM
CreateImageText	EntityImageText	이미지 텍스트: 윈도우 폰트를 래스터 이미지로 변환	FontName, Text, Style, IsFill
CreateText	EntityText	벡터 텍스트: 윈도우 폰트 윤곽선을 벡터 경로로 추출	FontName, Text, Style
CreateCircularText	EntityCircularText	원형 텍스트: 원호 경로를 따라 배치되는 텍스트	Radius, StartAngle, Text
CreateSiriusText	EntitySiriusText	Sirius 텍스트: 전용 단선 폰트(CXF/LFF). 고속 마킹용	FontName, Text, LetterSpace
CreateDataMatrix	EntityDataMatrix	2D 바코드(DataMatrix): 점, 선, 원, 사각형 등 셀 모양 설정 가능	Text, Cell, Width, Height
CreateQRCode	EntityQRCode	2D 바코드(QR Code): 점, 선, 원, 사각형 등 셀 모양 설정 가능	Text, Cell, Width, Height
CreatePDF417	EntityPDF417	2D 바코드(PDF417): 점, 선, 원, 사각형 등 셀 모양 설정 가능	Text, Cell, Width, Height

CreateBarcode	EntityBarcode1D	1D 바코드: Code128, Code39 등 일반 바코드	Text, Format, Width, Height

6.2 3D 메쉬 엔티티 (3D Mesh Entities)

CreateMesh	EntityMesh	메쉬 모델: 외부 3D 파일(.stl, .ply, .obj) 로드	Load(fileName)
CreateCube	EntityCube	육면체: 너비, 높이, 깊이를 가진 육면체 메쉬	Width, Height, Depth
CreateCylinder	EntityCylinder	원기둥: 반지름과 높이를 가진 원기둥 메쉬	Radius, Depth, Segments
CreateSphere	EntitySphere	구(Sphere): 반지름을 가진 구 형태 메쉬	Radius, Segments
CreateGridCloud	EntityGrids	그리드 클라우드: 격자 형태 점 구름 데이터	Rows, Cols, Interval, ZDepths

6.3 컨테이너 엔티티 (Container Entities)

CreateLayer	EntityLayer	레이어(Layer): 엔티티 계층 관리 컨테이너	Name, Children
CreateBlock	EntityBlock	블록 정의: 재사용 가능한 엔티티 집합 정의	BlockName, Entities
CreateBlockInsert	EntityBlockInsert	블록 참조: 정의된 블록을 실제 위치에 배치	BlockName, DeltaXyz
CreateMixedGroup	EntityMixedGroup	혼합 그룹: 서로 다른 타입의 엔티티 그룹화	Entities (목록)
CreateUniformGroup	EntityUniformGroup	단일 그룹: 동일 타입 엔티티 묶음	Entities (동일 타입 목록)
CreateDxf	EntityMixedGroup	DXF 가져오기: AutoCAD DXF 파일을 엔티티로 변환	DxfParser.Import
CreateHPGL	EntityMixedGroup	HPGL 가져오기: 플로터 파일(PLT, HPGL) 변환	HpglParser.Import
CreateGerber	EntityGerber	Gerber 가져오기: PCB 거버 파일을 엔티티로 변환	FileName, Color

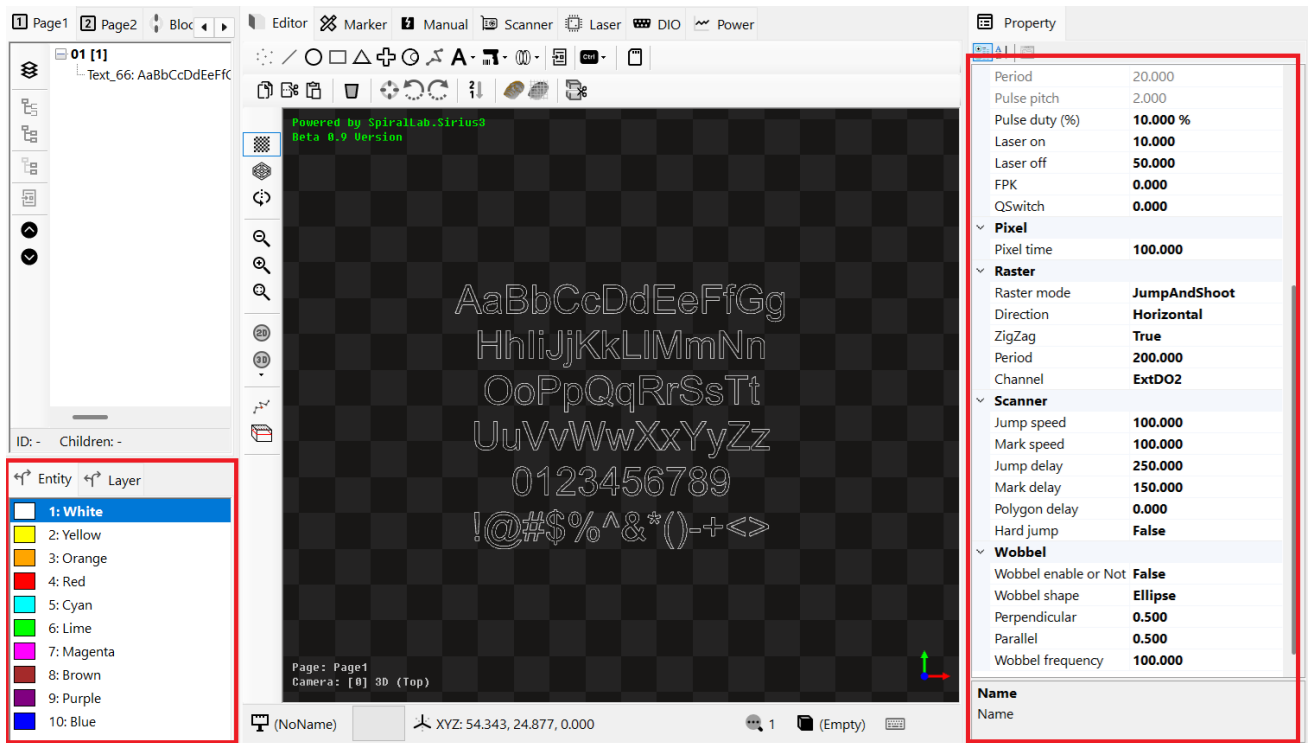
6.4 제어 엔티티 (Control Entities)

CreateDelay	EntityDelay	지연(Delay): 지정된 시간(ms) 만큼 대기	Mesc (지연 시간)
CreateJumpTo	EntityJump	점프(Jump): 레이저 OFF 상태로 고속 이동	JumpTo (목표 위치)
CreateMeasurementBegin	EntityMeasurementBegin	측정 시작: RTC 카드 데이터 샘플링 시작	SamplingFrequency, Channels
CreateMeasurementEnd	EntityMeasurementEnd	측정 종료: 샘플링 종료	-

<i>CreateMoFBegin</i>	<i>EntityMoFBegin</i>	MoF 시작: 이동체 가공(Marking on the Fly) 시작	Type, IsEncoderReset
<i>CreateMoFWait</i>	<i>EntityMoFWait</i>	MoF 대기: 엔코더 값이 특정 조건에 도달할 때까지 대기	Encoder, Condition, Position
<i>CreateMoFWaitRange</i>	<i>EntityMoFWaitRange</i>	MoF 범위 대기: 엔코더가 특정 범위 내 진입 시까지 대기	Min, Max
<i>CreateMoFEnd</i>	<i>EntityMoFEnd</i>	MoF 종료: 이동체 가공 종료	JumpTo (종료 후 위치)

7. 펜 속성 (Pen Properties)

시리우스 3 에서 펜(Pen)은 레이어(Layer)용 과 객체(Entity)용 이렇게 2 가지로 제공됩니다. 즉 레이어(Layer) 펜은 레이어 내부에 있는 모든 개체들에 대한 전역적인 설정 값들로 가공중에 적용되며, 개체(Entity) 펜은 개별 개체별로 지정된 펜 색상에 따라 그 설정 값이 가공중에 변경되어 적용됩니다.



이 펜 설정은 좌측 하단에 Entity (개체용 펜), Layer (레이어용 펜) 탭에서 사용하고 있는 펜 색상을 선택하여 속성값을 수정할 수 있습니다.

때문에 페이지(Page) 목록에서 특정 레이어(Layer)를 선택한 후 레이어(Layer)의 펜 색상(Pen color)을 변경하면 해당 레이어에 포함된 모든 개체들에 대해 전역적인 레이어 펜 설정 값들이 마커의 가공시에 반영됩니다.

(참고) 마커 가공시 레이어(Layer) 펜 속성이 적용되는 루틴을 사용자가 직접 구현(Override)하는 방법

```
marker.OnMarkLayerPen += OnMarkLayerPen;

bool Marker_OnMarkLayerPen(IMarker marker, EntityLayerPen pen)
{
    // 자세한 구현 사항은 editor_pen 예제 프로젝트 참고
}
```

(참고) 마커 가공시 개체(Entity) 펜 속성이 적용되는 루틴을 사용자가 직접 구현(Override)하는 방법

```
marker.OnMarkEntityPen += OnMarkEntityPen;

bool OnMarkEntityPen(IMarker marker, EntityPen pen)
{
    // 자세한 구현 사항은 editor_pen 예제 프로젝트 참고
}
```

(참고) 펜 개체에는 사용자 데이터가 저장가능한 **ExtensionData** 속성이 제공되며, 이를 이용해 사용자 데이터를 전달해 다양한 확장 처리도 가능합니다.

7.1 레이저 펜(Layer Pen) 속성 항목

Automatic Laser Control 사용을 위한 가공 속도, 위치, 또는 인코더 속도의 변화에 따라 레이저 에너지를 실시간으로 자동 조절하여 균일한 가공 품질을 유지하는 기능입니다. (참고) RTC5, RTC6에서 사용가능

Property	Type	Description
IsALC	bool	ALC(자동 레이저 제어)를 활성화하거나 비활성화할지 여부
AlcMode	AutoLaserControlModes	자동 레이저 제어 모드
AlcSignal	AutoLaserControlSignals	자동 레이저 제어 신호 유형
AlcPercentage100	double	ALC(자동 레이저 제어)의 100% 출력 값
AlcMinValue	double	ALC(자동 레이저 제어)의 최소 출력 값
AlcMaxValue	double	ALC(자동 레이저 제어)의 최대 출력 값
AlcByPositionTable	List<KeyValuePair<double; double>>	ALC(자동 레이저 제어) 위치 종속 보상 테이블

Sky Writing 은 레이저 가공 시 코너나 시작/끝 지점에서의 가공 품질을 극대화하기 위해 사용하는 기능입니다. 스캐너가 가속하거나 감속하는 구간을 마킹하려는 벡터의 바깥쪽(비가공 구간)으로 빼내어, 실제 레이저가 켜져 있는 구간에서는 항상 일정한 속도를 유지하도록 하는 기술입니다. (참고) RTC5, RTC6 에서 사용가능

Property	Type	Description
IsSkyWritingEnabled	bool	스카이라이팅 활성화 여부
SkyWritingMode	SkyWritingModes	스카이라이팅 모드
TimeLag	double	스카이라이팅 시간 지연 (usec)
LaserOnShift	double	스카이라이팅 레이저 켜기 시프트 (usec)
Prev	double	스카이라이팅 Prev(Run-in) 값 (usec)
Post	double	스카이라이팅 Post(Run-out) 값 (usec)
AngularLimit	double	스카이라이팅 각도 제한 (도)

Variable Polygon Delay (가변 폴리곤 지연)는 레이저 가공 시 연속된 벡터(Polyline) 사이의 코너를 처리할 때, 벡터 간의 각도에 따라 지연 시간(Delay)을 자동으로 조절하는 기능입니다. 이 기능을 사용하면 완만한 코너에서는 지연 시간을 줄여 가공 속도를 높이고, 급격한 코너에서는 지연 시간을 늘려 정밀도를 확보할 수 있습니다.

Variable Jump Delay (가변 점프 지연)는 레이저 스캐너가 이동(Jump)할 때, 이동 거리가 짧을수록 스캐너가 안정화되는 시간(Settling time)이 적게 든다는 점을 이용하여, 짧은 점프 구간에서 지연 시간(Jump Delay)을 자동으로 줄여 전체 가공 시간을 단축하는 기능입니다.

Property	Type	Description
IsVariablePolygonDelay	bool	가변 폴리곤 지연 활성화 여부
VariablePolygonDelayEdgeLevel	uint	가변 폴리곤 지연 엣지 레벨 (usec)
IsVariableJumpDelay	bool	가변 점프 지연 활성화 여부
VariableJumpDelayMin	double	가변 점프 지연 최소값 (usec)
VariableJumpDelayLimitLength	double	가변 점프 지연 한계 길이 (mm)

Bandwidth 는 **FilterBandwidth** 를 의미하며, 이는 **Scanner** 와 **Stage** 가 동작을 나누어 갖는 기준 주파수(Hz) 역할을 합니다. **MotionType** 은 **OperationMode** 를 의미하며, 이는 스캐너 단독, 스테이지 단독, 스캐너와 스테이지가 동시에 처리(**Bandwidth** 값이 사용됨)하는지 여부를 선택합니다. (참고) syncAXIS 에서 사용가능

Property	Type	Description
MotionType	MotionTypes	SyncAxis 모션 유형
BandWidth	double	SyncAxis 대역폭 (Hz)

7.2 개체 펜(Entity Pen) 속성 항목

레이저 출력 파워 및 주파수 및 펄스폭등에 대한 기본적인 정보를 설정합니다.

Property	Type	Description (Korean)
Power	double	레이저 출력 전력 (W)
PowerMax	double	레이저 소스 최대 전력 (W)
PowerPercentage	double	레이저 출력 비율 (%)
PowerMapCategory	string	레이저 파워 맵 범주
Frequency	double	펄스 주파수 (Hz)
PulseWidth	double	펄스 폭 (usec)
PulsePeriod	double?	펄스 주기 (usec)
PulsePitch	double?	추정된 펄스 피치 (um)
PulseDutyCycle	string	펄스 듀티 사이클 (%)
LaserFpk	double	FPK(First Pulse Killer) 값 (usec)
LaserQSwitchDelay	double	Q-스위치 지연 (usec)

레이저 지연 시간(Laser On, Off Delay) 및 스캐너 지연시간(Jump, Mark, Polygon Delay)을 설정합니다.
또한 점프 및 가공 속도(Jump, Mark Speed) 및 하드 점프 사용여부를 설정합니다.

Property	Type	Description (Korean)
LaserOnDelay	double	레이저 켜기 지연 (usec)
LaserOffDelay	double	레이저 끄기 지연 (usec)
JumpSpeed	double	스캐너 점프 속도 (mm/s)
MarkSpeed	double	스캐너 마킹 속도 (mm/s)
ScannerJumpDelay	double	스캐너 점프 지연 (usec)
ScannerMarkDelay	double	스캐너 마크 지연 (usec)
ScannerPolygonDelay	double	스캐너 폴리곤 지연 (usec)
IsHardJump	bool	하드 점프 활성화 여부

래스터 모드 설정, 방향, 주기, 출력 채널, 픽셀 가공 시간등을 설정합니다. (이미지 형태의 픽셀 정보를 가진 개체들의 가공시 사용됩니다.)

Property	Type	Description (Korean)
RasterMode	RasterModes	래스터 모드
RasterDirection	RasterDirections	래스터 방향
IsRasterZigZag	bool	래스터 지그재그 패턴 여부
PixelPeriod	double	픽셀 주기 (usec)
PixelChannel	ExtensionChannels	픽셀 출력 채널
PixelTime	double	픽셀 지속 시간 (usec)

와블(Wobble) 가공에 대한 모양, 크기, 주파수 등을 설정합니다.

Property	Type	Description (Korean)
IsWobbelEnabled	bool	와블 활성화 여부
WobbelShape	WobbelShapes	와블 모양
WobbelPerpendicular	double	와블 수직(횡단) 크기 (mm)
WobbelParallel	double	와블 평행 크기 (mm)
WobbelFrequency	double	와블 주파수 (Hz)

syncAXIS 에 사용되는 최소 속도, 레이저 온 쉬프트 시간등을 설정합니다. (syncAXIS 전용)

Property	Type	Description (Korean)
ApproxBlendLimit	double	SyncAxis 근사 블렌드 제한 (mm)
LaserOnShiftSCANa	double	SCANahead 레이저 켜기 시프트 (usec)
LaserOffShiftSCANa	double	SCANahead 레이저 끄기 시프트 (usec)
MinMarkSpeed	double	SyncAxis 최소 마킹 속도 (mm/s)

7.3 펜(EntityPen, EntityLayerPen) 생성시 초기화 처리

UI.Config.OnCreateEntityPen 이벤트에 핸들러 코드를 등록해 초기화시 값을 설정할 수 있습니다.

또한 PropertyVisibilityHelper.SetForType(typeof(EntityPen), false, "속성이름"); 이용해 사용하지 않는 속성항목을 감출수있습니다.

```
SpiralLab.Sirius3.UI.Config.OnCreateEntityPen += OnCreateEntityPen;
EntityPen OnCreateEntityPen(IDocument doc, Color color)
{
    ...
}
```

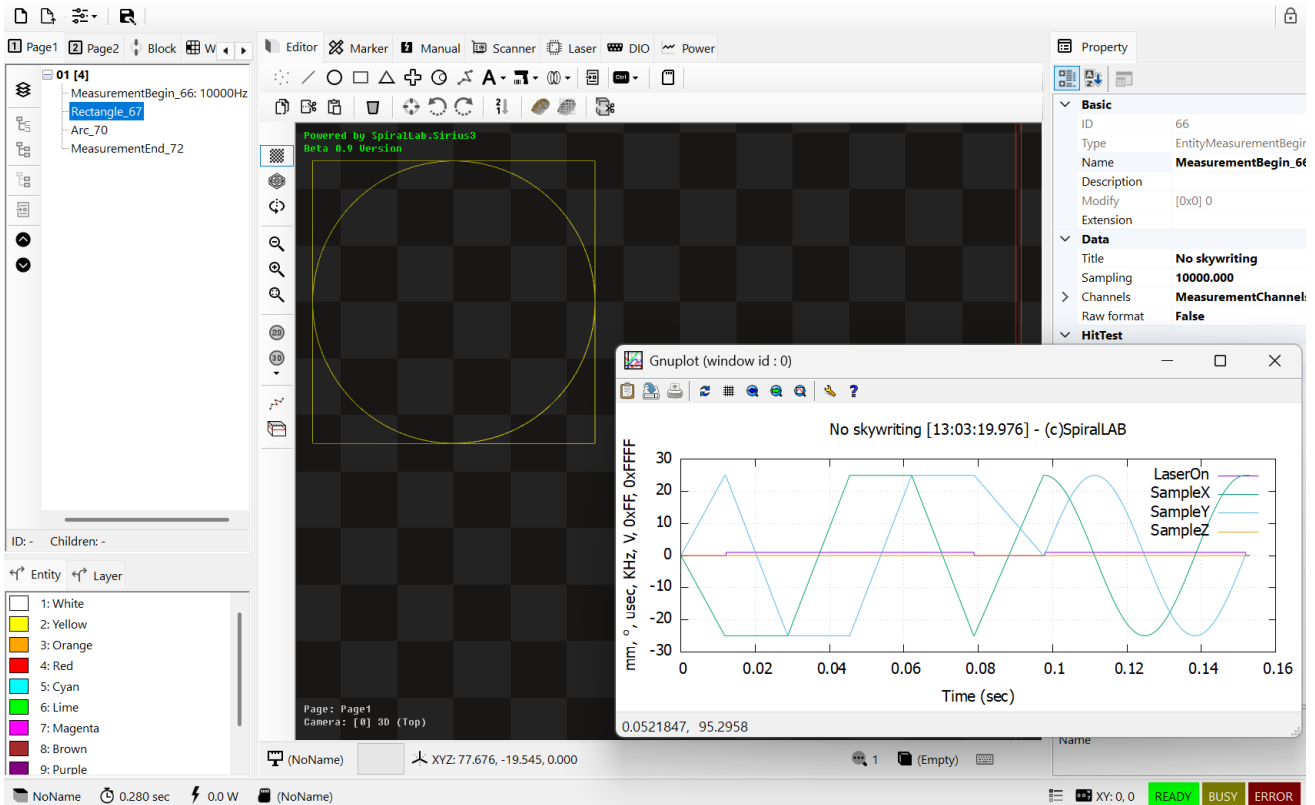
UI.Config.OnCreateLayerPen 이벤트에 핸들러 코드를 등록해 초기화시 값을 설정할 수 있습니다.

또한 PropertyVisibilityHelper.SetForType(typeof(LayerPen), false, "속성이름"); 이용해 사용하지 않는 속성항목을 감출수있습니다.

```
SpiralLab.Sirius3.UI.Config.OnCreateLayerPen += OnCreateLayerPen;
EntityPen OnCreateLayerPen(IDocument doc, Color color)
{
    ...
}
```

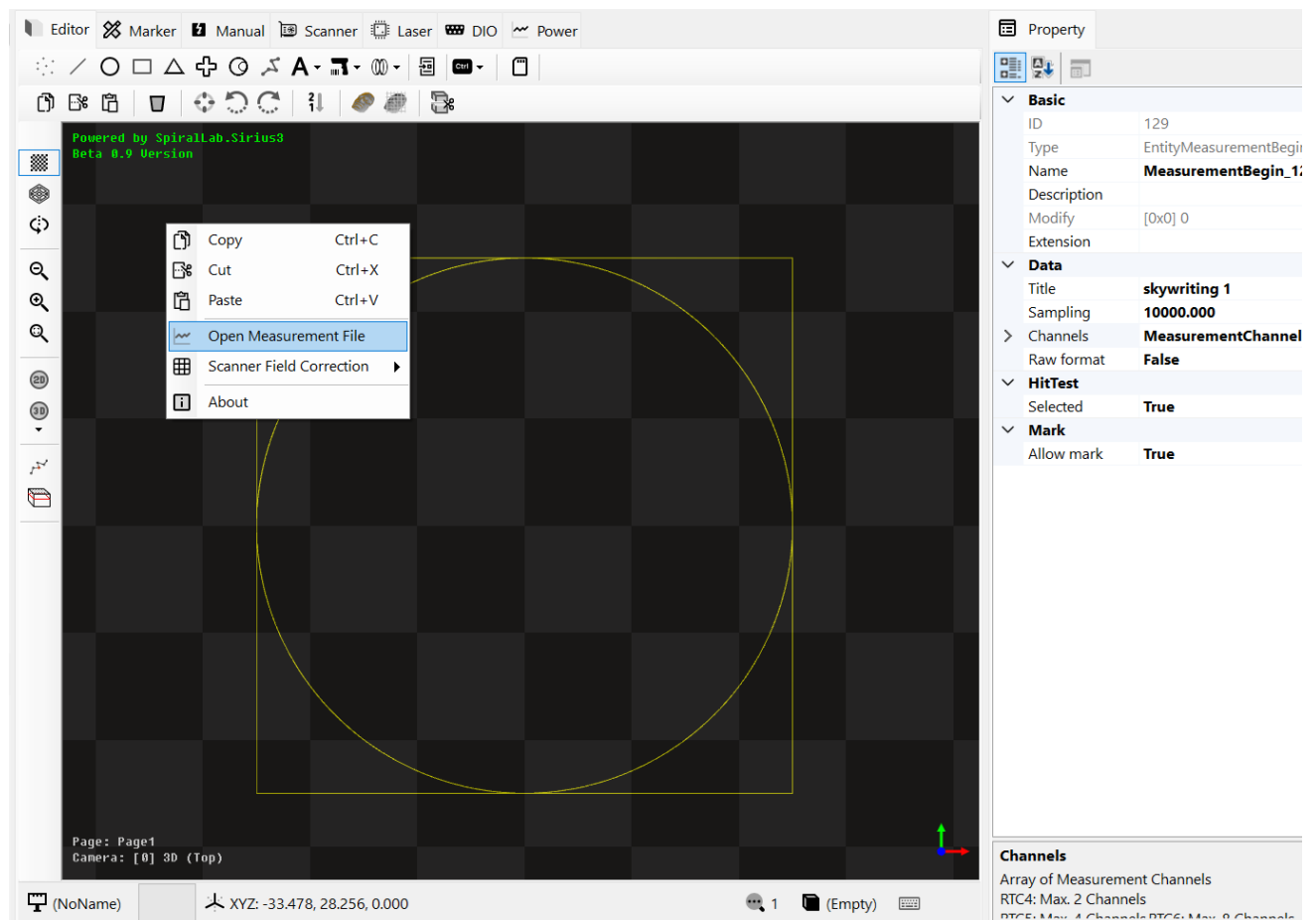
8. 계측 (Measurement)

계측 기능은 계측 시작 및 끝(Measurement Begin ~ End) 사이에 포함된 개체들에 대해서, 지정된 샘플링 주기(Sampling Frequency: 최대 100KHz)와 채널 정보를 사용하여 마커 가공이 완료되면, 채널 데이터를 모아(Gathering) 이를 그래프로 출력(Plot)하는 기능을 제공합니다.



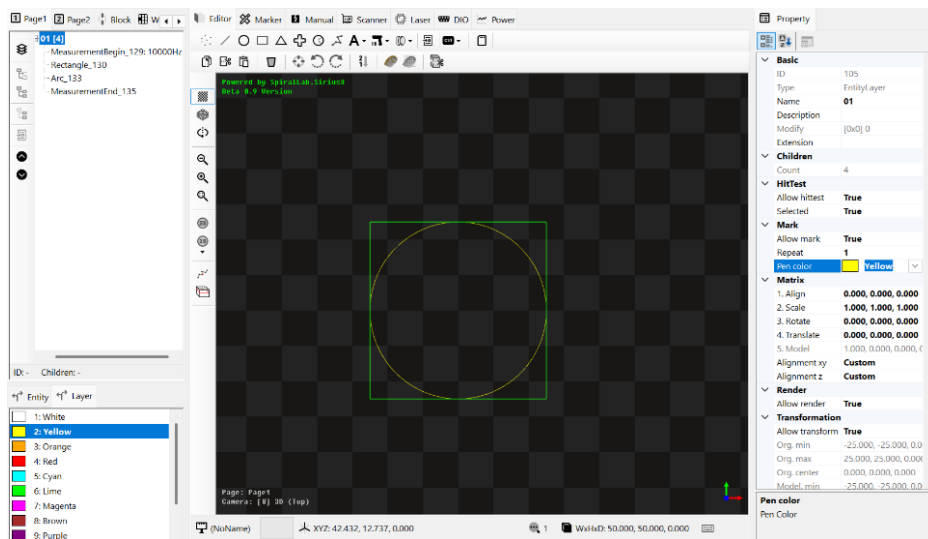
위 예제에서는 사각형과 원을 가공하는데 이때 4 개의 채널, 즉 Sample X,Y,Z 및 Laser On 정보를 대상으로 데이터를 시각화 한 모습입니다. 가로축은 시간, 세로축은 데이터값으로 Sample X,Y 채널의 의미는 RTC 제어기에서 내려진 지령된(Commanded) 위치값(mm) 을 의미하며, 기울기는 속도(Velocity)에 해당합니다. 또한 Laser On 은 0 혹은 1 값을 가지는데, 1 의 경우 LASER ON 이 진행된 구간을 의미합니다.

Marker 에서는 Plot 값이 True 가 기본값으로 설정되어 있으며, 이 의미는 Measurement 세션이 있는 경우 취득 데이터를 파일에 저장하고 자동으로 그래프 출력(gnuplot 프로그램 사용됨)이 진행됩니다.

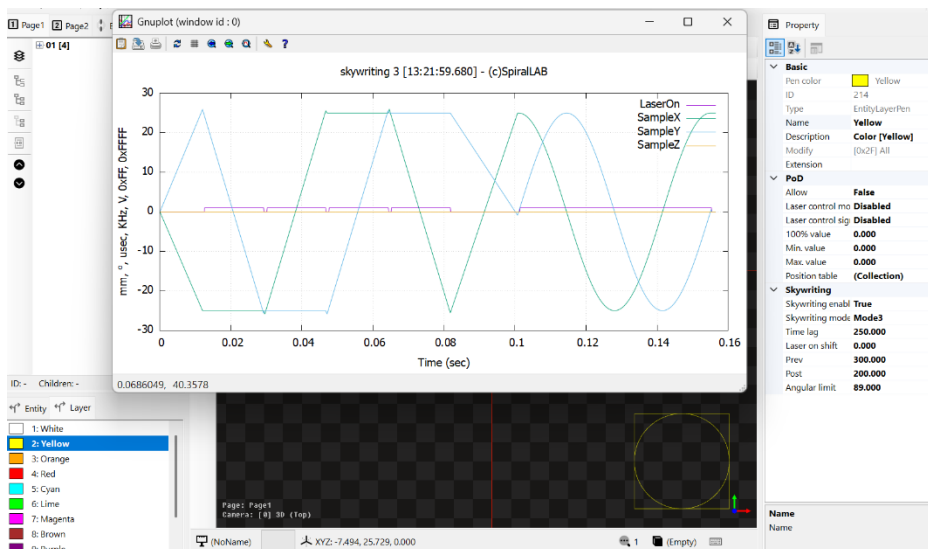


이전에 취득된 계측 데이터를 수동으로 그래프 출력하고자 하면, 편집기에서 마우스 오른쪽 버튼으로 보여지는 메뉴에서 **Open Measurement File** 을 이용해 확인이 가능합니다.

8.1 레이어 펜(Layer Pen) 속성의 Skywriting 기능을 사용한 결과



레이어(Layer)를 선택하여 펜 색상 (Pen color)를 노란색 (Yellow)으로 변경합니다.

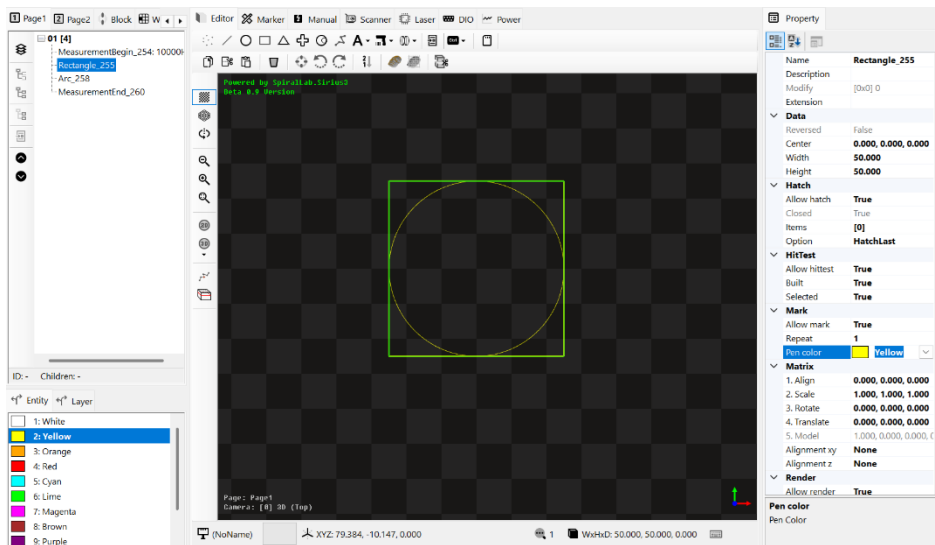


레이어 펜 (Layer Pen) 탭에서 노란색 레이어용 펜을 선택하고 그 속성에서 Skywriting 모드 3 을 선택 (활성화) 하고 Angular Limit 를 89 도(사각형의 모서리가 90 도 이므로 모서리에서 활성화하기 위해 89도로 설정) 로 설정한 후 마커의 가공을 시작합니다.

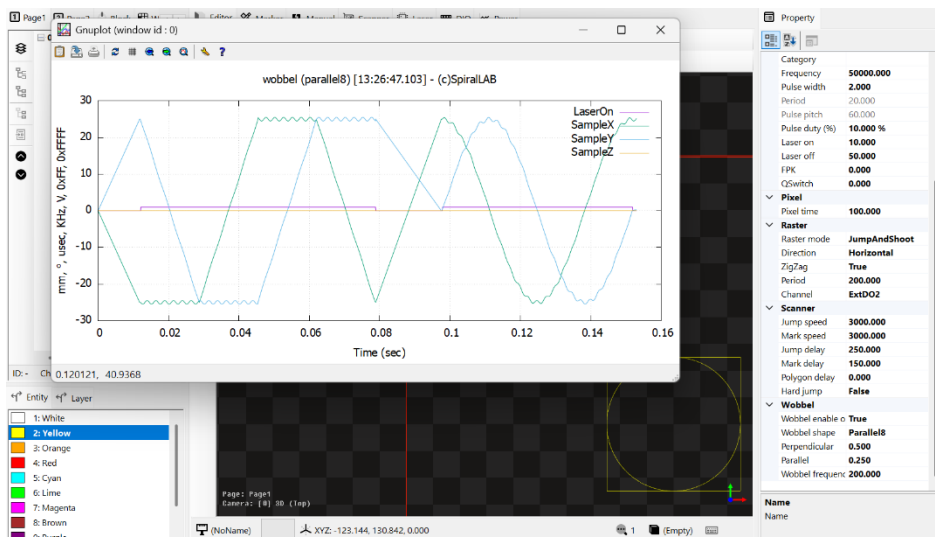
Mode 3을 사용한 결과 활성화 각도 (Angular Limit) 가 적용되었고 결국 사각형의

모서리 위치 에서는 전, 후진에 해당하는 스캐너의 이동이 추가되었고 또한 LASER ON/OFF 도 발생한 것을 확인할 수 있습니다.

8.2 개체 펜(Entity Pen) 속성의 Wobble 기능을 사용한 결과



개체(Entity)를 선택하여 펜 색상 (Pen color)를 노란색 (Yellow)으로 변경합니다.

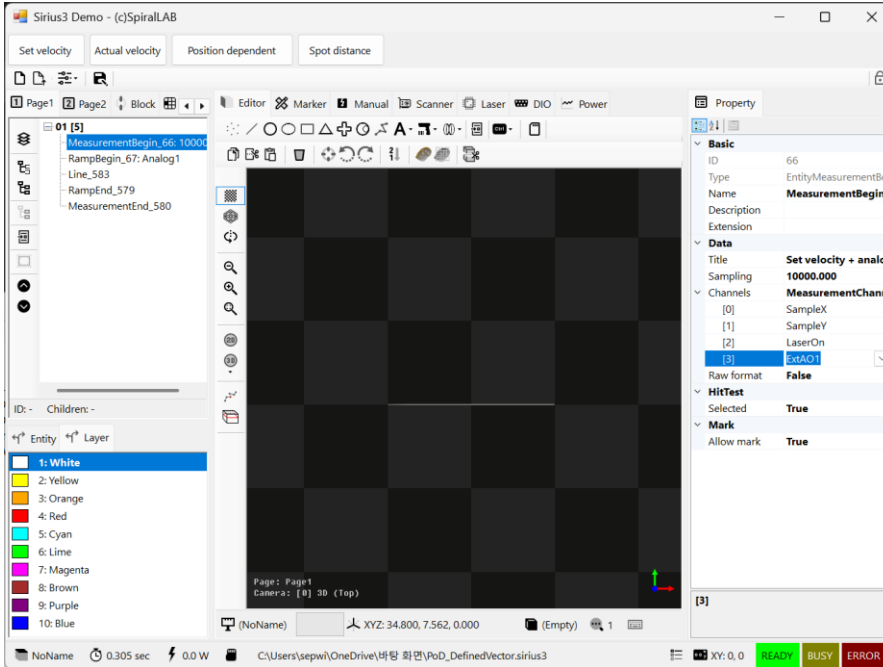


개체 (Entity Pen) 탭에서 노란색 개체용 펜을 선택하고 그 속성에서 Wobbel 모드를 활성화 하고 (여기에서는 Parallel8 모양 선택) 진폭 및 와블 주파수를 설정합니다. 그 결과 진행방향에 평행한 8자 형태의 스캐너 모션이 추가된 것을 확인할 수 있습니다.

(참고) RTC 제어기 장치별로 계측 가능한 최대 데이터 개수가 있어, 최대 계측 가능한 데이터를 넘게되면 초기 버퍼가 점차적으로 삭제됩니다. 또한 계측 채널 개수의 제한이 있어, 최대 RTC5 카드는 최대 4 채널, RTC6 카드는 최대 8 채널까지 사용이 가능합니다.

8.3 Defined Vector 기반 자동 레이저 제어 (Automatic Laser Control)

Defined Vector 방식은 가공 구간의 시작 출력과 종료 출력을 선형(Linear)으로 출력을 가변 하는 가공 방식입니다. 구체적으로는 **Ramp Begin** 과 **Ramp End** 사이에 위치해야 하며, 비율에 해당하는 **Ramp Scale** 값(허용 범위는 0~4 배까지)에 의해 선형적으로 가변 출력 됩니다.

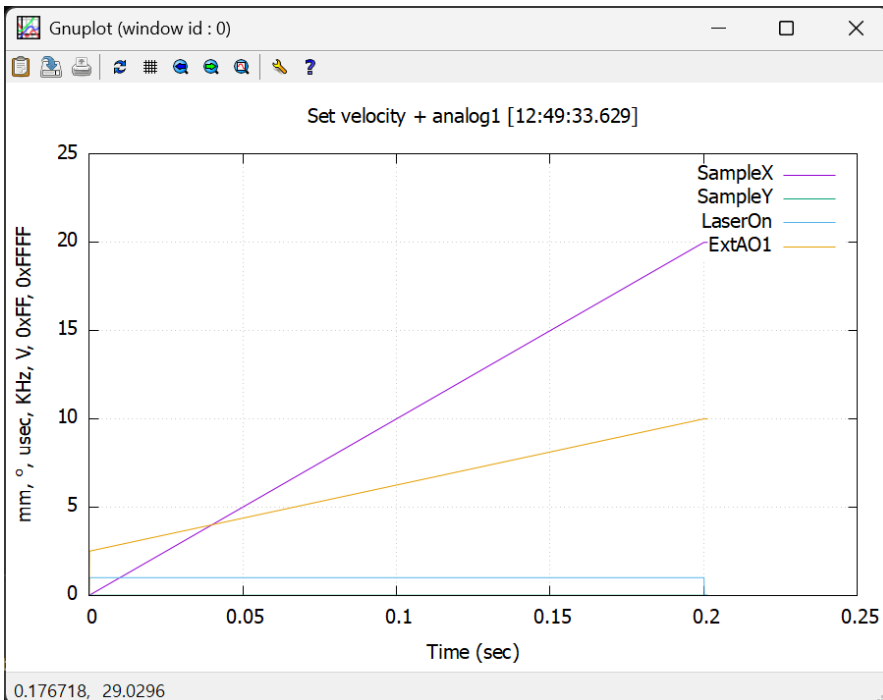


여기에서는 계측 채널에 아날로그 1 (ExtAO1)을 포함시킵니다. 또한 **Defined Vector** 용 **Ramp** 구간을 생성하기 위해서는 앞 뒤로 **Ramp Begin** 및 **Ramp End** 제어 객체 추가합니다.

Ramp Begin에서는 아날로그 1 출력을 제어 신호로 설정하고, 시작 출력값(Starting Value)을 5V 로 설정 하였습니다.

또한 **Ramp** 구간사이에 선분(EntityLine) 객체를 추가하였고, 시작 램프 비율(Start Ramp), 종료 램프 비율(End Ramp)을 각각 0.5,

2.0 으로 설정하였습니다.

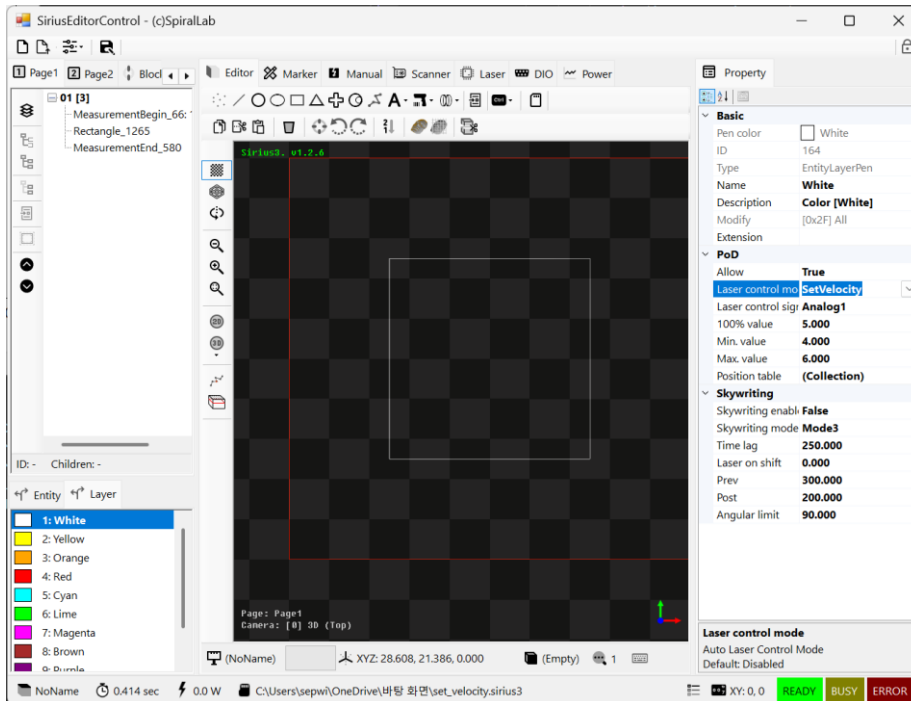


이 같은 **Defined Vector** 방식의 가공의 계측 결과를 보면, 시작 위치에서의 아날로그 1 포트의 출력값(Starting Value)은 0.5 비율로 실제 시작위치에서 $5V * 0.5 = 2.5V$ 로 출력이 시작하고, 2.0 비율로 끝나는 위치에서 $5V * 2.0 = 10V$ 로 출력이 종료되고 있는 것을 확인할 수 있습니다.

이를 응용하면 시작, 끝 구간을 가공하면서 지정된 제어 신호로 시작, 종료 신호의 출력을 선형적으로 가변시키는 가공이 가능합니다.

8.4 Speed Dependent 기반 자동 레이저 제어 (Automatic Laser Control)

Speed Dependent 방식은 가공시의 스캐너의 다양한 속도(Velocity)에 따라 출력신호를 실시간 보상하는 가공 방식입니다. 구체적으로는 레이어 펜(EntityLayerPen)의 PoD 기능을 True 시킨 후 제어 모드 및 신호를 선택하고, 출력 신호(100%, Min, Max) 범위를 설정해 사용하는 방식입니다.

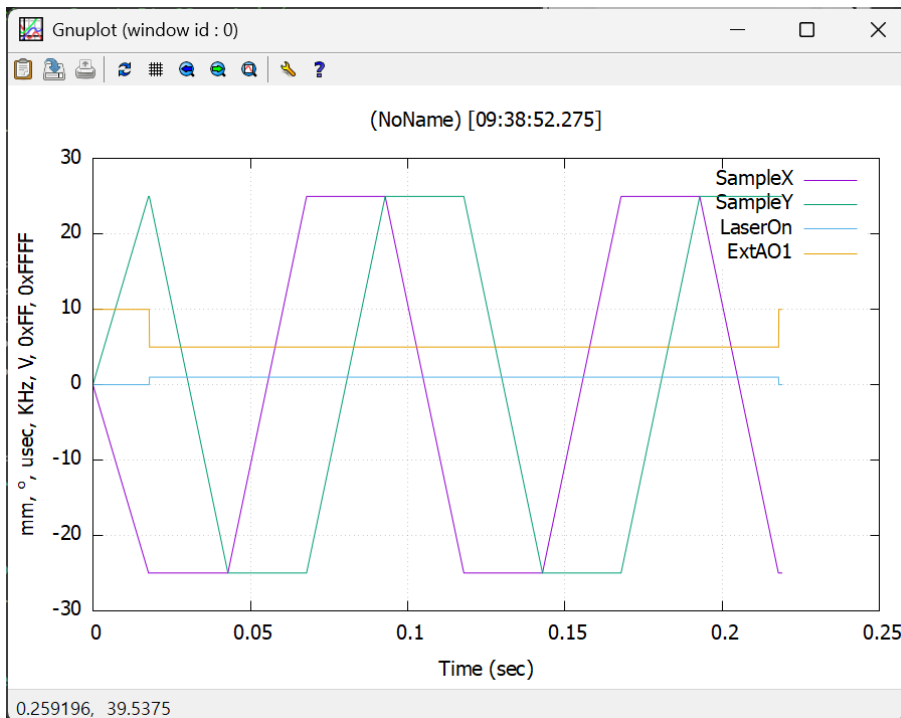


활성화된 레이어 펜 (LayerPen)에 해당하는 흰색(White) 레이어 펜 속성 목록에서 PoD 기능을 활성화 (Allow 를 True 로 설정) 합니다.

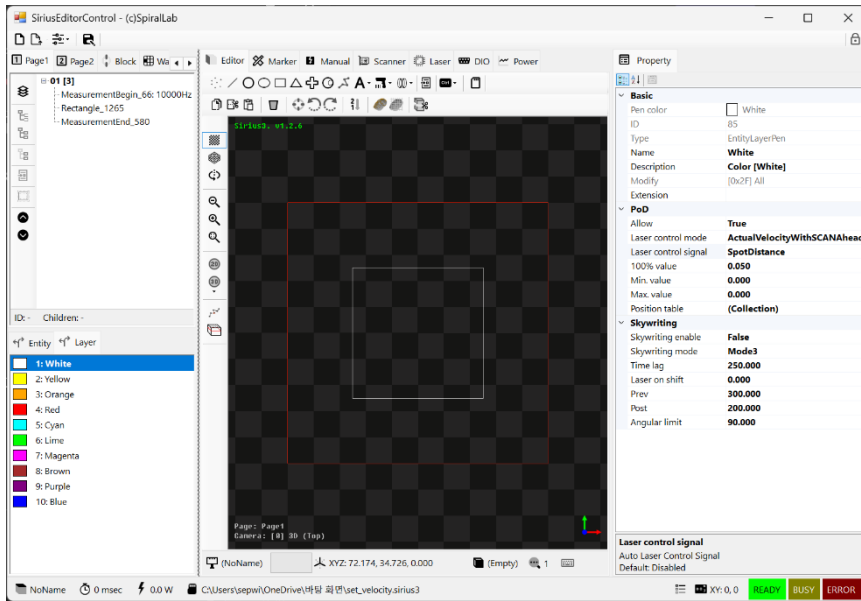
Control Mode 는 어떤 속도를 대상으로 할지 선택합니다. 여기에서는 SetVelocity 를 선택합니다. 또한 또한 가변할 출력 신호도 선택합니다. 여기에서는 Analog1 를 선택합니다.

100% 출력값과 최대(Min), 최소(Max) 값의 경우 아나로그이면 0~10V, ExtD08

이면 0~255, ExtD016 이면 0~65535, Pulse Width 이면 시간(usec)값, Frequency 이면 주파수(Hz) 값 단위로 입력해야 합니다.



이 같은 Speed Dependent 기반 자동 방식 가공의 계측 결과를 보면, 가공 구간(LASER ON)에서 100%에 해당하는 아나로그 5V 출력이 유지되는 것을 확인할 수 있습니다.



iDRIVE 를 지원하는 스캔 헤드의 경우 **ActualVelocity** 를 사용해 실시간 피드백되는 스캐너의 속도를 기반으로 좀더 고성능의 가변 출력 제어가 가능합니다.

더우기 RTC6 + SCANahead + iDRIVE 조합에서는 SDC(Spot Distance Control)를 이용해 가감속 구간에서 레이저 출력 스팟을 고정 간격으로 출력하는 고급 기능도 가능합니다.

이를 사용하기 위해서는 레이더펜(EntityLayerPen) 에서 제어 모드를 **ActualVelocity With**

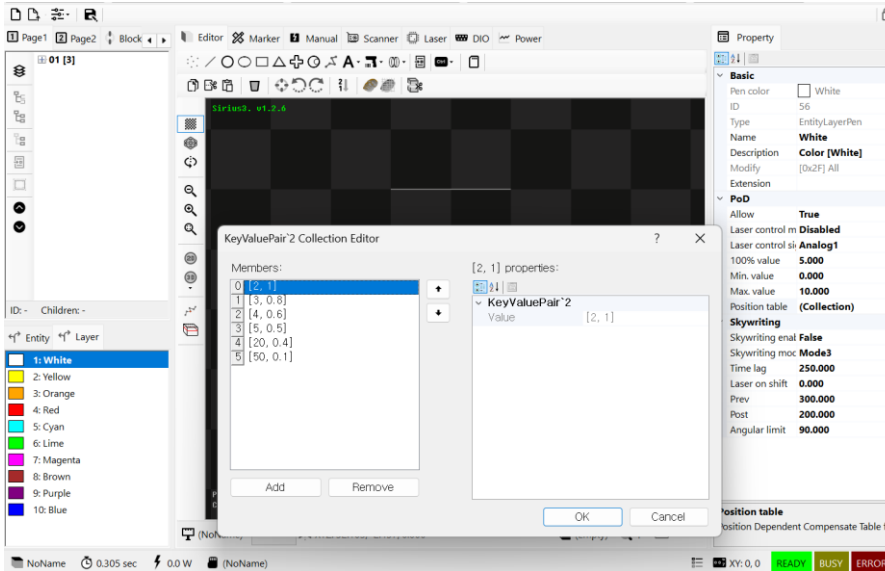
SCANahead 및 제어 신호는 **SpotDistance** 거리값을 선택합니다. 이때 100% 출력값에 사용자가 원하는 거리값(mm)을 지정합니다. (참고) SDC 의 경우 Min, Max 값은 무시됩니다.

(참고) SCANahead 기능을 위해서는 RTC6 카드에 SCANahead 옵션이 활성화되어 있어야 하고, 스캔헤드가 이에 호환되는 제품(Excelliscan, intelliSCAN IV 시리즈) 이어야 합니다.

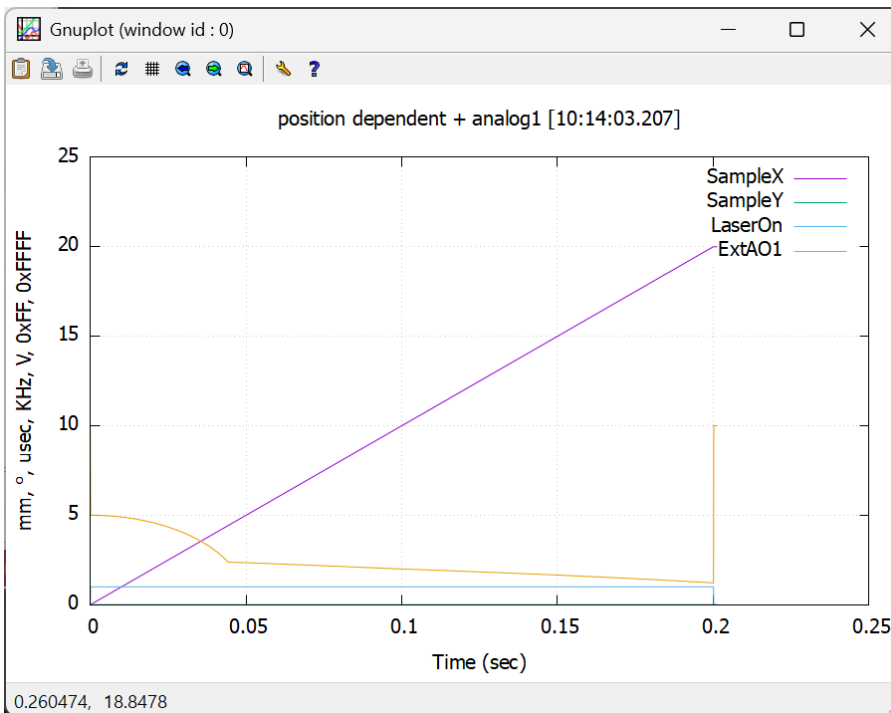
(참고) SCANahead 기능을 최대로 사용하기 위해서는 **Auto delays** 옵션을 활성화 하고, **Line, Acc, Corner Scale** 값을 통해 최적화를 해야 합니다.

8.5 Position Dependent 기반 자동 레이저 제어 (Automatic Laser Control)

Position Dependent 방식은 가공시의 스캐너의 X,Y 위치(Position)에 따라 출력신호를 실시간 보상하는 가공 방식입니다. 구체적으로는 레이어 펜(EntityLayerPen)의 PoD 기능을 True 시킨 후 제어 모드는 Disable 및 신호를 선택하고, 출력 신호(100%, Min, Max) 범위를 설정합니다. 또한 Position Table 에는 거리값(Distance) 및 비율(Scale: 0~4) 값을 키값 배열로 설정해야 합니다.



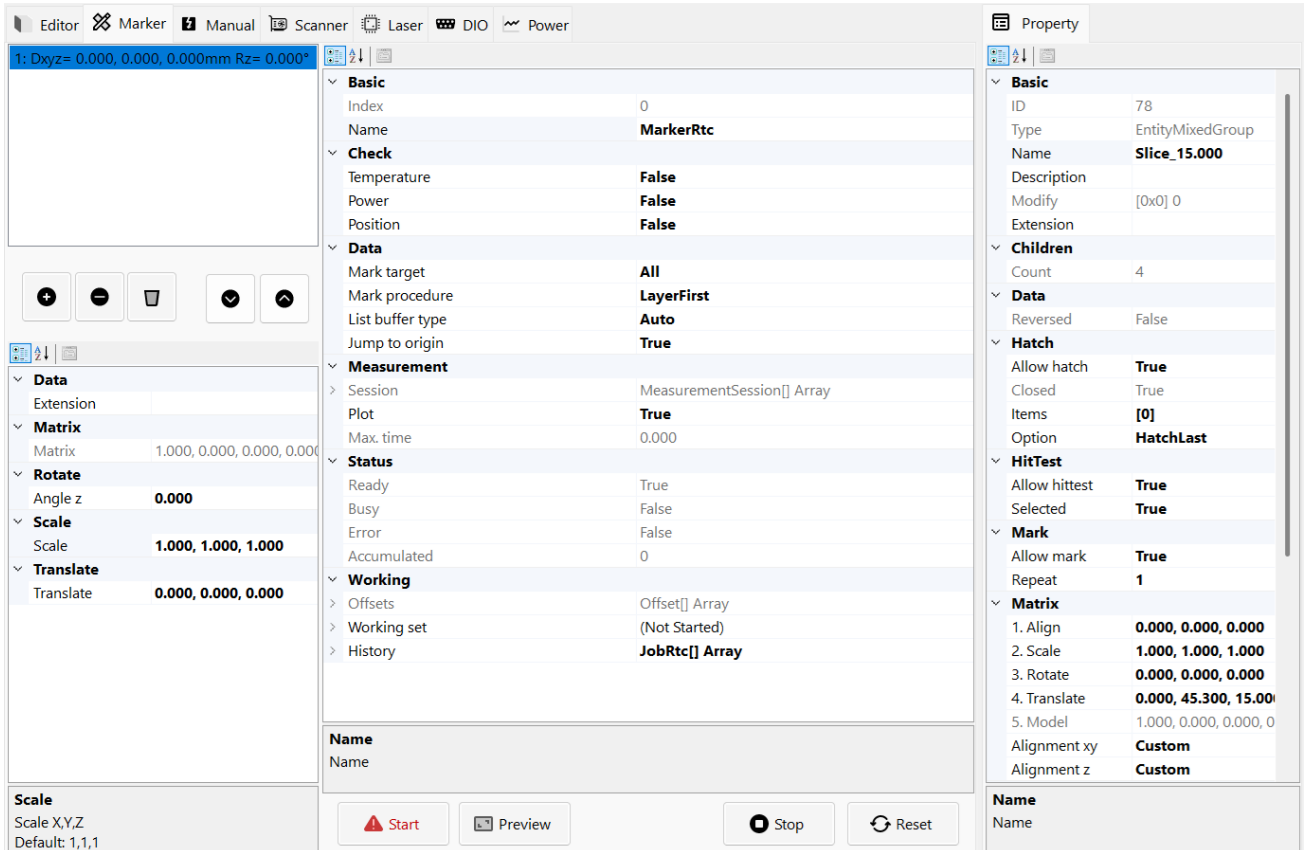
여기에서는 Position Table 에는 (EntityLayerPen. AlcByPositionTable 속성) 에 6 개의 거리, 비율 테이블 정보를 추가하고 계측을 진행합니다.



계측 결과 아날로그 출력이 스캐너의 위치(거리)에 따라 해당 실제 출력값 = 신호의 출력 * 위치에 따른 비율 이 처리된 것을 확인할 수 있습니다.

9. 마커(Marker)

특정 스캐너 제어기(IRtc)에 따라 마커(IMaker) 제어 객체를 생성해 사용하게 되면 아래와 같이 마커 탭에서 해당 객체 상태 및 정보를 확인할 수 있습니다.



마커의 역할은 가공 데이터(IDocument)를 이용해 지정된 페이지(IPage) 데이터에 대해서 레이저 가공을 진행할 때 가공 순서 및 옵션을 처리하는 제어용 객체입니다.

또한 가공 데이터들을 오프셋(Offset) 위치에 대해서 반복적으로 가공이 진행되며, 사용자는 오프셋 위치들을 여러 개 등록해서 동일한 데이터를 다양한 위치에 반복 가공이 가능합니다. (참고) 오프셋은 크기(Scale) -> 회전(Angle Z) -> 이동(Translate) 순서로 적용됩니다.

최종적으로는 가공을 위한 정점 데이터가 최종 출력되기 위한 변환 순서는 다음과 같습니다.

개체의 정점 원본 좌표(Original x, y, z) ->

개체의 모델 행렬(ModelMatrix: 정렬(Align) -> 크기(Scale) -> 회전(Angle Z, Y, X) -> 이동(Translate)) ->

마커(IMarker)의 오프셋 행렬(Offset: 크기(Scale) -> 회전(Angle Z) -> 이동(Translate)) ->

변환된 개체의 정점 좌표(Transformed x, y, z) ->

9.1 MarkerRtc 객체

- Check

- **Temperature:** 마커 가공 시작(Marker.Start)시에 스캐너의 온도가 적절한지 검사하는 옵션입니다.
- **Power:** 마커 가공 시작(Marker.Start)시에 스캐너의 전원이 정상적으로 공급 중 인지 여부를 검사하는 옵션입니다.

- **Position:** 마커 가공 시작(Marker.Start)시에 스캐너의 위치 응답(Position ACK)이 정상적인지를 검사하는 옵션입니다.
- **Data**
 - **Mark Target:** 선택된 개체(Selected)만 가공할지 아니면 모두(All) 가공할지 여부를 선택합니다.
 - **Mark procedure:** 오프셋(Offset)과 레이어(Layer) 간에 가공 순서를 결정합니다.
 - **Layer First:** 오프셋 1 -> 레이어 1, 레이어 2, ... -> 오프셋 2 -> 레이어 1, 레이어 2, ...
 - **Offset First:** 레이어 1 -> 오프셋 1, 오프셋 2, ...-> 레이어 2, -> 오프셋 1, 오프셋 2, ...
 - **List buffer type:** 스캐너 제어장치(IRtc)의 리스트 버퍼 모드를 결정합니다.
 - **Auto:** 더블 버퍼 처리가 자동으로 사용됨. (리스트 버퍼에 데이터가 충분히 저장되면 자동으로 가공 시 시작됨)
 - **Single:** 단일 버퍼 처리가 사용됨. (리스트 버퍼가 허용하는 데이터 개수만 입력되며, 가공 시작 명령(ListExecute)이 있어야 가공이 시작됨)
 - **Jump to origin:** 가공 완료 후 스캐너 장치의 원점(0,0)으로 자동으로 점프할지 여부를 결정합니다.
- **Measurement**
 - **Plot:** 가공 완료 후 플롯 출력을 할지 여부
 - **RTC 제어기 사용시:** 계측 개체(Measurement Begin/End)를 이용하고 있을 때, True 인 경우에는 가공이 완료된 후 계측 데이터를 저장 후 gnuplot 를 이용해 지정된 채널 데이터에 대한 프로파일 출력을 자동으로 실행합니다.
 - **syncAXIS 제어기 사용시:** 시뮬레이션 모드를 사용하고 True 인 경우 가공이 완료되면 syncAXISViewer 프로그램을 이용해 스캐너, 스테이지에 대한 모션 프로파일을 출력 및 실행을 자동으로 실행합니다.
- **Status**
 - **Ready:** 마커가 가공이 가능한 준비 상태인지 여부
 - **Busy:** 마커가 가공중인 상태인지 여부
 - **Error:** 마커가 에러 상태인지 여부
- **Working**
 - **Offsets:** 가공중에 있는 현재 오프셋 정보입니다.
 - **Working set:** 가공중에 있는 현재 작업 집합 정보입니다. 주요 정보는 다음과 같습니다.
 - 현재 페이지(IPage) 및 페이지 번호
 - 현재 레이어(EntityLayer) 및 레이어 번호
 - 현재 개체(IEntity) 및 개체 번호
 - **History:** 누적된 가공 이력 정보들입니다.

Start 버튼: 현재 작업중인(화면에서 활성화된) 페이지에 대해 마커 가공 시작을 시도합니다. (주의) 마커의 상태가 **Ready** 가 **True** 이고 **Error** 가 **False** 인지 여부를 확인해야 합니다.

Preview 버튼: 현재 선택된 개체들이 있는 상태에서 이 버튼을 누르면 개체들을 감싸는 외곽 사각영역에 대해 레이저 소스의 지시빔(**Guide Beam**)을 활성화 한 후 반복적으로 해당 영역에 대해 지시빔 출력을 진행합니다.

(주의) 레이저 소스 객체가 지시빔 기능(**ILaserGuideControl** 인터페이스)을 상속 구현하고 있어야 합니다.

Stop 버튼: 가공을 강제로 중단(**Abort**) 합니다.

Reset 버튼: 마커의 에러 상태를 초기화 하기 위해 리셋을 시도합니다.

(참고 1) 마커에 대한 키보드 단축키(**Marker Keyboard Shortcut**) 정보

F4: 마커 가공 프리뷰(**Preview**) 시작

F5: 마커 가공 시작(**Start**)

F6: 마커 가공 중지(**Stop**)

F8: 마커 에러 리셋

(참고 2) **IMarker** 내부에서는, 각 개체의 모델 행렬(**ModelMatrix**)과 마커의 오프셋 행렬(**Offset**)은 **IRtc** 의 행렬 스택(**MatrixStack**) 에 자동으로 **Push**, **Pop** 되어, 전달되는 모든 정점 좌표들이 자동으로 변환 처리됩니다.

9.2 MarkerRtcFast 객체

(공개된 **MarkerRtcFast.cs** 코드를 참고해 주시기 바랍니다)

9.3 MarkerSyncAxis 객체

(공개된 **MarkerSyncAxis.cs** 코드를 참고해 주시기 바랍니다)

10. 수동(Manual)

사용자는 레이저 출력 파워(W)를 변경하고 일정시간동안 레이저를 출사하고 스캐너의 위치를 이동시키는 수동 작업을 수동 화면 탭에서 수행할 수 있습니다.

The screenshot shows the 'Manual' control interface. At the top, there are tabs: Editor, Marker, **Manual** (highlighted with a red box), Scanner, Laser, DIO, and Power. Below the tabs, the 'Power (W)' section has a text input field showing '0.000' and a slider below it, with a 'Max: 20.000' label. The 'Power Category' has a dropdown menu and a 'LookUp' checkbox. The 'Frequency (Hz)' is set to '50000.000'. The 'Pulse Width (usec)' is set to '2.000'. The 'Pulse Cycle (%)' is set to '10.000' with a 'Set' button. Below these, the 'Position' is '0, 0, 0' and 'Speed (mm/s)' is '500', with 'Move', 'Origin', and 'Jog' buttons. At the bottom, there are checkboxes for 'Turn Off (s): 10.0' and 'Show Warning', and large 'On' and 'Off' buttons.

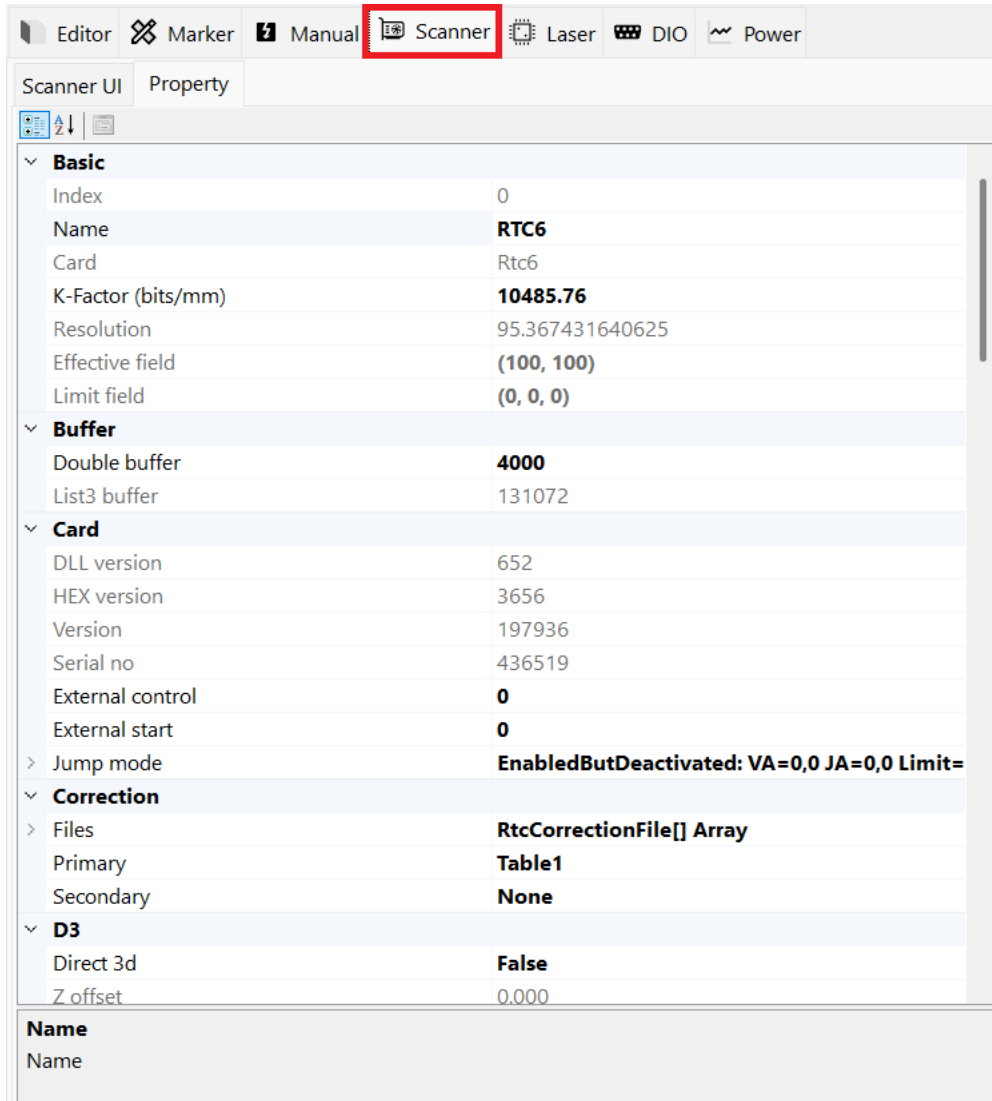
Manual 탭에서는 레이저 소스의 출력값을 수동으로 변경이 가능하고, 스캐너의 반사 X,Y 위치 이동 등 다양한 수동 작업이 가능합니다. 여기에서는 레이저 빔의 정렬(Alignment) 혹은 수동 파워 측정등의 용도로 사용 가능합니다.

(참고 1) RTC 제어기의 LASER 1,2 및 LASERON 신호선에서 발생하는 펄스 신호가 레이저 소스 입력 처리되어 발진됩니다. 이때 신호 레벨(Active Low, High)을 확인해야 하고, Yag 1,2 와 같이 어떤 레이저 모드(Laser Mode)를 사용하고 있는지 확인해야 합니다. 또한 레이저 소스 제품의 사용자 매뉴얼을 참고해 RTC 제어기와 레이저 소스간에 정상적인 결선이 되어 있어야 합니다.

(참고 2) 레이저 발진(On)을 시도할 경우에는 반드시 레이저 빔에 의한 위험 요소를 모두 확인하고 진행해야 합니다.

11. 스캐너(Scanner)

스캐너(**IScanner**) 제어 장치에 대한 상태 및 설정을 변경할 수 있습니다. 예를 들어 RTC6 제어를 위해 **Scanner** 객체를 생성해 사용한다면 아래와 같이 RTC 내부의 다양한 상태를 확인할 수 있고, 나아가 세부적인 설정을 변경할 수 있습니다.



ScannerFactory에 의해 생성된 **IRtc** 객체 정보가 출력됩니다. 만약 레이저 소스 UI를 삽입하는 것처럼 **Scanner UI**에 외부에서 만든 사용자 컨트롤을 **Config.OnCreateScannerUI** 이벤트를 사용해 추가할 수 있습니다.

```
SpiralLab.Sirius3.UI.Config.OnCreateScannerUI+= OnCreateScannerUI;
private Control OnCreateScannerUI (IScanner scanner)
{
    return new YourScannerUIControl()
    {
        Scanner = scanner
    };
}
```

12. 레이저(Laser) 소스

현실적으로 매우 다양한 레이저 소스 디바이스가 존재하며, 그 제어 방식 또한 사용자의 요구사항에 맞도록 사용하기 때문에, 모든 환경에 맞는 레이저 소스 구현부를 라이브러리 내부에서 모두 지원하기는 불가능합니다. 때문에 사용자는 손쉽게 레이저 소스 객체를 직접 구현하고 이를 라이브러리 내에 포함시켜 확장 사용이 가능합니다.

12.1 레이저 소스 제어부 구현

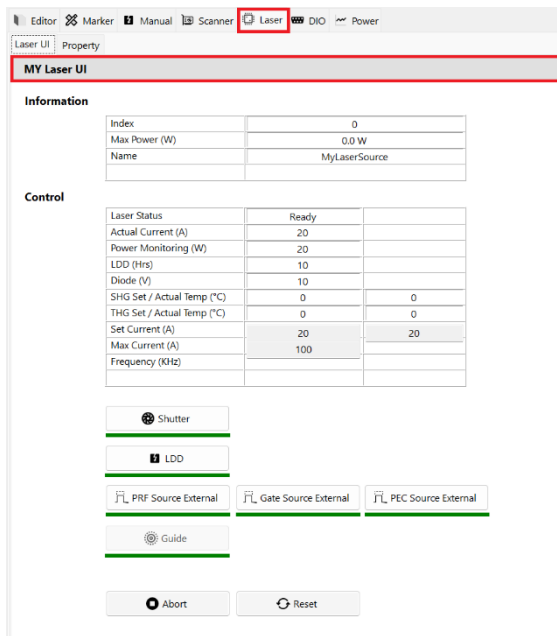
레이저 소스를 구현하기 위해서는 라이브러리에서 제공하는 **ILaser** 인터페이스를 상속 구현해야 합니다. (레이저 파워 제어를 위한 **ILaserPowerControl** 와 지시빔 제어를 위한 **ILaserGuideControl** 인터페이스도 제공됩니다.)

가장 손쉬운 방법은 다음과 같이 추상 객체인 **LaserBase** 를 상속 구현하는 것을 추천드립니다.

```
public class MyLaserSource
    : LaserBase
    , ILaserPowerControl
    , ILaserGuideControl
{
    ...

    // 레이저 소스 제어 로직을 구현
}
```

12.2 레이저 소스 UI 구현



앞선 레이저 소스 제어부 맞는 UI는 사용자 컨트롤 (**UserControl**)로 만들어 **SiriusEditorControl**의 **Laser** 탭 내부 컨트롤을 손쉽게 교체할 수 있습니다. 이를 위해서는 **YourLaserUIControl**에 해당하는 사용자 컨트롤 UI 객체를 직접 디자인 및 구현한 후 다음과 같은 이벤트 핸들러를 이용해 레이저 소스 UI를 변경(오버라이드) 할 수 있습니다.

```
SpiralLab.Sirius3.UI.Config.OnCreateLaserUI += OnCreateLaserUI;
private Control OnCreateLaserUI(ILaser laser)
{
    return new YourLaserUIControl()
    {

```

```
        LaserSource = laser // MyLaserSource 의 인스턴스 전달  
    };  
}
```

(참고) `editor_laser_ui` 예제 프로젝트를 통해 실제 구현 및 사용된 예제가 제공됩니다.

13. 디지털 입출력 (DIO)

RTC 제어기에는 다양한 디지털 입출력 포트가 제공됩니다. 각각의 포트를 다음과 같이 생성 및 초기화 하고 이를 편집기에 설정합니다.

```
editor.DIExt1 = IOFactory.CreateInputExtension1(rtc);
editor.DIExt1.Initialize();
editor.DOExt1 = IOFactory.CreateOutputExtension1(rtc);
editor.DOExt1.Initialize();
editor.DOExt2 = IOFactory.CreateOutputExtension2(rtc);
editor.DOExt2.Initialize();
editor.DILaserPort = IOFactory.CreateInputLaserPort(rtc);
editor.DILaserPort.Initialize();
editor.DOLaserPort = IOFactory.CreateOutputLaserPort(rtc);
editor.DOLaserPort.Initialize();
```

(참고) syncAXIS 제어기 사용시 확장 1 포트만 사용 가능합니다.

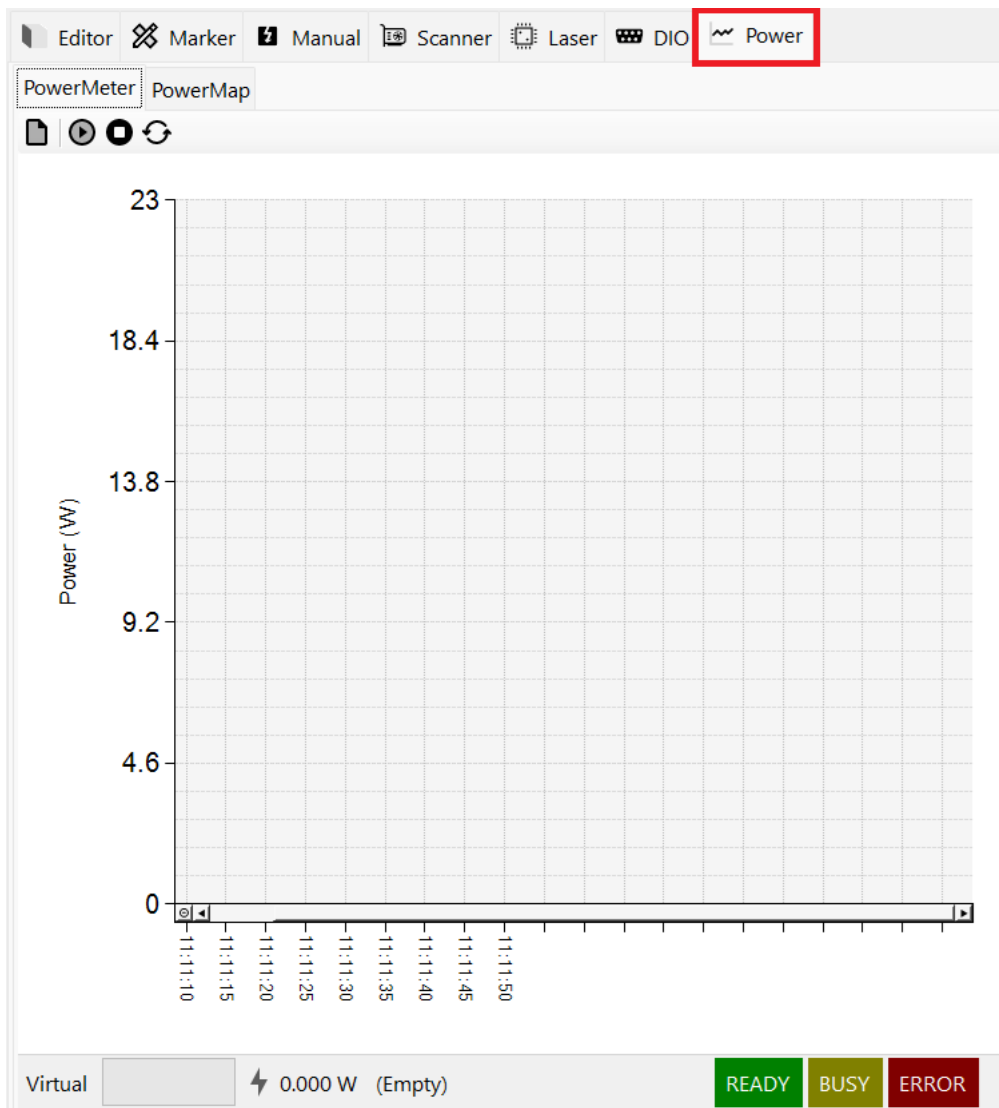
Editor Marker Manual Scanner Laser DIO Power					
No	DIN EXTENSION1 PORT	Status	No	DOUT EXTENSION1 PORT	Status
0	D0	OFF	0	D0	OFF
1	D1	OFF	1	D1	OFF
2	D2	OFF	2	D2	OFF
3	D3	OFF	3	D3	OFF
4	D4	OFF	4	D4	OFF
5	D5	OFF	5	D5	OFF
6	D6	OFF	6	D6	OFF
7	D7	OFF	7	D7	OFF
8	D8	OFF	8	D8	OFF
9	D9	OFF	9	D9	OFF
10	D10	OFF	10	D10	OFF
11	D11	OFF	11	D11	OFF
12	D12	OFF	12	D12	OFF
No	DIN LASER PORT	Status	No	DOUT EXTENSION2 PORT	Status
0	D0	ON	0	D0	OFF
1	D1	ON	1	D1	OFF
			2	D2	OFF
			3	D3	OFF
			4	D4	OFF
			5	D5	OFF
No	DOUT LASER PORT	Status			
0	D0	OFF			
1	D1	OFF			

D.IO 탭으로 이동하면 위와 같이 시각적으로 상태를 확인할 수 있으며, 우측 출력 포트의 경우 마우스 더블 클릭을 사용해 수동으로 출력을 On/Off 할 수 있습니다. 입력 포트의 특정 비트가 상태가 변경되어 이를 처리하는 등의 루틴이 필요하다면 `editor_dio` 예제 프로젝트를 참고해 주시기 바랍니다.

14. 파워미터(PowerMeter)와 파워맵(PowerMap)

파워미터는 발진하는 레이저 소스의 에너지를 측정하는 장치로 다양한 벤더의 제품들이 제공됩니다. 이를 위해 **IPowerMeter** 인터페이스가 제공되며 이를 상속 구현해 파워미터 장치를 구현합니다. 다음과 같이 생성 및 초기화하고 편집기에 **IPowerMeter** 속성에 지정해 사용합니다.

```
var powerMeter = PowerMeterFactory.Create... ;
powerMeter.Initialize();
```



파워미터 장치 측정을 시작하고, 종료 및 측정된 데이터를 주기적으로 그래프로 출력하는 것은 **Power** 탭에서 사용자가 버튼을 눌러 처리할 수 있습니다.

통상 파워미터를 이용하는 이유는 지령된 값(설정 파워)과 실제 취득된 값(측정 파워)의 편차가 발생하기 때문에 이를 보상(Compensate) 하기 위해 파워맵(IPowerMap) 을 사용합니다. 라이브러리에서는 다음과 같이 파워맵 객체를 생성하고 카테고리를 생성해 사용이 가능합니다. 또한 레이저 소스는 **ILaserPowerControl** 인터페이스를 상속구현하게 되면 **PowerMap** 속성이 존재하며, 이를 사용해 레이저 소스와 파워맵을 서로 연결시켜 사용할수 있습니다.

```
var powerMap = PowerMapFactory.CreateDefault(0, "default");
powerMap.Reset1to1("10000", laserMaxPower);

// or 기존 파일이 있다면 PowerMapSerializer.Open 을 이용해 매핑 데이터를 로드 합니다.
```

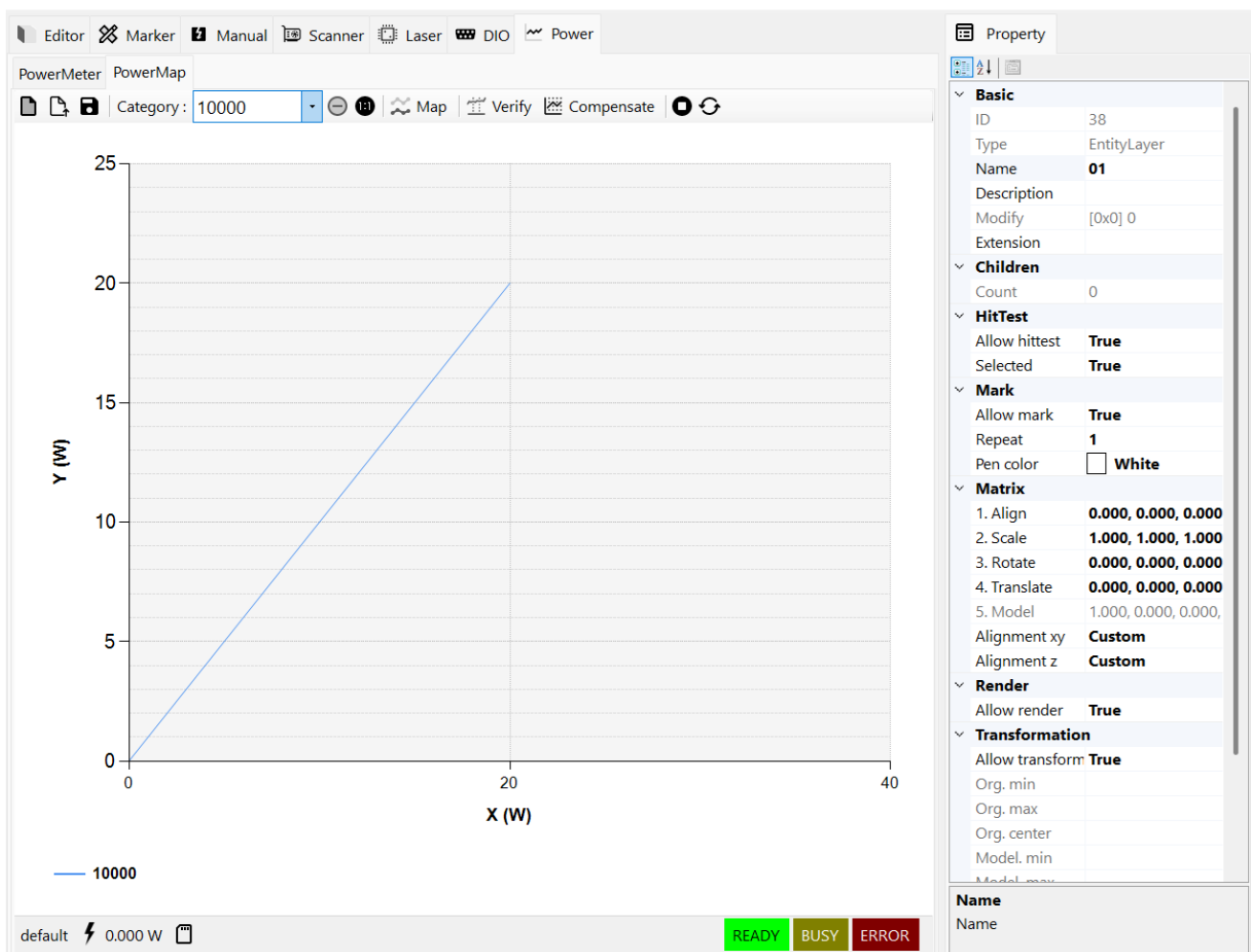
```
laser.PowerMap = powerMap;
```

파워맵의 주요 기능은 다음과 같습니다.

Mapping: 여러 카테고리/목표 X 파워 집합에 대해 파워 맵핑을 시작합니다.

Verify : 지정된 카테고리별 목표 Y 파워를 사용해 검증을 수행합니다

Compensate: 카테고리/목표 Y(W)에 맞추기 위해 보상(재보정)을 수행합니다. 허용 범위를 벗어나면 재시도/테이블 갱신을 진행합니다



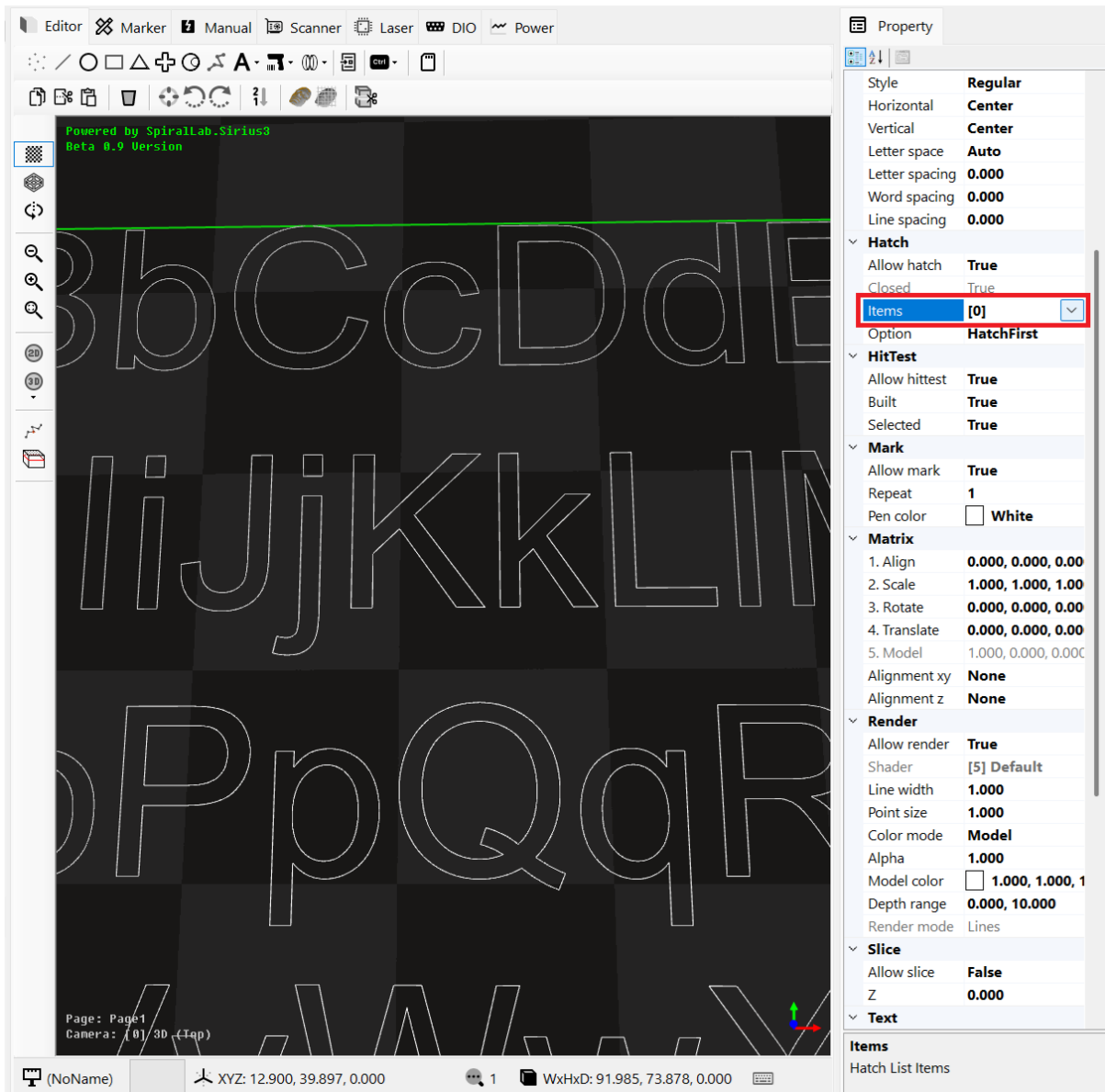
이 같은 기능은 PowerMap 탭에서 사용자가 버튼을 눌러 처리할 수 있습니다. 또한 PowerMapSerializer.Open, PowerMapSerializer.Save 함수로 파일 저장 및 파일 열기가 가능합니다. 파워맵의 자세한 구현 사항은 editor_powermap 예제 프로젝트에 공개되어 있습니다. 이 코드를 수정해 사용자가 원하는 다양한 방식으로 구현

2026 COPYRIGHT TO ©SPIRALLAB. <https://spirallab.co.kr>

및 확장이 가능합니다.

15. 해치(Hatch)

폐곡선² 영역을 가지는 개체들에 대해서는 내부 영역을 채우는 해치를 추가할 수 있습니다. 예를 들어 텍스트(Text) 개체에 대해 해치를 추가하기 위해서는 아래와 같이 Hatch 항목의 Items 버튼을 눌러 해치를 추가/삭제/수정이 가능합니다.

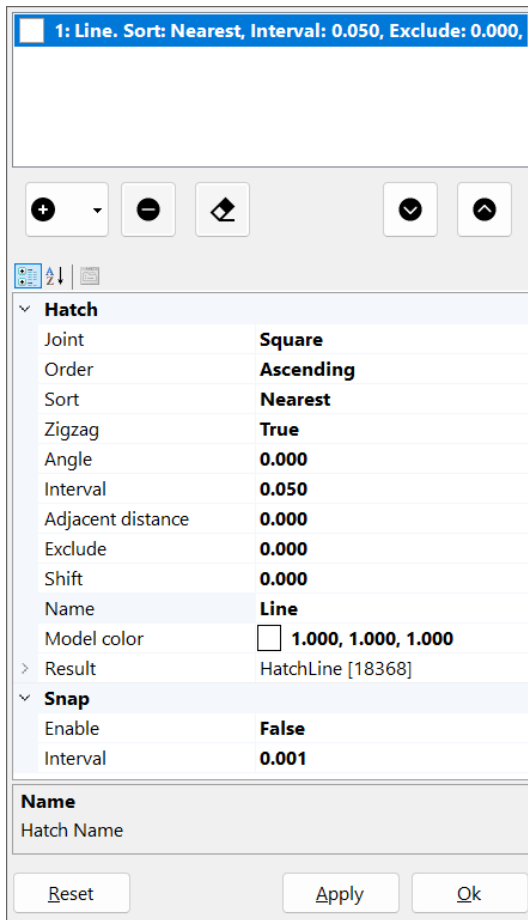


15.1 선분 해치

만약 선분(Line) 해치를 추가할 경우 ‘+’ 버튼을 눌러 Line 해치를 추가하고 적용(Apply)을 누르면 미리보기가 가능해집니다. 또한 폴리곤(Polygon) 해치 모양도 지원되며, 여러 개의 선분, 폴리곤 해치를 무제한으로 추가, 삭

² 원, 폴리 라인(IsClosed 값이 True), Text 과 같이 닫힌 경로를 가진 객체들

제가 가능합니다. 또한 위, 아래 이동 버튼을 이용해 해칭들 간의 가공 순서를 이동시킬 수 있습니다.



선분(Line) 해치 옵션 설명

- **Hatch Joint**: 불록한 각도의 접합부에서 오프셋을 관리하는 방식을 지정합니다. 오목한 접합부는 항상 **Miter** 접합으로 오프셋 됩니다.
- **Square**: 사각형 조인트 유형. 불록한 접합부는 '스퀘어링' 모서리로 잘립니다. 그리고 이 스퀘어링 모서리의 중간점은 원래(또는 시작) 정점에서 정확히 오프셋 거리만큼 떨어져 있습니다.
- **Round**: 둥근 조인트 유형. 모든 불록한 조인에 반경이 오프셋 거리인 원호가 적용되며, 원래 조인 정점이 원호의 중심이 됩니다.
- **Miter**: 마이터 조인트 유형. 가장자리는 먼저 원래(즉 시작) 가장자리 위치에서 지정된 거리만큼 평행하게 오프셋 됩니다. 이 오프셋 된 가장자리는 인접한 가장자리 오프셋과 교차하는 지점까지 연장됩니다.

Hatch Order: 선분 생성 순서를 지정합니다.

- **Ascending**: 오름차순 정렬됩니다.
- **Descending**: 내림차순 정렬됩니다.

Hatch Sort: 선분들 간의 정렬 상태를 지정합니다.

- **None**: 생성된 순서를 그대로 이용합니다. (정렬 없음)
- **Nearest**: 가장 가까운(인접한) 선분을 탐색하여 정렬합니다.
- **Global**: 선분들을 전역적으로 경로 최적화를 통해 정렬합니다.

Hatch ZigZag: 선분 생성시 지그재그 형태로 생성할지 여부를 지정합니다.

Hatch Angle: 선분 생성시 회전 각도를 지정합니다.

Hatch Interval: 선분 생성시 선분간 간격을 지정합니다. (0 보다 커야 합니다)

Hatch Adjacent Distance: 선분간 최소 인접 거리를 지정합니다. (0 인 경우 사용되지 않습니다. **Sort** 가 설정되었을 때에만 사용됩니다)

Hatch Exclude: 제외할 영역에 대한 외곽 거리 값입니다. 폐곡선이 해당 값만큼 축소된 후 해치 생성이 이루어집니다.

Hatch Shift: 선분 생성시 해당 거리 값만큼 이동(오프셋) 시켜 생성됩니다.

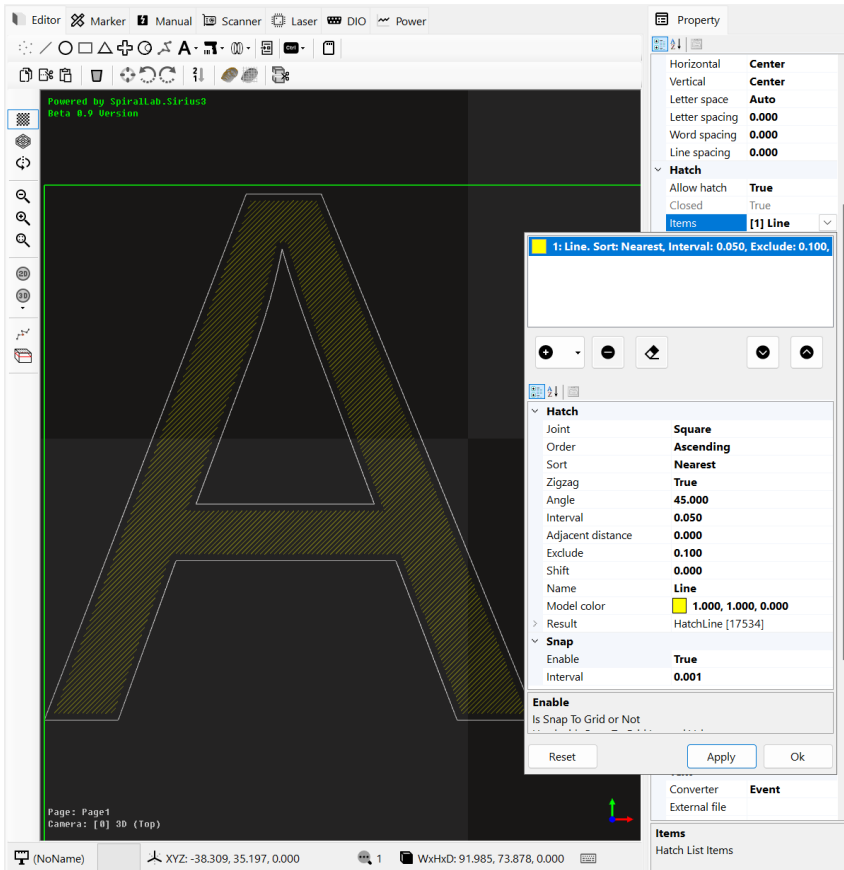
Hatch Model color: 해치에 사용할 펜 색상(Pen color)을 지정합니다. 색상 변경을 통해 레이어 가공 시 서로 다른 스캐너 펜(Scanner pen) 속성들을 적용해 가공할 수 있습니다.

Hatch Result: 내부적으로 생성된 결과 저장용 객체입니다.

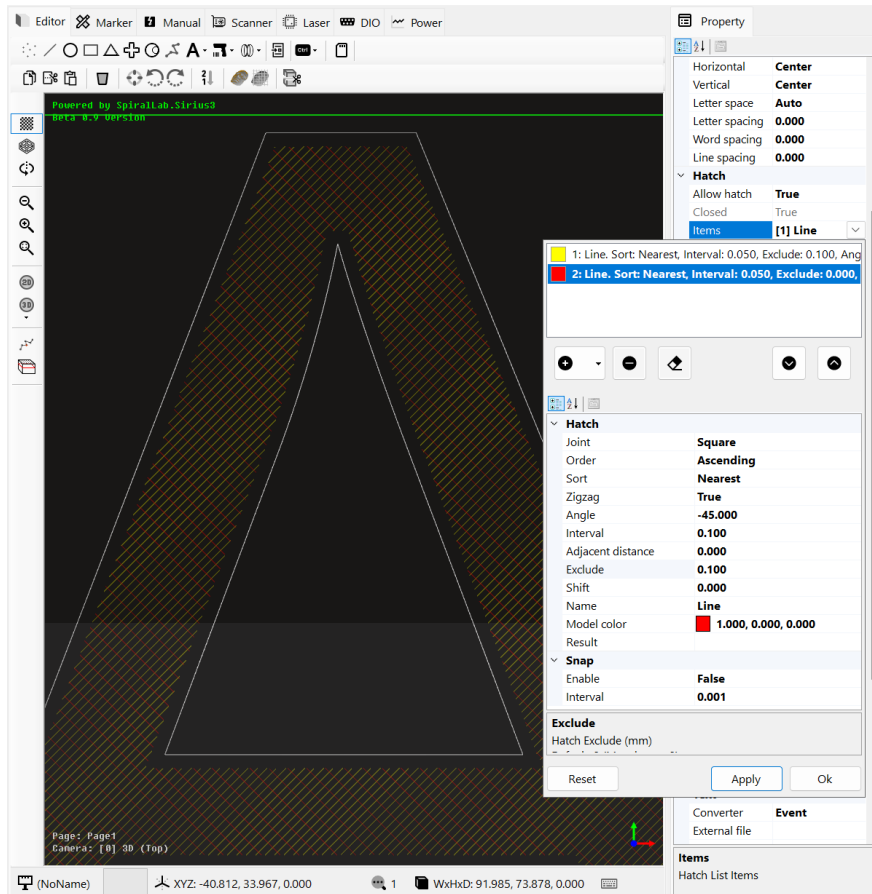
Snap Enable: 선분 생성시 **Snap Interval** 값의 배수 간격 위치에 생성시킬지 여부입니다. (**Snap to Grid**를 의미합니다)

Snap Interval: 스냅 간격에 대한 거리 값입니다. (스냅 **Snap Enable** 이 **True** 일 때만 사용됩니다)

생성된 선분 해치를 예제를 보면,

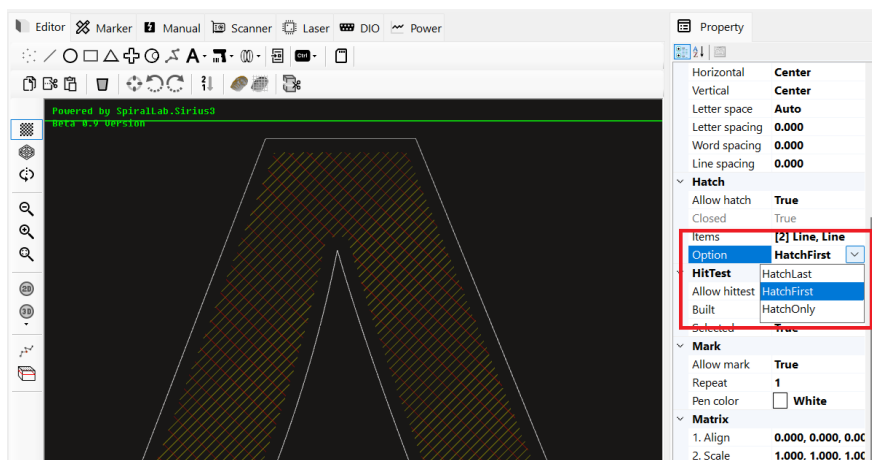


Angle: 45도, Exclude: 0.1mm, 펜
색상: 노랑 등으로 생성한 예



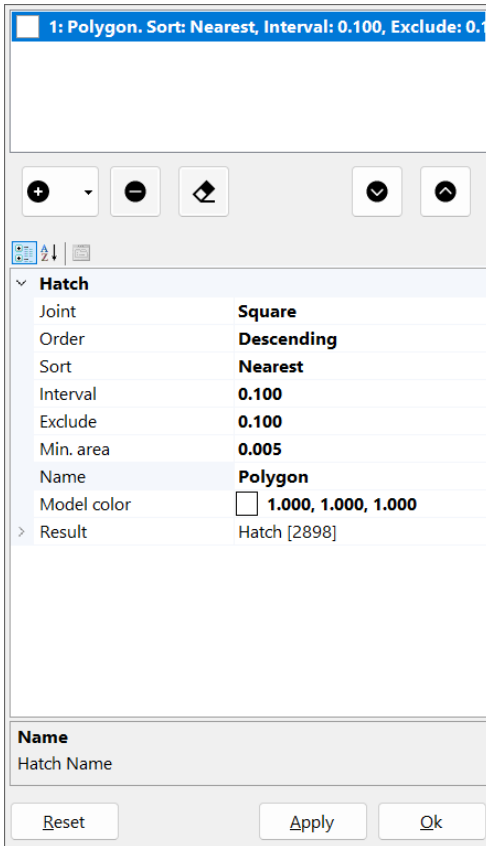
추가적으로 Angle: -45도, 펜 색 상: 빨강 등을 추가한 예

Apply(혹은 Ok)를 통해 이렇게 생성된 해치는 아래와 같이 2 개의 선분 해치들로 구성된 것을 확인할 수 있습니다.



또한 Hatch Option을 통해 외곽 + 해치의 가공 방식(해치를 먼저 가공할지, 외곽선을 먼저 가공하지 등) 변경할 수 있습니다.

15.2 폴리곤 해치



폴리곤(Polygon) 해치 옵션 설명

- **Hatch Joint**: 불록한 각도의 접합부에서 오프셋을 관리하는 방식을 지정합니다. 오목한 접합부는 항상 **Miter** 접합으로 오프셋 됩니다.
- **Square**: 사각형 조인트 유형. 불록한 접합부는 '스퀘어링' 모서리로 잘립니다. 그리고 이 스퀘어링 모서리의 중간점은 원래(또는 시작) 정점에서 정확히 오프셋 거리만큼 떨어져 있습니다.
- **Round**: 둥근 조인트 유형. 모든 불록한 조인에 반경이 오프셋 거리인 원호가 적용되며, 원래 조인 정점이 원호의 중심이 됩니다.
- **Miter**: 마이터 조인트 유형. 가장자리는 먼저 원래(즉 시작) 가장자리 위치에서 지정된 거리만큼 평행하게 오프셋 됩니다. 이 오프셋된 가장자리는 인접한 가장자리 오프셋과 교차하는 지점까지 연장됩니다.

Hatch Order: 폴리곤 생성 순서를 지정합니다.

- **Ascending**: 오름차순 정렬됩니다. (내부에서 외부 순서로)
- **Descending**: 내림차순 정렬됩니다. (외부에서 내부 순서로)

Hatch Sort: 폴리곤들 간의 정렬 상태를 지정합니다.

- **None**: 생성된 순서를 그대로 이용합니다. (정렬 없음)
- **Nearest**: 가장 가까운(인접한) 폴리곤을 탐색하여 정렬합니다.
- **Global**: 폴리곤 들을 전역적으로 경로 최적화를 통해 정렬합니다.

Hatch Interval: 폴리곤 생성시 폴리곤 간 간격을 지정합니다. (0 보다 커야 합니다)

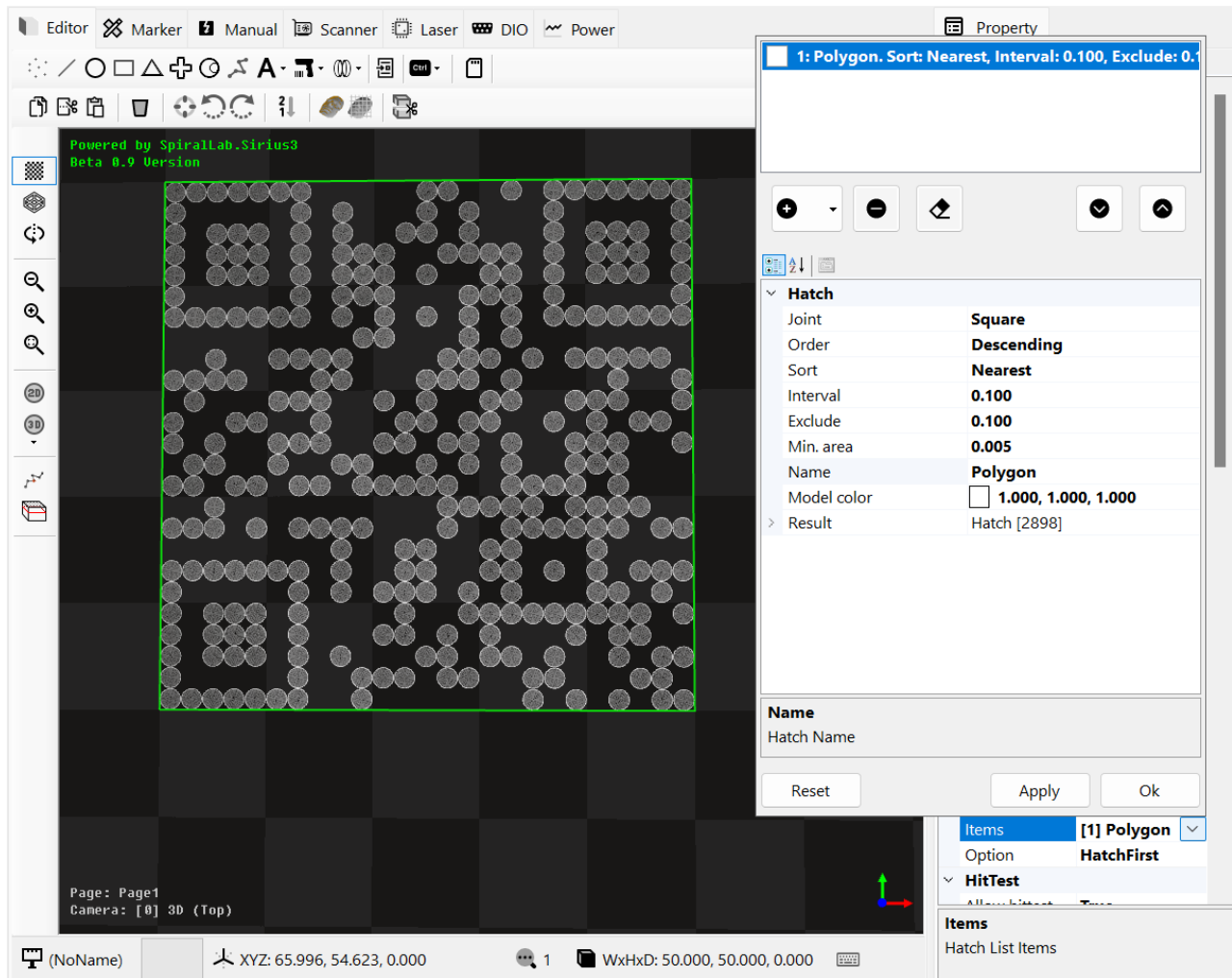
Hatch Exclude: 제외할 영역에 대한 외곽 거리 값입니다. 폐곡선이 해당 값만큼 축소된 후 해치 생성이 이루어집니다.

Hatch Min. Area: 폴리곤 생성시 제외한 최소 면적 크기입니다.

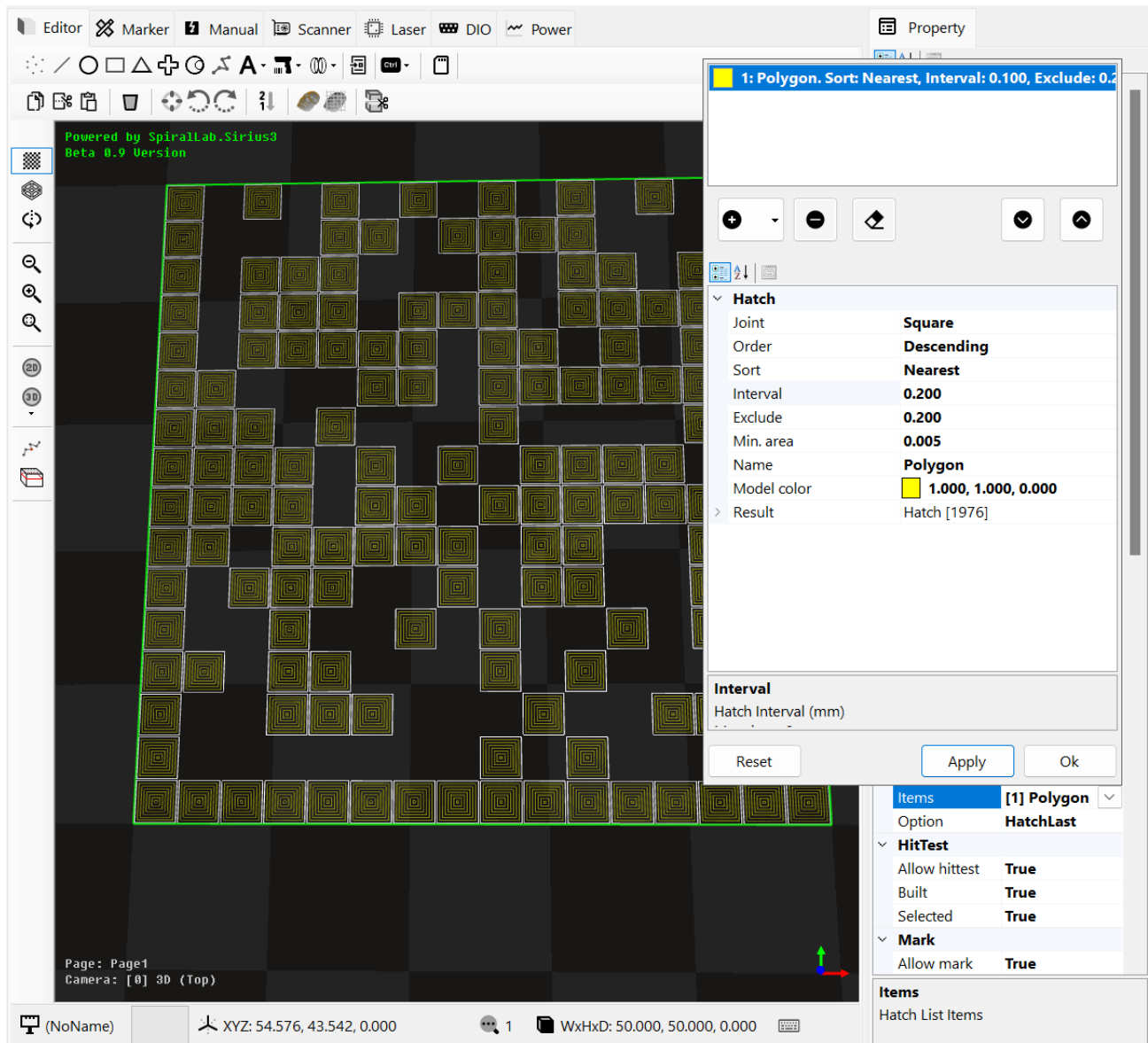
Hatch Model color: 해치에 사용할 펜 색상(Pen color)을 지정합니다. 색상 변경을 통해 레이저 가공시 서로 다른 스캐너 펜(Scanner pen) 속성들을 적용해 가공할 수 있습니다.

Hatch Result: 내부적으로 생성된 결과 저장용 객체입니다.

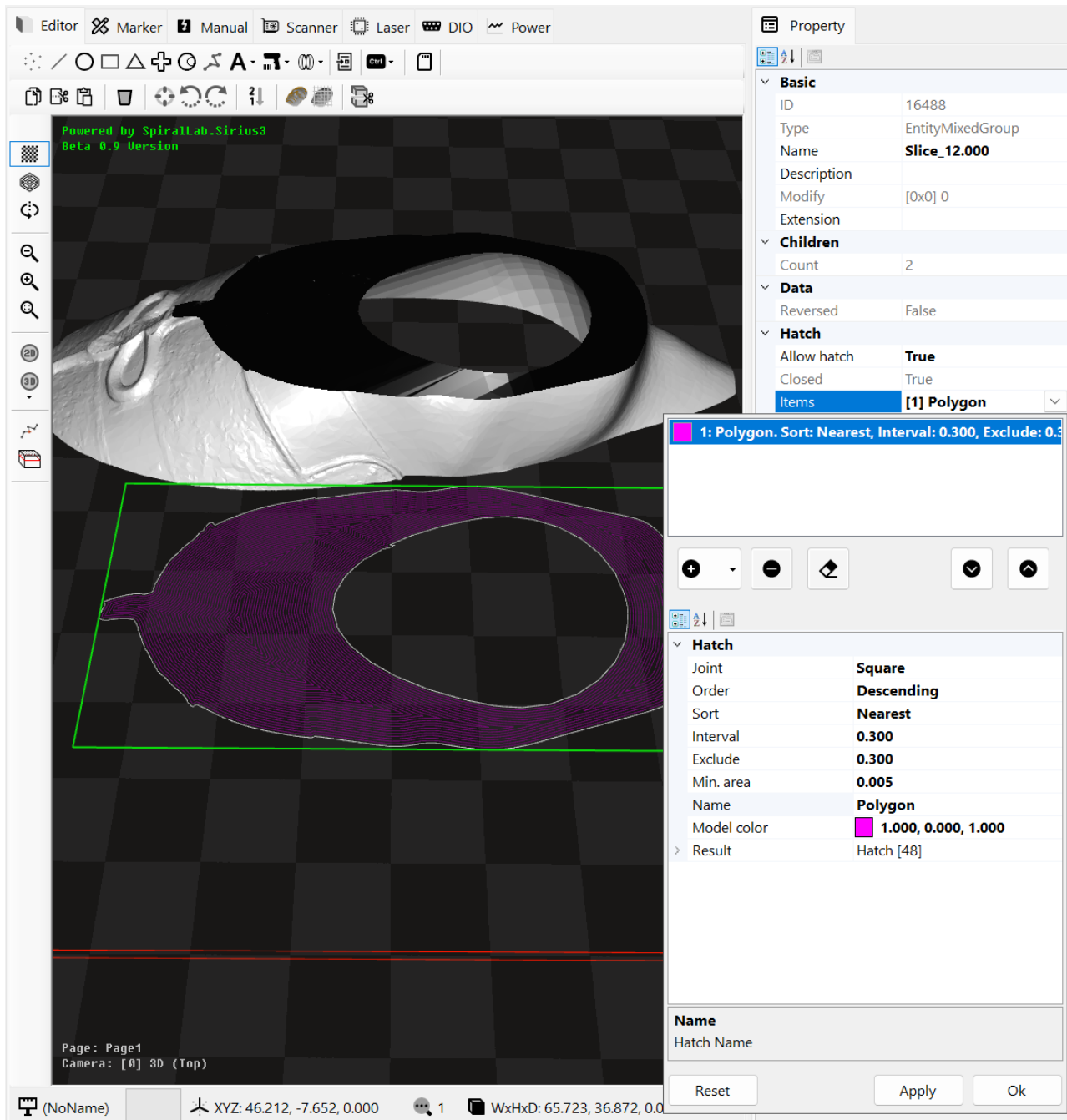
생성된 폴리곤 해치를 예제를 보면,



원 형태의 셀(Cell Circle)로 구성된 QR 바코드에 대해서 Descending 내림차순 (외부에서 내부로), Interval 은 0.1 mm 등으로 생성한 예



사각형 형태의 셀(Cell Square)로 구성된 DataMatrix 바코드에 대해서 Descending 내림차순 (외부에서 내부로), Interval 은 0.1 mm 등으로 생성한 예



메쉬(Mesh)에 대해 Slice 후 생성된 2개의 폴리 라인(EntityPolyline2D)에 대해 폴리곤 해치를 적용한 모습

15.3 코드를 통한 해치 처리

해치를 지원하는 개체는 IHatchable 인터페이스를 상속 구현하고 있고, HatchFactory에서 제공하는 함수를 이용하면 다음과 같이 해치 객체를 생성 및 추가할 수 있습니다.

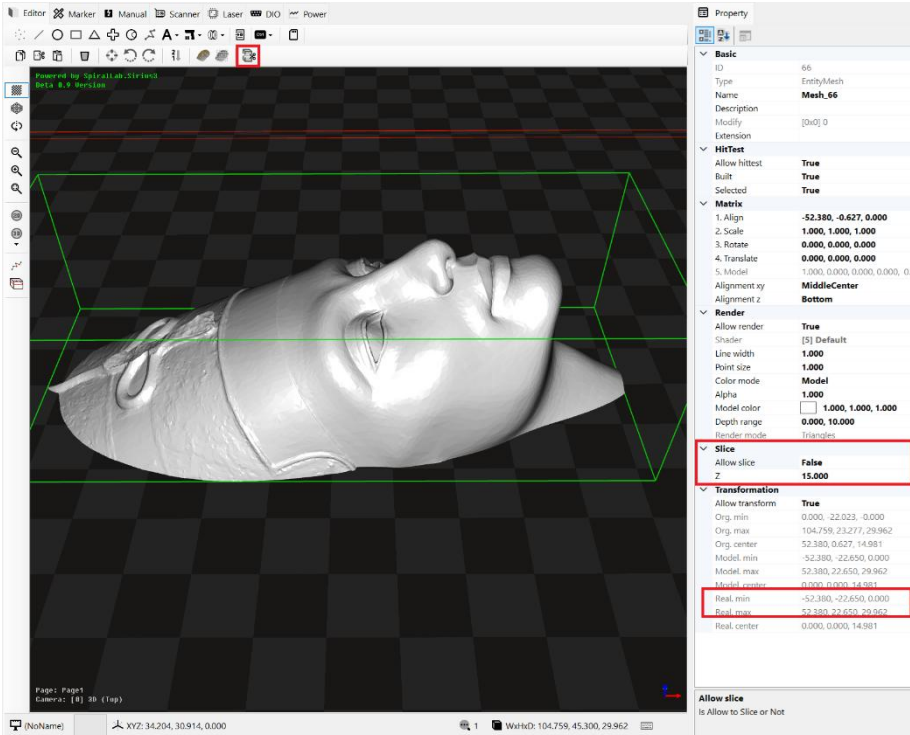
```
var hatch = HatchFactory.CreateLine(...);
//var hatch = HatchFactory.CreatePolygon(...);
hatch.ModelColor = Color.White;

var hatchable = entity as IHatchable;
hatchable.AddHatch(hatch); // hatchable.Hatches 에 해치 객체 정보가 추가됨
document.ActRegen();
```

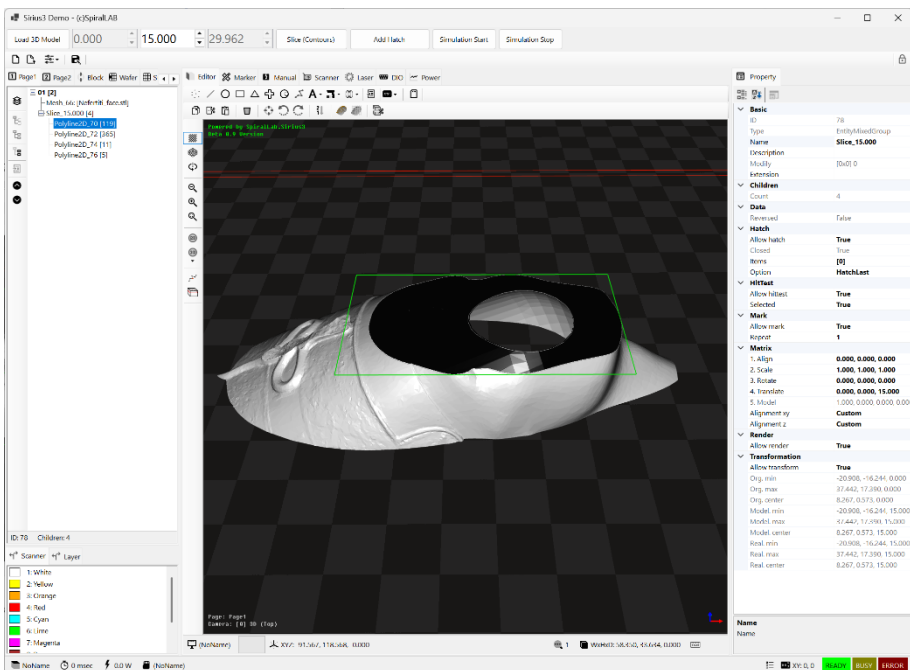
(참고) 자세한 사용법은 `editor_hatch` 예제 프로젝트의 코드를 참고해 주시기 바랍니다.

16. 슬라이서 (Slicer)

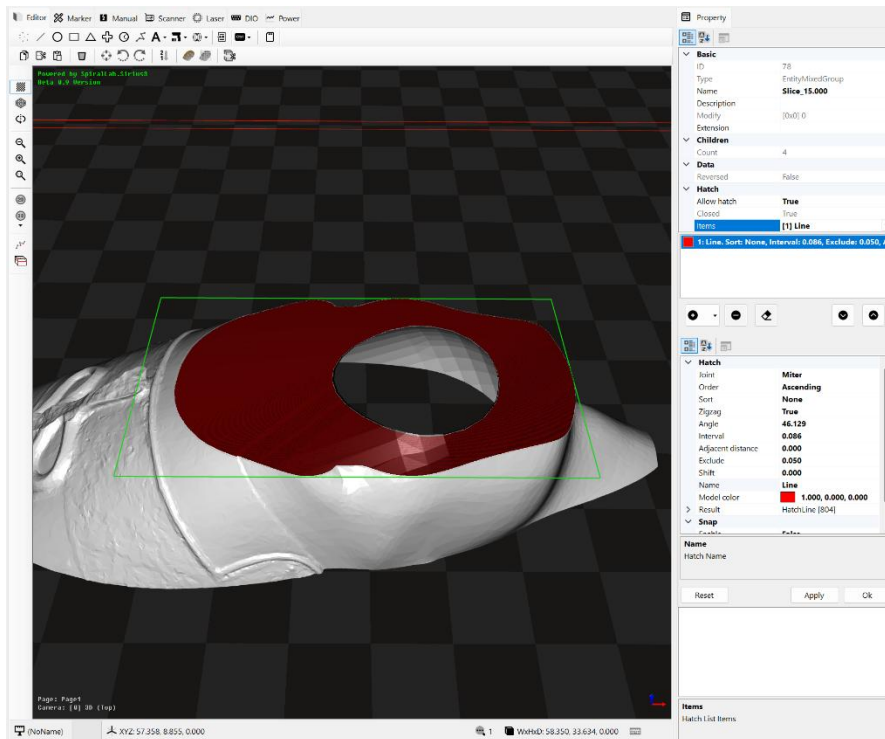
파일 가져오기(Import)를 통해 STL, OBJ, PLY 과 같은 3D 메쉬 모델(Mesh Model)을 가져와 특정 Z 위치로 평면을 자르고 그 경로(Contour)를 추출할 수 있습니다. 예를 들어 아래와 같은 메쉬 객체를 Import 하면 Z=0 위치에 바닥면이 자동으로 정렬(Align) 되어 로딩됩니다.



이후 Slice Z 값을 15mm 로 설정(Real. Min ~ Real. Max 의 Z 범위 내에 해당하는지 확인) 하고, Allow slice 항목을 활성화(True) 하게 되면, 지정된 Z=15 평면에서 메쉬가 잘린채 렌더링됩니다. 이때 Slice Mesh 버튼을 누르게 되면 해당 메쉬에 지정된 Z 평면위치를 사용해 잘린 평면을 구성하는 경로(Contour)들을 추출하고, 이를 폴리라인(PolyLine) 들로 모아 하나의 그룹 개체를 생성해 추가하게 됩니다.



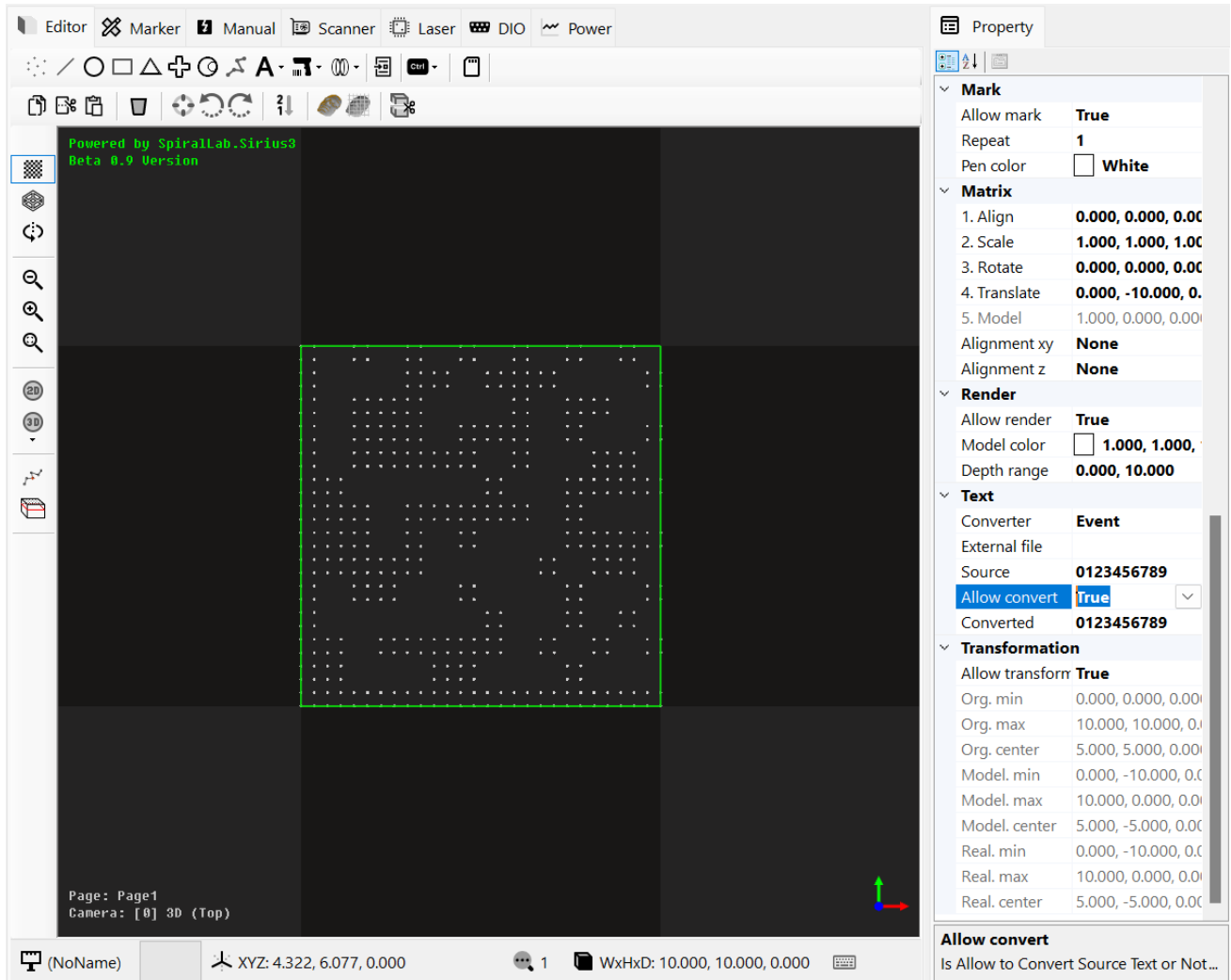
Slice Mesh 기능으로 평면으로 절단 후 생성된 Contour 그룹 개체



Contour 그룹 개체에 해치(Hatch)를 적용한 모습으로, 3D 메쉬에 Slice Z 위치값을 변경하면서 절단면에 해치를 적용하고 이를 레이저 가공하는 방식을 반복적으로 활용하면 3D 레이저 프린터 가공 방식인 AM(Additive Manufacturing)이 가능해집니다.

17. 텍스트 데이터 변환 (Text Converter)

텍스트 데이터 변환을 지원하는 개체들³은 레이저 가공 직전 최종 데이터 변환이 적용됩니다. 이를 위해 아래와 같이 **Allow Convert** 기능을 활성화하고 변환기(Converter)를 선택해야 합니다. 또한 입력된 데이터는 **Source**, 최종 변환된 데이터는 **Converted** 항목에 반영되어 출력됩니다.



텍스트 데이터 변환기(Converter)는 4가지가 제공됩니다.

17.1 이벤트 (Event)

이벤트는 라이브러리 내부에서 제공되는 **IMaker.OnTextConvert** 이벤트에 함수를 등록해 사용하는 방식입니다. 이벤트 핸들러 함수에서 리턴(return)된 문자열을 사용하는 방식으로, 해당 코드를 직접 구현해 사용하므로, 복잡한 구현이 가능합니다.

코드 구현 예:

```
var marker = siriusEditor1.Marker;
marker.OnTextConvert += OnTextConvert;
...
```

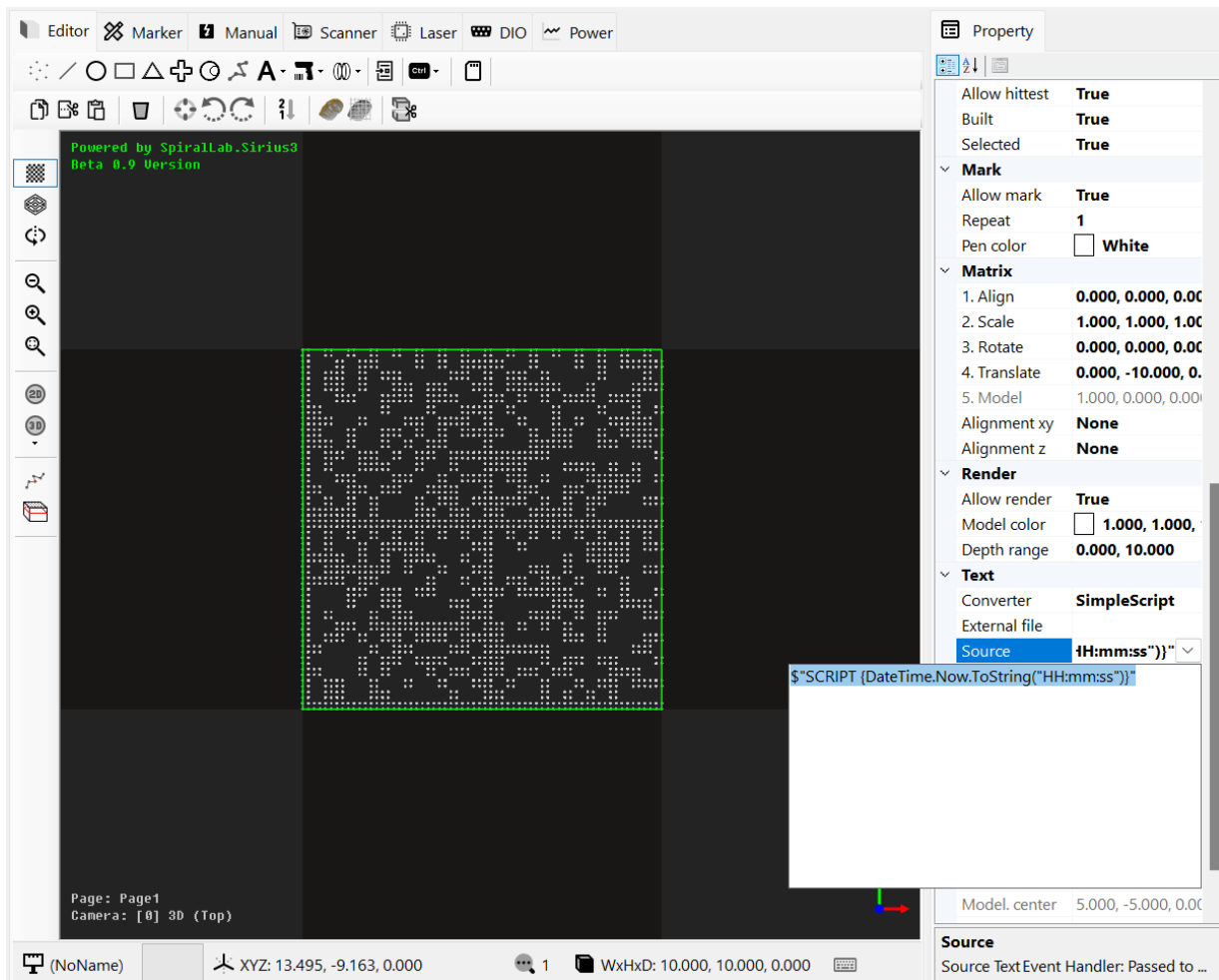
³ 텍스트, 바코드 등의 개체(EntityText, EntitySiriusText, EntityCircularText, EntityBarcode1D, EntityQRCode, EntityDataMatrix 등)들과 같이 **ITextConvertible**를 상속 구현하는 객체들


```
string OnTextConvert(IMarker marker, ITextConvertible textConvertible)
{
    var entity = textConvertible as IEntity;
    switch (entity.Name)
    {
        case "MyBarcode":
            return $"EVENT {DateTime.Now.ToString("HH:mm:ss")}";
        default: // Not modified
            return textConvertible.SourceText;
    }
}
```

17.2 단순 스크립트 (Simple Script)

스크립트 변환기는 입력 데이터에 간이 C# 스크립트 코드를 입력하고, 레이저 가공 직전에 해당 스크립트 코드가 실행되어 리턴(return)된 문자열을 사용하는 방식입니다. 아래와 같이 매우 단순한 코드만 사용이 가능합니다.

C# 스크립트 구현 예: `$"SCRIPT {DateTime.Now.ToString("HH:mm:ss")}"`



17.3 파일 (File)

외부 텍스트 파일을 지정하고 그 내용을 사용하는 방식입니다. 레이저 가공 직전에 해당 텍스트 파일의 가장 첫번째 라인을 읽어 문자열을 레이저 가공에 사용하는 방식입니다. (해당 문자열은 자동으로 삭제됩니다)

때문에 파일 변환기 사용시에는 반드시 외부파일(External File)을 미리 지정해 두어야 합니다. 또한 파일의 문자열이 모두 소진(삭제)되면, 더 이상 가공이 진행되지 않습니다.

17.4 오프셋 (Offset)

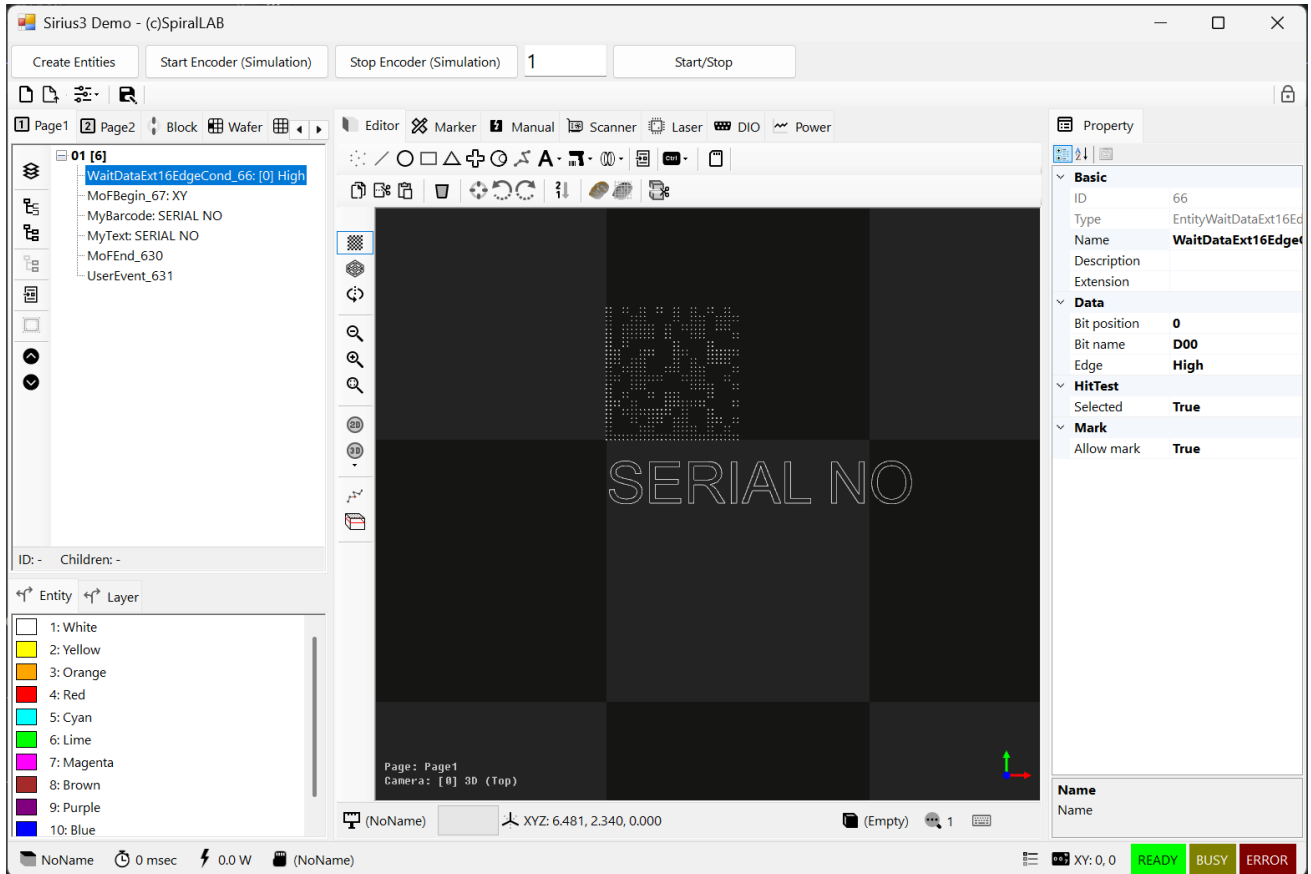
마커에서 가공 시작시 오프셋(Offset) 배열을 전달하면, 여러 위치에 반복적인 가공이 가능합니다. 가공이 진행중 오프셋(Offset) 객체의 `ExtensionData` 속성에 저장된 값을 읽어 텍스트 변환 가공해 사용하는 방식입니다.

(주의) 리턴(return) 된 문자열이 없으면 (null 혹은 `string.Empty`) Marker 의 가공이 중단됩니다.

(참고) 자세한 사용방법은 `editor_barcode_textconvert` 예제 프로젝트의 코드를 참고해 주시기 바랍니다.

18. 마킹 온더 플라이(Marking On the Fly)

MOF (혹은 PoF:Processing On the Fly) 기능은 외부 물체의 움직임에 대한 엔코더 신호를 입력받아 스캐너가 실시간으로 이동량을 보상하면서 가공하는 기능입니다. 여기에서는 `editor_mof_trigger` 샘플 프로그램을 이용한 사용방법을 설명해 보겠습니다.



우선 **Create Entities** 를 이용해 트리거 대기, MoF 시작, 바코드, 텍스트, MoF 끝, 사용자 이벤트 개체들을 생성하게 됩니다. 이때 트리거 신호는 RTC 카드의 확장1 포트의 **D.IN0** 신호핀이 **High** 로 토글될때까지 대기하는 조건을 위해 생성됩니다. 외부 트리거가 신호가 입력되면 즉시 엔코더 값이 초기화(리셋)되고 동시에 **MoF** 시작으로 인해 스캐너의 위치는 실시간으로 외부 엔코더 증감량을 추종하게 됩니다. 또한 텍스트 데이터 변경은 자체적인 이벤트 핸들러에 의해 변환된 데이터로 내부 리스트 버퍼에 저장되며, **MoF** 끝에 의해 더 이상 엔코더 증감량을 추종하지 않고 원점위치로 점프합니다. 이후 사용자 이벤트 시점에 텍스트 데이터, 여기에서는 일련번호를 증가시킵니다.

데모에서는 레이어의 반복 가공 회수를 **100** 회로 설정하였고, 이를 마커(**IMarker**)를 이용해 가공을 시작(**Start**)하게 되면, 다음과 같이 진행됩니다. RTC 카드의 리스트 버퍼가 허용하는 만큼 트리거 대기, **MoF** 시작, 바코드, 텍스트, **MoF** 끝, 사용자 이벤트 실행(일련번호 변수값 증가) 순서로 명령들이 RTC 버퍼로 삽입됩니다. 이를 최대 **100**번(레이어에 설정된 반복 가공회수) 반복해야 하나, 실제로는 RTC 내부 버퍼가 준비될때까지 명령어들은 삽입된 순서대로 조금씩 처리되고, 나머지 명령들은 대기 되는 방식입니다. 이때 외부 트리거(**D.IN0**)가 **High** 신호로 전환(토글)되면 버퍼에 있는 가공명령들이 빠져나가면서 레이저 출사가 시작됩니다.

즉 리스트 버퍼에 삽입된 명령들과 실제 가공중인 데이터(좌표 등)간에는 시간차이가 존재하며, 편집기 화면상에 데이터(바코드 및 텍스트)는 마지막 버퍼에 추가된 상태를 의미합니다.

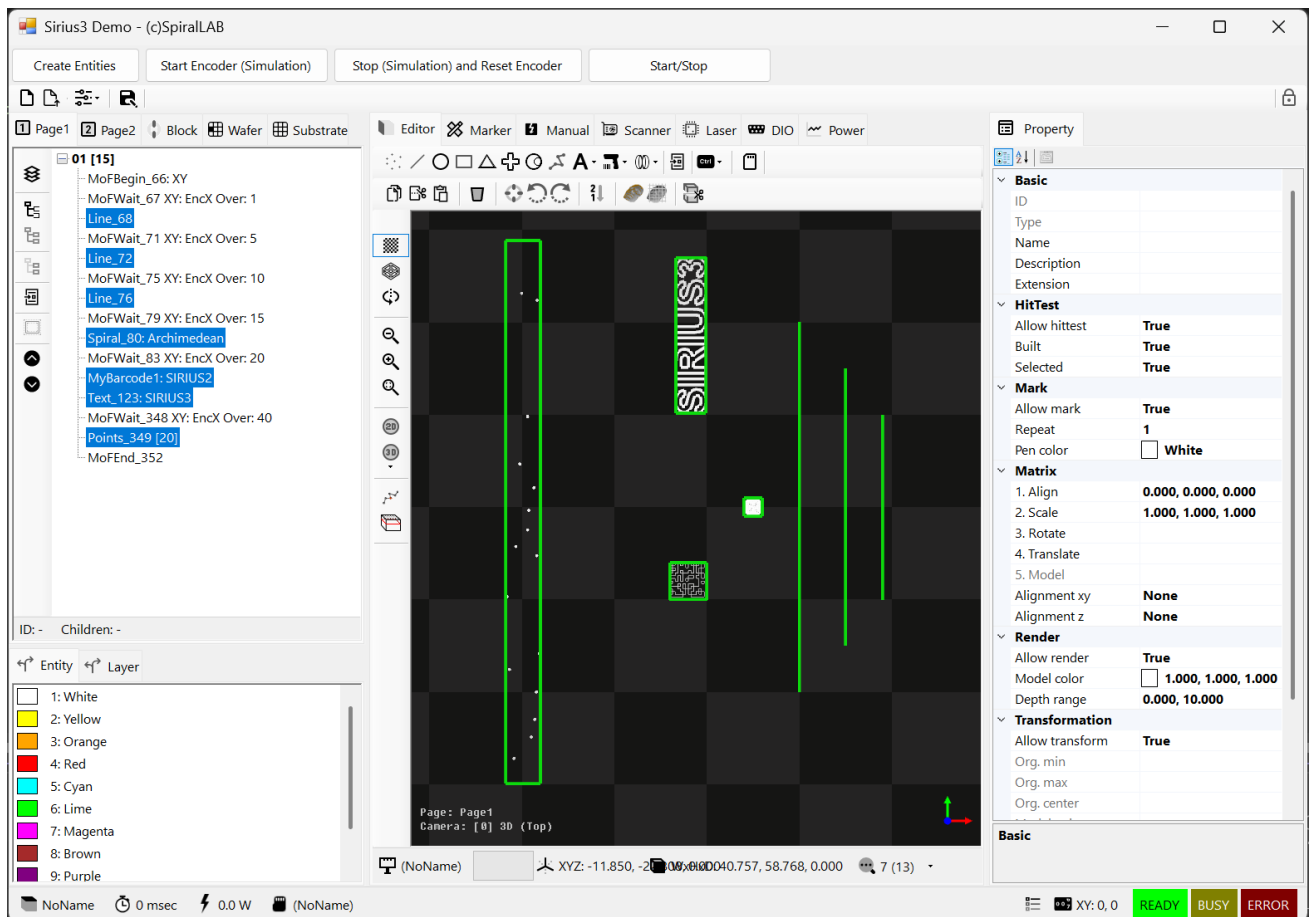
(참고) RTC 카드의 리스트 버퍼는 내부적으로 2개의 버퍼(더블버퍼)로 동작하게 됩니다.

(참고) RTC 제어기에 **Processing on the fly** 옵션 및 SIRIUS3 라이선스에 **MoF** 옵션이 필요합니다.

(참고) 외부 엔코더 신호가 연결되어 있지 않으면, 가상으로 엔코더 신호를 시뮬레이션 할 수 있습니다.

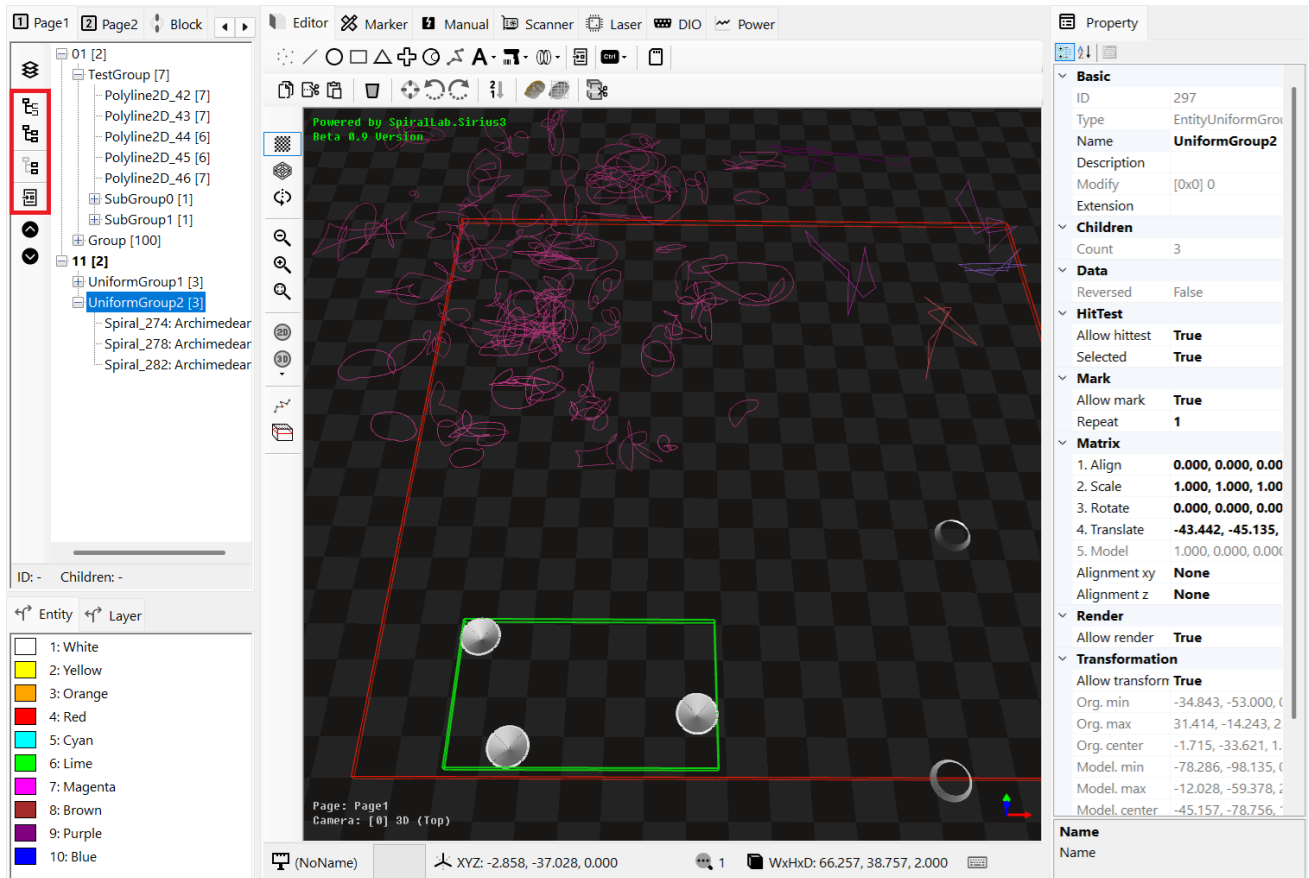
(참고) RTC 제어기에 단위 mm 당 엔코더 펄스 개수 설정이 미리 되어 있어야 합니다.

editor_mof_xy 샘플 프로그램에서는 외부 트리거(D.IN) 대신 엔코더의 위치를 대기하는 조건에 대한 사용방법을 확인할 수 있습니다.

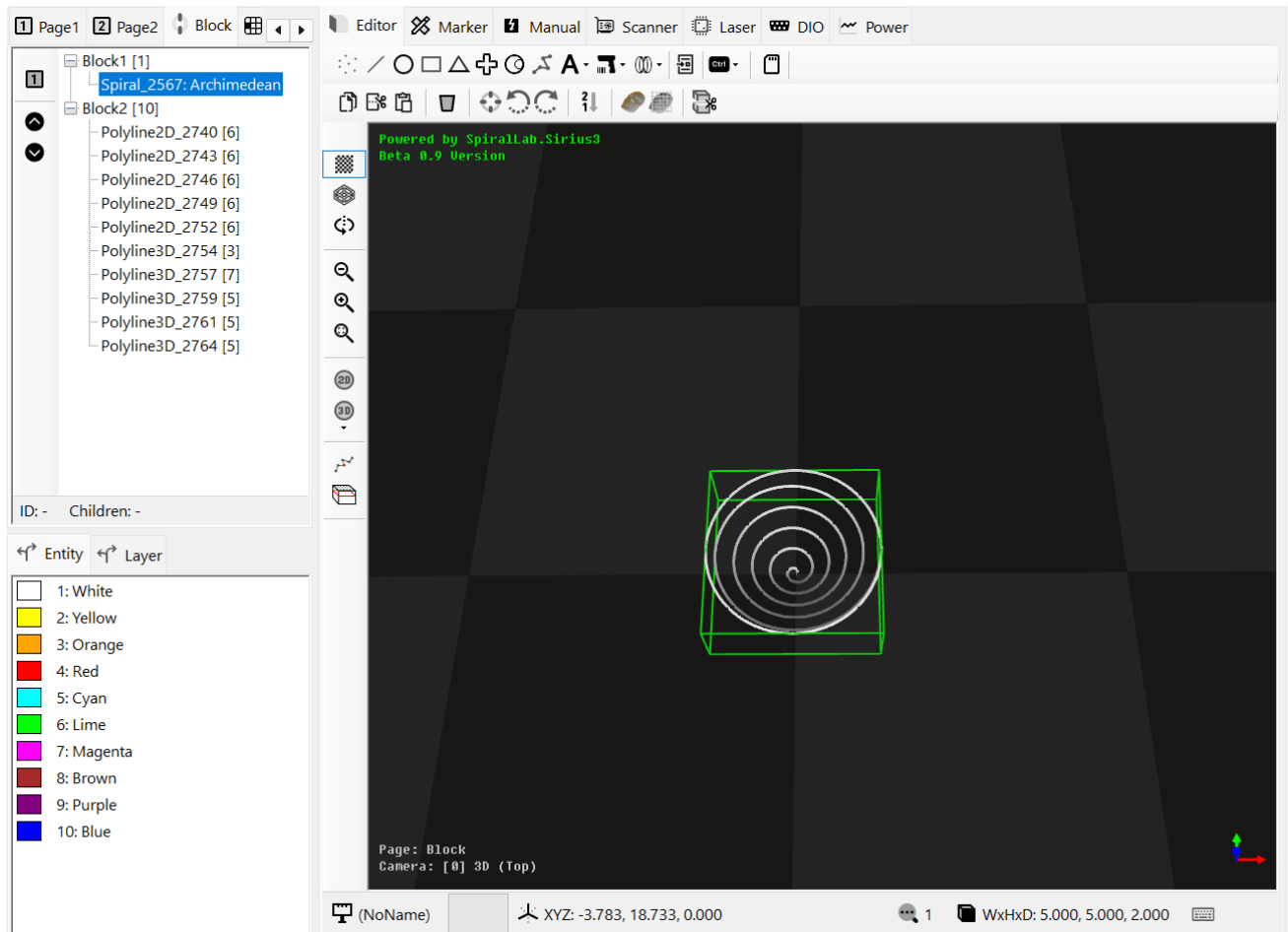


19. 그룹(Group) 및 블록(Block)

다양한 개체(IEntity)들을 하나의 그룹(EntityGroup)으로 생성하기 위해서 페이지 트리뷰 왼쪽에는 다양한 그룹 개체 생성 기능이 제공됩니다.



1. **Mixed Group**: 선택된 개체(IEntity)들을 하나의 혼합 그룹으로 변환 생성합니다.
(참고1) 트리뷰(TreeView)의 동일한 레벨(깊이)의 노드들만 허용됩니다.
(참고2) 레이어(EntityLayer), 펜(EntityPen) 등의 개체들은 혼합될 수 없습니다.
2. **Uniform Group**: 선택된 개체(IEntity)들을 하나의 단일 그룹으로 변환 생성합니다.
(참고1) 트리뷰(TreeView)의 동일한 레벨(깊이)의 노드들만 허용됩니다.
(참고2) 레이어(EntityLayer), 펜(EntityPen) 등의 개체들은 혼합될 수 없습니다.
(참고3) 단일 그룹은 고속 렌더링 (하나의 VAO 로 처리)을 위한 특수 목적으로 제공되기 때문에, 각 개체들이 모두 동일한 렌더링 모드(RenderMode)를 가져야 합니다.
3. **Ungroup**: 선택된 그룹 개체를 해체해 개별 개체들로 다시 복구합니다.
4. **Block**: 선택된 개체들을 블록(EntityBlock) 개체로 변환 생성 합니다.
(참고1) 레이어(EntityLayer), 펜(EntityPen) 등의 개체들은 혼합될 수 없습니다.
(참고2) 생성된 블록 개체는 Block 페이지에 생성되며, 추후 블록 삽입(BlockInsert)을 통해 다수의 참조 개체로 사용할 수 있습니다. BlockInsert 된 참조 개체들은 다양한 변환(크기, 회전, 이동 등) 적용이 가능합니다.

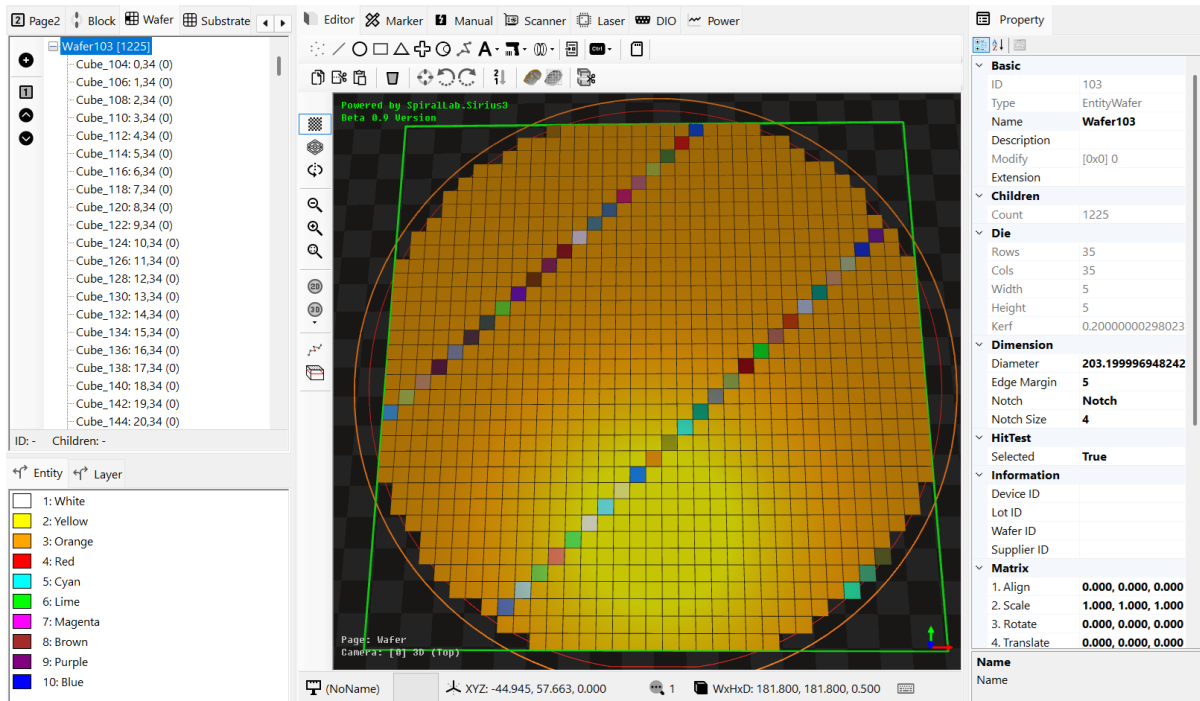


블록 페이지에서는 생성된 블록(EntityBlock) 목록이 나타나며, 그중에 제일 첫(상단) 위치에 있는 블록만 실제 화면에 렌더링 됩니다. 만약 다른 블록을 렌더링 시켜 편집 등의 작업이 필요하다면 좌측의 '1' 버튼 (Set as First or Top) 을 눌러 위치를 이동이 가능합니다.

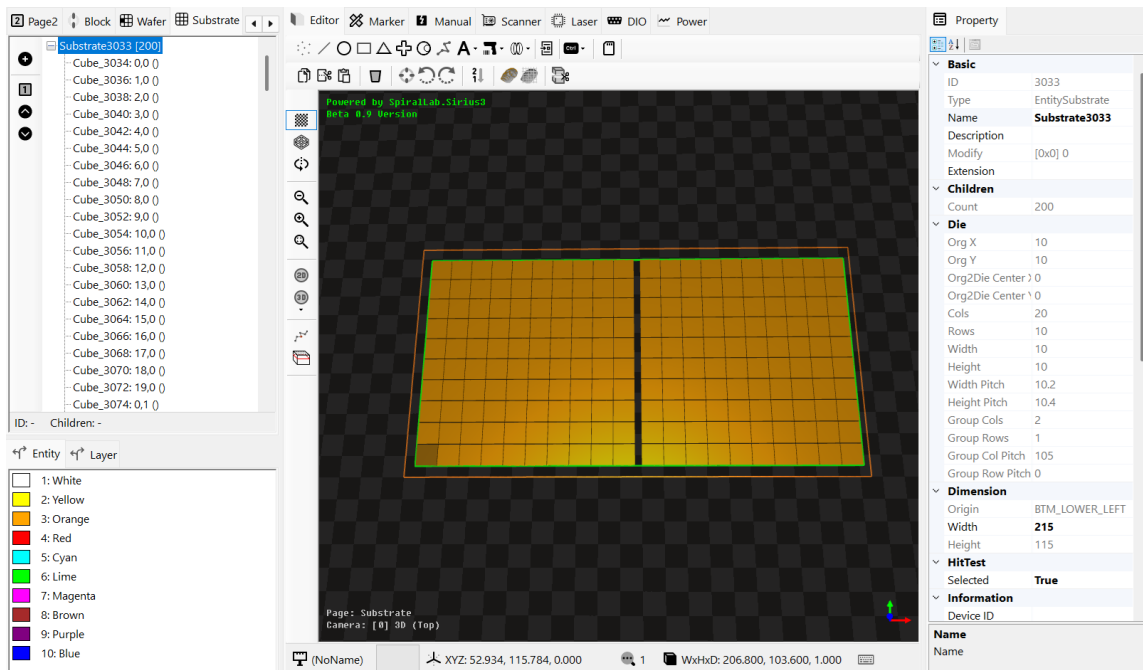
(주의) 블록 페이지에서 블록(EntityBlock) 개체를 삭제하면 이를 참고하고 있는 블록 삽입(EntityBlockInsert) 개체들이 무효화되기 때문에, 불필요해진 개체들을 모두 사용자가 직접 삭제해 주어야 합니다.

20. 웨이퍼(Wafer), 기판(Substrate) 맵(Map)

웨이퍼 (혹은 기판) 맵을 시각화하는 기능을 제공합니다. ‘+’ 버튼을 누르면 신규 웨이퍼(혹은 기판) 맵이 추가됩니다. 그러나 대부분 특정 크기 정보를 가진 Die 들을 직접 생성하는 것이 일반적이므로, Config.OnCreateWafer 이벤트에 핸들러 함수를 등록해 직접 웨이퍼 (혹은 기판) 맵 생성이 가능합니다.

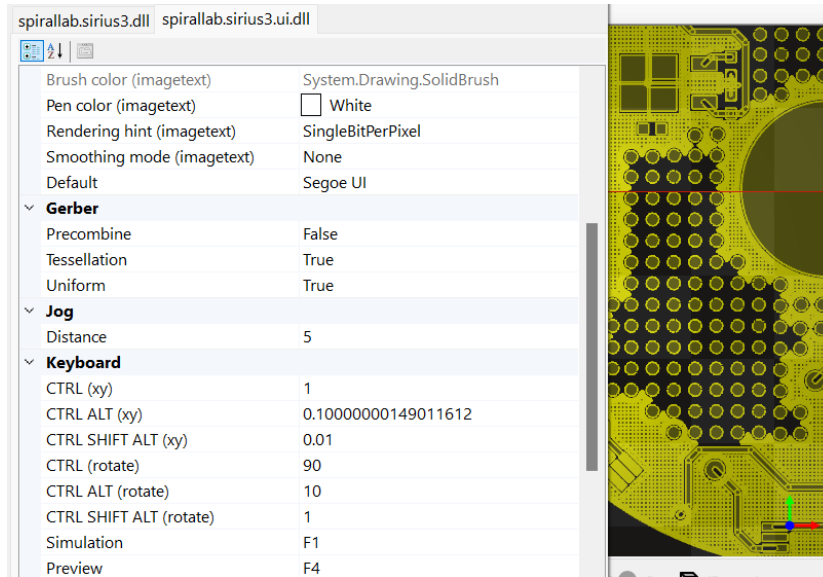


웨이퍼 (혹은 기판) 맵 페이지에서는 그중에 제일 첫(상단) 위치에 있는 맵 개체만 실제 화면에 렌더링 됩니다. 만약 다른 맵을 렌더링 시켜 편집 등의 작업이 필요하다면 좌측의 ‘1’ 버튼 (Set as First or Top) 을 눌러 위치를 이동이 가능합니다.



21. 파일 가져오기(Import)

파일 가져오기(Import)의 경우 dxf, plt, hpgl 그리고 3D 메쉬(obj, stl, ply) 및 이미지(bmp, png, jpeg 등) 파일 포맷을 제외한 모든 기타 포맷은 거버 파일로 분석을 시도하여 가져오기가 진행됩니다.



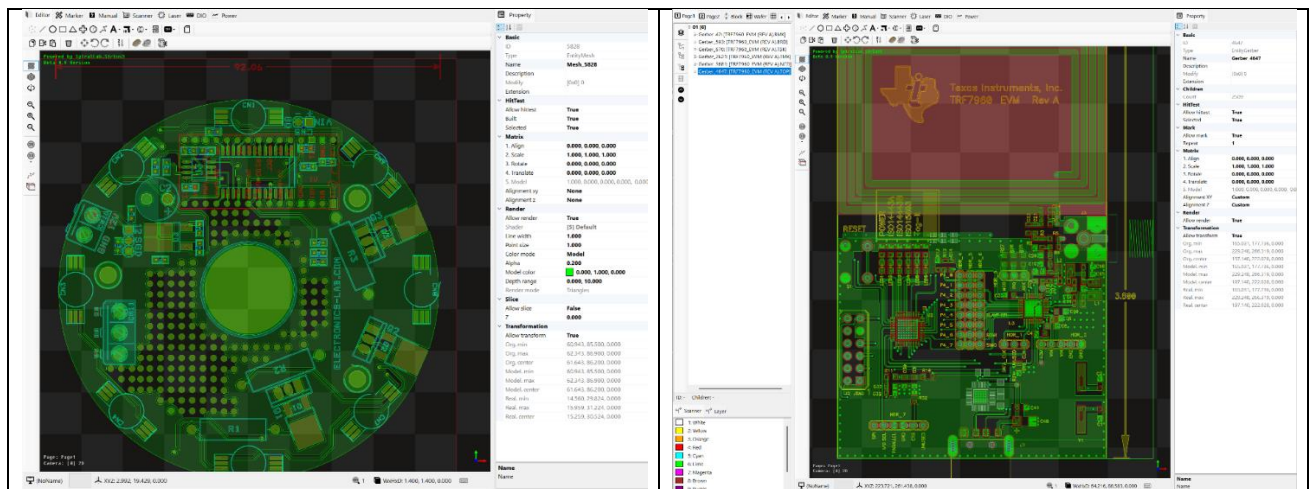
거버 파일 가져오기 시도시 다음과 같이 주요한 3가지 설정값이 사용됩니다.

Precombine: 중첩된 영역이나 중복 데이터들을 병합 처리합니다. (불필요한 데이터가 줄어드나 연산에 상당한 시간 소요됨. 기본값 False)

Tessellation: 폐 곡선 영역을 채우기(Fill) 합니다. (폐영역을 색상으로 손쉽게 구분가능 연산에 시간 소요됩니다. 기본값 False)

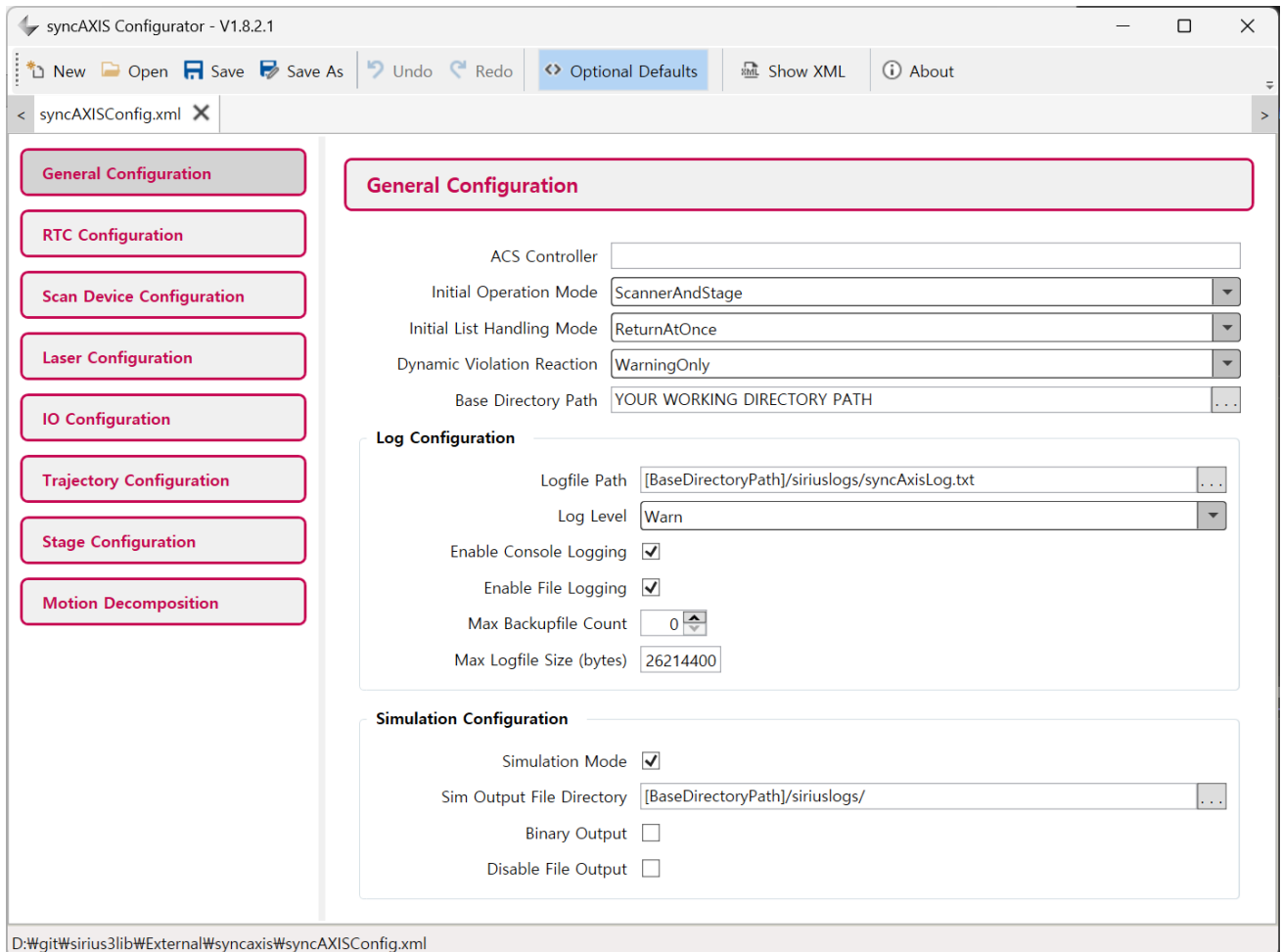
Uniform: 동일한 개체 들을 한데 모아 렌더링 속도를 향상하는 기능 (메모리 사용량 증가 하나 렌더링 속도가 향상될

니다. 기본값 True)



22. syncAXIS (XL-SCAN) 사용법

syncAXIS 를 이용해 대면적 레이저 가공 기능을 사용하기 위해서는 필수적으로 RTC6 (SCANahead) 제어기 + ExcelliSCAN(or IntelliSCAN IV) 스캔 헤드 + ACS 모션 제어기 구성이 만족되어야 합니다. 또한 SPiiPlus MMI Application Studio 설치 및 스테이지에 대한 정밀 튜닝 등을 모두 거쳐 준비가 모두 완료되었다면, syncAXIS_Configurator 프로그램을 이용해 아래와 같이 설정을 모두 확인 후 syncAXISConfig.xml 와 같은 설정 파일을 생성시킵니다. 이후 실제 구동을 위해서는 SCANLAB에서 구매한 syncAXIS 전용 라이센스 dongle이 시스템에 장착되어 있어야 합니다. (Simulation Mode이라도 dongle 필수)



이때 주의할 것은 Base Directory Path 경로 설정이 정확한지 여부, ACS 컨트롤러의 IP 주소, 스테이지의 물리적 한계(이동 범위 한계 및 속도, 가속도, 가가속도와 같은 역학 한계 등) 설정 등이며, 안전을 위해 동적 한계 침범시 처리(Dynamic Violation Reaction)시 중지 및 보고(Stop and Report)가 추천됩니다. 또한 Simulation Mode로 우선 가공 경로에 대한 다양한 검증 및 테스트 이후 실제 가공시점에 Hardware Mode로 전환하는 것이 바람직합니다

시리우스3에서는 syncAXIS 인스턴스를 다음과 같이 생성 및 초기화 합니다.

```
string configXmlFilePath = Path.Combine(Config.SyncAxisPath, "syncAXISConfig.xml");
var rtc = ScannerFactory.CreateRtc6SyncAxis(index, configXmlFilePath);
rtc.Initialize();
```

또한 syncAXIS 전용 마커인 MarkerSyncAxis 을 생성해 사용해야 합니다.

결과적으로 초기화가 성공하면 아래와 같은 로그가 출력되어야 합니다.

Time	Level	Message
10:14:39. 674	Information	GPU Vendor: ATI Technologies Inc., Renderer: AMD Radeon(TM) Graphics, OpenGL Version: 4.6.0 Compatibility Profile Conte
10:14:39. 675	Information	Free GPU Texture Memory: 384MB
10:14:44. 529	Information	syncaxis [0]: handle id= 1. xml= D:\git\sirius3\bin\net481\syncaxis\syncAXISConfig.xml. v1.8.2
10:14:44. 553	Information	syncaxis [0]: motion mode has changed to Unfollow

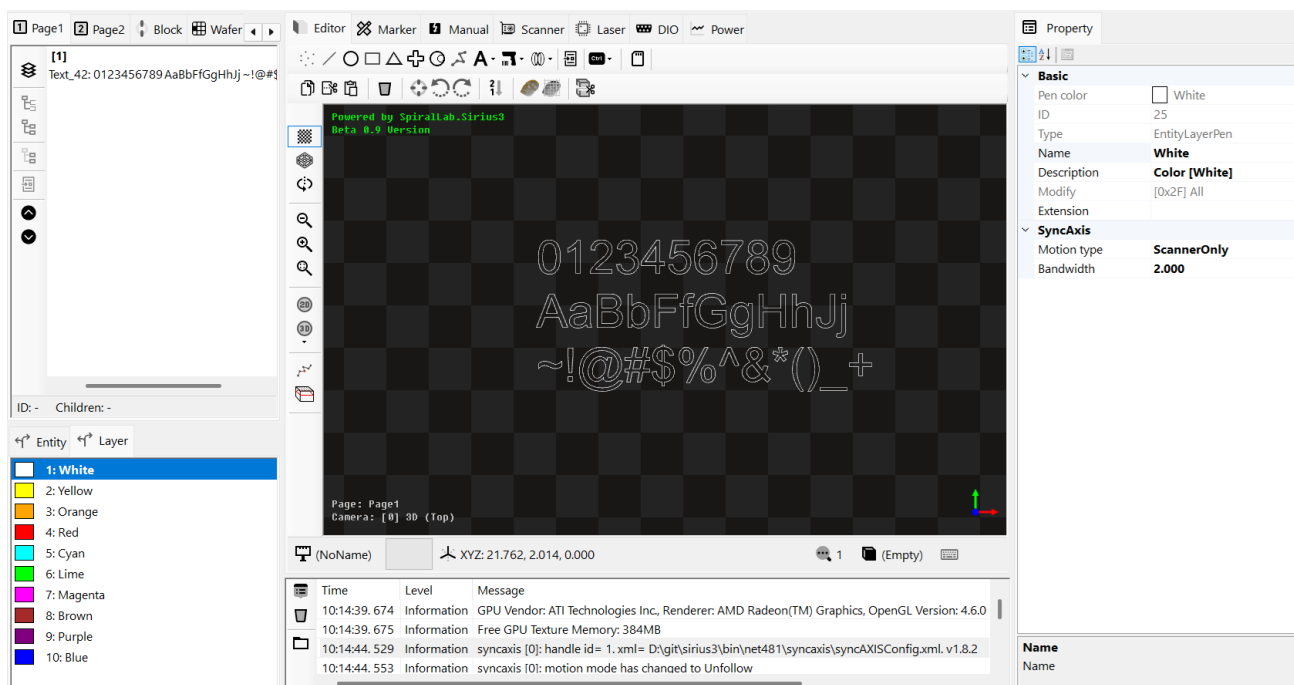
(참고1) 보정 파일은 xml 파일내에서 관리되므로 CtlLoadCorrectionFile 등의 함수들은 사용하지 않습니다.

(참고2) 디지털 입출력 포트는 확장1 포트(Extension1) 만 사용합니다.

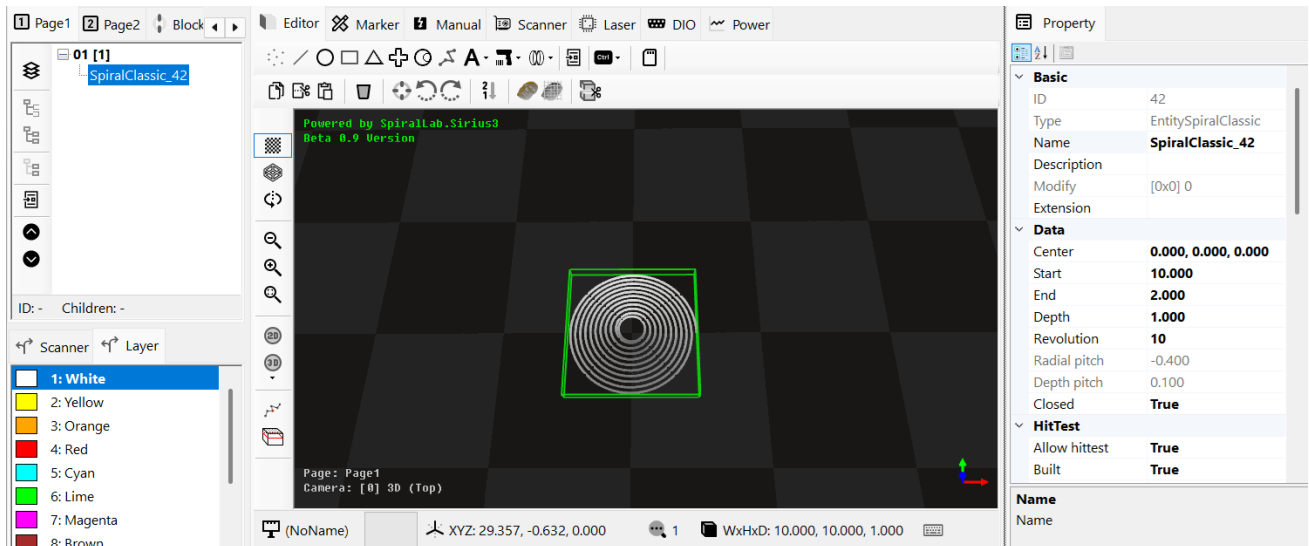
(참고3) 모든 3D 좌표(Z 좌표)는 Z=0 mm 으로 투영됩니다.

생성된 Rtc6SyncAxis를 편집기의 스캐너(IScanner) 항목에 지정하면 해당 제어기에 맞도록 다양한 속성들이 변경됩니다. 예를 들어 레이어 펜(EntityLayerPen)을 선택했을 경우 아래와 같이 syncAXIS 전용의 모션 타입, LPF 주파수 항목이 표시됩니다.

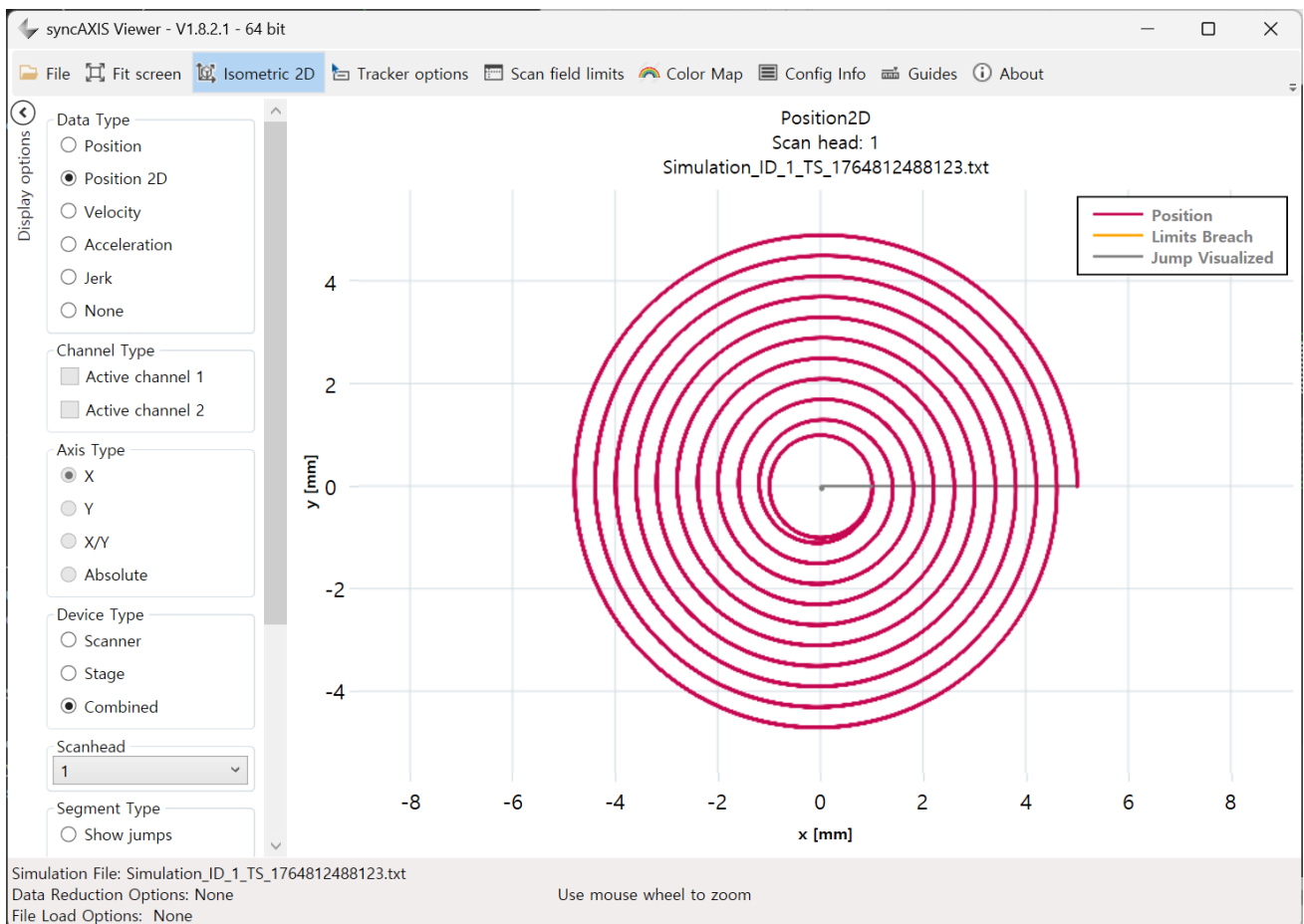
(참고) 개체 펜(EntityPen)의 일부 속성들도 syncAXIS 기능을 위해 출력이 변경됩니다.



아래에서는 간단한 나선 모양을 Stage + Scanner 모션 타입으로 시뮬레이션 가공을 진행한 결과 입니다.



시뮬레이션 모드(Simulation Mode)에서는 100KHz 샘플링 주기로, 스캐너와 스테이지의 경로(Trajectory)가 모두 수집되어 Sim Output File Directory에 기록됩니다. 이 출력 파일은 syncAXIS_Viewer 프로그램에 자동으로 전달되며, 시간 위치 그래프 분석, 가감속도 분석, 범위를 침범했는지 여부 등 다양한 동역학 항목을 분석할 수 있습니다.



또한 마커(Marker) 탭의 이력(History) 항목을 살펴보면, 좀더 구체적인 분석 정보가 제공됩니다.

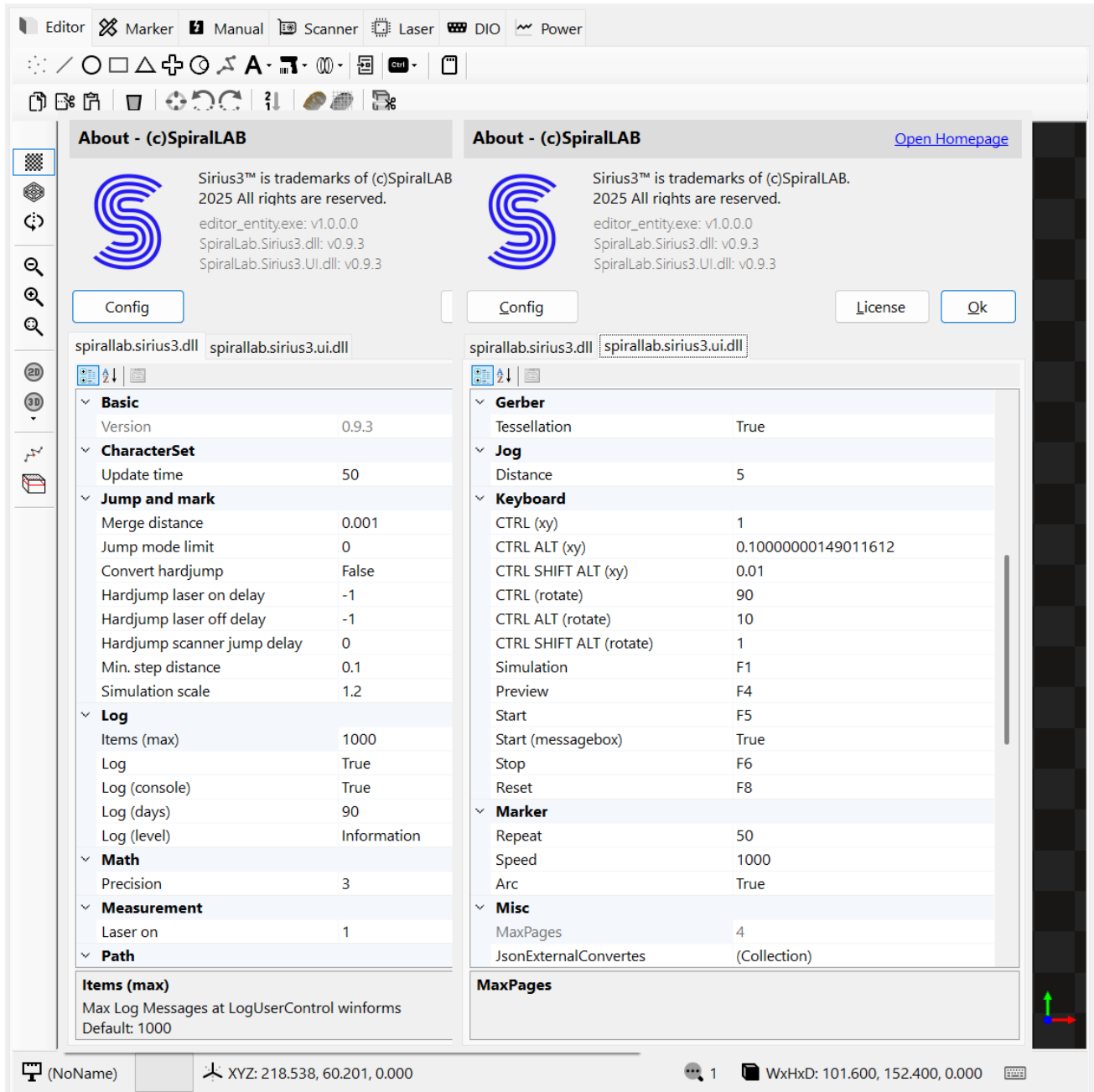
History	JobSyncAxis[] Array
[0]	ID: 1, Success, 10:41:28, 3.921s
ID	1
Name	
Description	
Bandwidth	2
Motion type	StageAndScanner
Config.xml	D:\git\sirius3lib\bin\net481\syncaxis\syncAXISCor
Relative coordinate system	False
Calculation	InProgress
Transfer	LoadedEnough
Execution	Finished
Result	Success
List counts	1986
Started	10:41:28.108
Ended	10:41:32.029
Execution	00:03.920
RTC	2.25773
Characteristic	
Scanner	
> Position Max	3.33091130153022, 3.22174908542804
> Velocity Max	107.461853397606, 100.533242138023
> Acc Max	5793373.65365248, 7674742.9318269
> Position Max + Laser On	3.33091130153022, 3.22174908542804
> Min/Max X Position	-3.222 / 3.331
> Min/Max Y Position	-3.218 / 3.222
> Min/Max X Position + Laser On	-3.222 / 3.331
> Min/Max Y Position + Laser On	-3.218 / 3.222
Stage	
> Position Max	1.73491809911825, 2.29295437125394
> Velocity Max	33.3771119446123, 36.4651926179738
> Acc Max	689.723735902703, 703.748499475409
> Jerk Max	50000.0145464696, 50000.031897035
> Position Max + Laser On	1.69436003415024, 2.29295437125394
> Min/Max X Position	-1.694 / 1.735
> Min/Max Y Position	-1.567 / 2.293
> Min/Max X Position + Laser On	-1.694 / 1.682
> Min/Max Y Position + Laser On	-1.567 / 2.293
Motion MicroSteps	225773
Min/Max Mark Speed	98.581 / 100.000
Min	98.580899312304467
Max	100.00000000000001
Inserted Skywritings	0
Active channel count	0
Active channel	
Head count	1
Head1 offset	0.000, 0.000, 0.000, 0.000
Head2 offset	0.000, 0.000, 0.000, 0.000
Head3 offset	0.000, 0.000, 0.000, 0.000
Head4 offset	0.000, 0.000, 0.000, 0.000
Scanner	Pos: 12.337 %
Position	12.337, 11.932
X	12.337
Y	11.932
Stage count	1
Stage1	Pos: 1.529%, Acc: 7.037%, Jerk: 50.000%
Position	1.157, 1.529
X	1.157
Y	1.529
Position	
Utilized Scanner X,Y Position Area (%)	

스캐너 헤드가 이동한 최대 좌표, 속도, 가속도 정보 및 동적 한계를 어느 정보 비율까지 사용했는지 등의 정보

스테이지가 이동한 최대 좌표, 속도, 가속도, 가가속도 정보 및 동적 한계를 어느 정보 비율까지 사용했는지 등의 정보

23. 환경 설정(Config)

라이브러리의 환경 설정값을 통해 다양한 기능을 처리할 수 있습니다. `spirallab.sirius3.dll` 모듈에서 제공되는 설정값은 `SpiralLab.Sirius3.Config` 으로 편집이 가능하며, `spirallab.sirius3.ui.dll` 모듈에서 제공되는 설정값은 `SpiralLab.Sirius3.UI.Config` 으로 편집이 가능합니다. 또한 편집기에서는 About -> Config 메뉴를 통해 시각화된 상태로 설정이 가능합니다. Config 설정값에 대한 자세한 설명은 별도의 개발자 매뉴얼을 참고해 주시기 바랍니다.



24. 편집기 단축키(Shortcut)

CTRL + C: 선택된 개체(들)을 클립보드로 복사

CTRL + X: 선택된 개체(들)을 클립보드로 자르기

CTRL + V: 클립보드의 개체(들)을 붙여넣기

CTRL + Delete: 선택된 개체(들)을 삭제

CTRL + H: 선택된 개체(들)을 X,Y 원점으로 정렬

CTRL + SHIFT + H: 선택된 개체(들)을 X,Y,Z 원점으로 정렬

CTRL + F: 선택된 개체들이 있을 경우 개체 중심으로 카메라 맞춤, 없을 경우 활성화된 페이지의 개체들을 중심으로 카메라 맞춤

CTRL + E: 다음 카메라로 전환

CTRL + Q: 이전 카메라로 전환

CTRL + R : 선택된 개체들에 대해 화면에 렌더링을 허용할지 여부(`IRenderable.IsAllowToRender`)를 토글

CTRL + M : 선택된 개체들에 대해 가공을 허용할지 여부(`IMarkerable.IsAllowToMark`)를 토글

CTRL + 방향키: 선택된 개체(들)을 X(혹은 Y) 방향으로 1mm 이동

CTRL + ALT + 방향키: 선택된 개체(들)을 X(혹은 Y) 방향으로 0.1mm 이동

CTRL + SHIFT + ALT + 방향키: 선택된 개체(들)을 X(혹은 Y) 방향으로 0.01mm 이동

CTRL + [또는] : 선택된 개체(들)을 시계(혹은 반시계) 방향으로 90도 회전

CTRL + ALT + [또는] : 선택된 개체(들)을 시계(혹은 반시계) 방향으로 10도 회전

CTRL + SHIFT + [또는] : 선택된 개체(들)을 시계(혹은 반시계) 방향으로 1도 회전

F1: 선택된 개체(들)에 대해 시뮬레이션 가공 시작

F4: 선택된 개체(들)에 대해 미리보기 가공 시작

(참고) ILaser 가 IGuideControl 지원시

F5: 마커(IMarker) 가공 시작

(참고) 전체 혹은 선택된 개체(들)만 할지 여부는 마커 대상(Mark target) 옵션 참고

F6: 마커(IMarker) 가공 중지

F8: 마커(IMarker) 리셋

ESC: 시뮬레이션 가공 중지, 선택 개체(들) 드래그(Drag)시 취소

